DiceMania

Universidade Técnica do Porto - UTP

Licenciatura de Engenharia de Telecomunicações e Informática

Unidade Curricular - Desenvolvimento de Software e de Sistemas Móveis

Professor Paulo Baltarejo

Professor Carlos Freitas

Shantall Camacho - 1251460
João Bessa - 1251089

Resumo: O presente documento descreve todas as fases de planeamento, conceção e desenvolvimento do DiceMania, uma aplicação móvel interativa. O principal objetivo deste projeto consistiu em aplicar práticas de engenharia de software na criação de um jogo de perguntas e respostas (trivia), acompanhando todo o ciclo de vida do seu desenvolvimento.

Ao longo deste relatório, é documentado o processo desde o seu início funcional até à sua concretização técnica. Isto inclui o levantamento de requisitos, a modelação do sistema através de diagramas UML, e o desenho das arquiteturas que suportam a aplicação. Por fim, abordam-se os métodos de implementação escolhidos e os testes realizados para garantir a qualidade e o correto funcionamento do projeto.

Palavras-Chave: Aplicação Móvel, Engenharia de Software, Jogo de Perguntas e Respostas, Modelação UML, Arquitetura de Software, Testes de Software

Índice

1. Contexto e Problema da Aplicação
..... Página 7

2. Modelo de Domínio
..... Página 9

2.1. Entidades, Conceitos e Regras do Domínio
..... Página 10
2.1.1. Entidades e Conceitos Centrais
..... Página 10
2.1.2. Partida e Estado do Jogo
..... Página 11
2.1.3. Regras de Negócio
..... Página 11
2.1.4. Serviços Externos de Apoio
..... Página 12

2.2. Explicação das Associações e Multiplicidades do Domínio ..... Página 13
2.2.1. Associações Estruturais
..... Página 13
2.2.2. Relações Comportamentais e Serviços Externos
..... Página 14

3. Análise de Requisitos
..... Página 15
3.1. Requisitos Funcionais (RF)
..... Página 16

3.2. Requisitos Não Funcionais (RNF) — Modelo FURPS+ ..... Página 19

4. Casos de Uso ..... Página 23
   4.1. Especificação dos Casos de Uso ..... Página 23

5. Diagramas de Sequência do Sistema (SSD) ..... Página 28
   5.1. UC01 — Registar Utilizador ..... Página 29
   5.2. UC02 — Jogar Partida ..... Página 30
   5.3. UC03 — Consultar Leaderboard ..... Página 33
   5.4. UC04 — Ativar Ferramentas de Depuração (Debug Mode) ..... Página 34
   5.5. UC05 — Consultar Perfil e Estatísticas ..... Página 35

6. Arquiteturas ..... Página 37
   6.1. Arquitetura Física ..... Página 38
   6.2. Arquitetura Lógica ..... Página 39

7. Diagrama de Classes ..... Página 42
   7.1. Entidades de Dados

..... Página 42
7.2. Controlador e Estados da Partida
..... Página 44
7.3. Sensor Físico e Classes Utilitárias
..... Página 46
7.4. Contratos e Repositórios
..... Página 48
7.5. Rede, DTOs e Persistência Local
..... Página 50
7.6. Activities, Vistas e Serviços de Apoio
..... Página 52

8. Diagramas de Sequência
..... Página 54

8.1. Elementos Essenciais dos Diagramas
……..... Página 54
8.1.1. UC01 — Registar Jogador
..... Página 55
8.1.2. UC02 — Jogar Partida
..... Página 57
8.1.3. UC03 — Consultar o Leaderboard
..... Página 67
8.1.4. UC04 — Ativar ou Desativar o Modo Deus
..... Página 70
8.1.5. UC05 — Consultar Perfil, Definições e Terminar Sessão ..... Página 71

9. Interface de Utilizador — Mock-ups (UI)
..... Página 73
9.1. Login
..... Página 74
9.2. Home

..... Página 76
9.3. Lançamento dos Dados
..... Página 78
9.4. Resumo da Ronda
..... Página 80
9.5. Pergunta
..... Página 82
9.6. Leaderboard
..... Página 84

10. Implementação
..... Página 86

10.1. Deteção do Abanão do Dispositivo
..... Página 86
10.2. Conversão das Perguntas Recebidas da OpenTDB
..... Página 89
10.3. Envio Assíncrono de Resultados para o Supabase
..... Página 92

11. Testes ..... Página 96

11.1. Testes Unitários (Unit Tests)
..... Página 96

11.1.1. Validação do Anti-Batota (AntiCheatTimer)
..... Página 96
11.1.2. Validação do Hardware Matemático
(AndroidShakeSensor) ..... Página 98
11.1.3. Teste do Mapeamento do DTO e da Permutação
..... Página 100

11.2. Testes Funcionais / End-to-End (E2E Tests)
..... Página 107

11.2.1. UC01
..... Página 111
11.2.2. UC02
.....Página 113
11.2.3. UC03
..... Página 119
11.2.4. UC04
..... Página 121
11.2.5. UC05
.....Página 125

12. Conclusão ..... Página 127

12.1. Estado do Sistema
..... Página 127
12.2. Desafios e Dificuldades Técnicas
..... Página 128
12.3. Considerações Finais
..... Página 129

13. Glossário
.....
Páginas 130–133

1. Contexto e Problema da Aplicação

No âmbito da unidade curricular de Desenvolvimento de Software e Sistemas Móveis (DSSMV), lecionada no 2º ano da Licenciatura em Engenharia de Telecomunicações e Informática (LETI) do ISEP, foi proposta a conceção e implementação de uma aplicação nativa para o ecossistema Android, suportada pela linguagem Java no ambiente de desenvolvimento IntelliJ IDEA.

Para responder a este desafio técnico, concebeu-se o DiceMania, uma aplicação móvel interativa focada no entretenimento através de um jogo dinâmico de conhecimentos gerais (trivia). O problema central que este projeto visa solucionar é a monotonia, a estaticidade e a falta de envolvimento físico tipicamente presentes nas aplicações de quiztradicionais do mercado.

Para colmatar esta lacuna, o DiceMania introduz uma componente háptica e multissensorial. A mecânica principal da aplicação exige que o utilizador agite fisicamente o dispositivo móvel para simular o "lançamento de dados". Esta ação cinemática é capturada através dos sensores de hardware do smartphone (nomeadamente o Acelerómetro) e atua como o gatilho principal para o ciclo de vida do jogo.

Para alimentar as sessões de jogo, o sistema foi desenhado com uma arquitetura distribuída: o Frontend (a aplicação Android em si) consome a REST API externa OpenTDB (Open Trivia Database) para a obtenção aleatória de perguntas em tempo real, e integra a plataforma Supabase (como infraestrutura de base de dados relacional e Backend-as-a-Service) para alojamento e persistência das métricas e pontuações globais dos jogadores.

Para garantir uma experiência de jogo fluida, imersiva e tolerante a falhas, a aplicação operacionaliza as seguintes funcionalidades core:

- Registo e Sessão Contínua: Criação de perfil através de um Nickname com geração de um ID universal. A sessão é persistida localmente (recorrendo a SharedPreferences), eliminando a necessidade de logins subsequentes.
- Interação Física e Feedback: Deteção matemática do abalo mecânico para descarregar pacotes de perguntas,suportada por feedback áudio-visual e vibração.

- Mecânica Temporal e Progressão: Resolução de blocos de perguntas cronometradas (10 segundos), validadas por um algoritmo de pontuação que premeia tempos de resposta reduzidos e penaliza quebras na série de vitórias (Streak).
- Competitividade Síncrona: Gravação transparente da pontuação final na tabela PostgreSQL do Supabase e consulta de uma tabela classificativa global (Leaderboard).
- Auditoria e Qualidade: Inclusão de um menu de depuração oculto (Debug Menu) acessível aos promotores, permitindo manipular os estados transitórios do jogo e validar cenários de erro e respostas da API.

2. Modelo de Domínio

O Modelo de Domínio é uma representação concetual que mapeia as entidades do mundo real e as regras de negócio centrais do sistema numa estrutura de Programação Orientada a Objetos (POO). Na Engenharia de Software, o modelo de domínio desempenha um papel crítico: atua como a "espinha dorsal" da aplicação, estabelecendo o vocabulário comum entre a análise de requisitos e a implementação técnica em código Java.

A sua importância reside nos seguintes fatores:

1. Rigor Estrutural: Define explicitamente as responsabilidades de cada classe, os seus atributos internos, tipos de dados e métodos essenciais, evitando o surgimento de classes "deusas" (God Objects) que acumulam responsabilidades excessivas.

2. Definição de Fronteiras e Contratos: Separa as entidades de dados puras dos controladores de estado, das ferramentas utilitárias e dos contratos de infraestrutura (interfaces de rede e persistência).

3. Mapeamento de Relações e Multiplicidades: Especifica rigorosamente como as entidades se associam (ex: quantas perguntas pertencem a uma sessão) e como dependem umas das outras em tempo de execução, prevenindo erros de alocação de memória ou inconsistências de dados.

O diagrama estrutural do Modelo de Domínio do DiceMania divide-se em quatro módulos fundamentais, complementados pelos componentes de infraestrutura de rede integrados para o consumo da API externa OpenTDB, como podemos verificar na figura 1.

2.1. Entidades, Conceitos e Regras do Domínio

O Modelo de Domínio do DiceMania representa os conceitos do jogo e as regras que os relacionam. As caixas descrevem o que cada conceito representa e como participa numa partida, sem detalhar atributos ou métodos de implementação.

2.1.1. Entidades e Conceitos Centrais

Estes conceitos representam o jogador e os desafios que constituem o jogo.

- Jogador: representa a pessoa que participa no jogo. Mantém a sua identidade, o nickname e os resultados associados, incluindo a pontuação e o streak máximo registados.
- Pergunta: representa um desafio apresentado ao jogador. Inclui um enunciado, uma resposta correta e as opções de resposta adequadas ao seu formato.
- Categoria: identifica o tema de uma pergunta. No jogo atual, os temas possíveis são ART, SCIENCE, HISTORY, CINEMA, VIDEOGAMES e MUSIC.

- Formato da pergunta: determina como o jogador responde ao desafio. Os formatos possíveis são escolha múltipla (MULTIPLE_CHOICE) e verdadeiro ou falso (TRUE_FALSE).
- Bloco atual: representa o conjunto de perguntas associado a um lançamento dos dados. Cada bloco carregado contém exatamente três perguntas.

2.1.2. Partida e Estado do Jogo

Este conjunto de conceitos representa o progresso e o fluxo temporal de uma partida.

- Partida: representa uma sessão de jogo de um jogador. Coordena o progresso, mantém a pontuação e o streak, processa as respostas e determina quando o jogador deve continuar, lançar os dados novamente ou terminar.
- Estado da partida: representa a fase atual do jogo. A partida encontra-se num único estado de cada vez: à espera de um lançamento, a carregar perguntas, a apresentar o resumo entre perguntas, a aguardar uma resposta ou terminada.
- Lançamento dos dados: representa o início de uma nova ronda. O abanão do telemóvel inicia o lançamento; o resultado do primeiro dado escolhe a categoria e o resultado do segundo escolhe o formato da pergunta.

Uma partida normal tem quinze perguntas, organizadas em cinco blocos de três. No modo Deus, a partida tem apenas um bloco de três perguntas, para facilitar os testes.

2.1.3. Regras de Negócio

Estas regras determinam como a partida avalia as respostas e aplica as condições do jogo.

- Regra de pontuação: calcula os pontos atribuídos quando o jogador responde corretamente dentro do tempo. A pontuação considera o tempo restante e o streak atual. No código, o cálculo é 10 + (tempo restante × 15) + (streak × 25); se o tempo terminar, não são atribuídos pontos.
- Regra de integridade temporal: aplica uma penalização quando o jogador sai da aplicação durante uma pergunta e regressa depois de mais de dez segundos. A regra só é avaliada quando a partida está à espera de uma resposta.
- Modo de jogo: define a duração e o comportamento da partida. O modo normal tem quinze perguntas. O modo Deus é uma variante de teste com três perguntas e valores de pontuação e streak predefinidos; os resultados desse modo não são enviados para o leaderboard.

O ThemeRandomizer existe no código, mas não participa no fluxo atual: a categoria e o formato são escolhidos pelo resultado dos dados. O DebugManager gere o mecanismo técnico que ativa ou desativa o modo Deus; no modelo conceptual, o conceito relevante é o Modo de jogo.

2.1.4. Serviços Externos de Apoio

Estes serviços apoiam o jogo, mas não fazem parte das entidades centrais do domínio.

- OpenTDB: fonte externa de perguntas. A aplicação pede três desafios de acordo com a categoria e o formato escolhidos e adapta as respostas recebidas para o jogo.
- Leaderboard global: recebe a pontuação e o streak máximo das partidas normais e disponibiliza a classificação dos dez melhores jogadores. O modo Deus não envia resultados para esta classificação.

2.2. Explicação das Associações e Multiplicidades do Domínio

As associações seguintes representam as relações entre os conceitos do jogo. As multiplicidades descrevem quantas instâncias podem participar em cada relação.

2.2.1. Associações Estruturais

- Jogador — Partida
  Multiplicidade: Jogador 1 — Partida 0..*
  Cada partida pertence a exatamente um jogador. Um jogador pode participar em várias partidas ao longo do tempo.
- Partida — Bloco atual
  Multiplicidade: Partida 1 — Bloco atual 0..1
  Uma partida pode ainda não ter um bloco carregado, por exemplo antes do primeiro lançamento. Quando existe um bloco atual, a partida mantém um único bloco de cada vez.
- Bloco atual — Pergunta
  Multiplicidade: Bloco 1 — Pergunta 3
  Cada bloco carregado contém exatamente três perguntas. A partida apresenta essas perguntas uma de cada vez.
- Partida — Estado da partida
  Multiplicidade: Partida 1 — Estado 1 em cada instante
  A partida encontra-se num único estado de cada vez. O estado muda conforme o jogador lança os dados, responde, termina o tempo ou conclui a partida.
- Pergunta — Categoria
  Multiplicidade: Pergunta 1 — Categoria 1
  Cada pergunta pertence a uma única categoria.

- Pergunta — Formato da pergunta
  Multiplicidade: Pergunta 1 — Formato 1
  Cada pergunta tem um único formato de resposta.

2.2.2. Relações Comportamentais e Serviços Externos

- Lançamento dos dados — Categoria e formato
  O primeiro dado seleciona uma das seis categorias. O segundo seleciona o formato: os resultados de um a três correspondem a verdadeiro ou falso; os resultados de quatro a seis correspondem a escolha múltipla. Os dois dados podem apresentar o mesmo número.
- Partida — Regra de pontuação
  Quando o jogador responde corretamente antes de o tempo terminar, a partida aplica a regra de pontuação usando o tempo restante e o streak atual. Uma resposta errada ou um timeout quebra o streak.
- Partida — Regra de integridade temporal
  Se o jogador sair da aplicação enquanto responde a uma pergunta, a partida regista esse momento. Ao regressar, se tiverem passado mais de dez segundos, a resposta é tratada como timeout.
- OpenTDB — Bloco atual
  O OpenTDB fornece as perguntas solicitadas para a categoria e o formato selecionados. A aplicação prepara as respostas e entrega à partida um bloco de três perguntas.
- Partida — Leaderboard global
  No fim de uma partida normal, são enviados o jogador, a pontuação final e o streak máximo. O leaderboard guarda os resultados e devolve a classificação dos dez melhores jogadores. Os resultados do modo Deus são excluídos desta submissão.

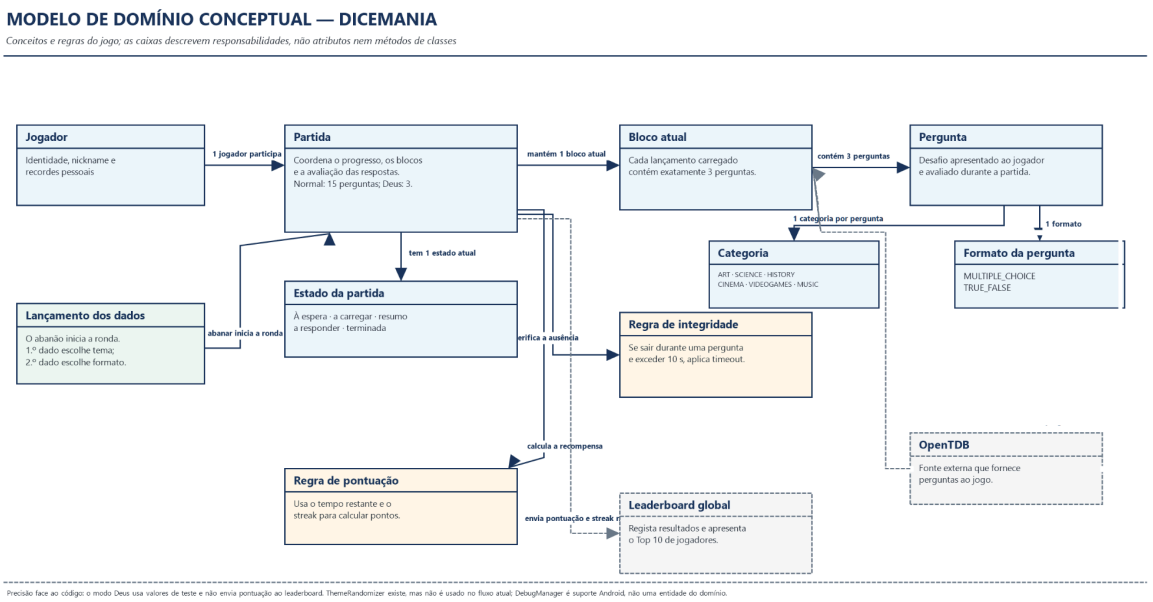


Figura 1 - Modelo de Domínio

3. Análise de Requisitos

A fase de levantamento e análise de requisitos é um dos pilares fundamentais no ciclo de vida da Engenharia de Software. É nesta etapa que se estabelece o "contrato" técnico e funcional do sistema, definindo com clareza o âmbito do projeto antes de se iniciar o desenvolvimento da base de código. Para o projeto DiceMania, a documentação rigorosa dos requisitos foi crucial não apenas para guiar a implementação da arquitetura lógica e física, mas também para garantir que a visão inicial, criar uma experiência interativa, multissensorial e à prova de falhas, é traduzida de forma mensurável em funcionalidades concretas.

De forma a organizar a especificação do sistema, os requisitos foram divididos em duas categorias clássicas: Requisitos Funcionais (que definem o comportamento do sistema) e Requisitos Não Funcionais (que definem os

atributos de qualidade e restrições técnicas do sistema), sendo estes últimos categorizados através do modelo padrão FURPS+.

3.1. Requisitos Funcionais (RF):

Os Requisitos Funcionais (RF) descrevem o que o sistema deve fazer. Eles representam as ações específicas, o processamento de dados e as interações diretas que a aplicação tem de suportar para satisfazer as necessidades do jogador. No caso do DiceMania, englobam desde a mecânica principal do jogo até à comunicação com as APIs externas.

- RF01 — Registo local do jogador: A aplicação deve permitir inserir um nickname obrigatório, com um máximo de nove caracteres. Deve gerar localmente um identificador único, guardar o identificador e o nickname no `SharedPreferences` privado e recuperar esses dados em utilizações posteriores. Este processo identifica o jogador no dispositivo; não corresponde a uma autenticação numa conta remota.
- RF02 — Início de partida e deteção do abanão: Ao selecionar Let's Play, a aplicação deve abrir o ecrã de jogo e ativar o acelerómetro. Um abanão deve ser aceite quando a força calculada ultrapassar `2.7 G`, respeitando um intervalo mínimo de 500 milissegundos entre eventos. A sessão só deve aceitar o lançamento quando estiver no estado WAITING_FOR_ROLL.
- RF03 — Sorteio de categoria e formato: Depois de aceitar um abanão, a aplicação deve gerar dois resultados numéricos aleatórios. O primeiro seleciona uma das seis categorias — Arte, Ciência, História, Cinema, Videojogos ou Música. O segundo seleciona o formato: valores de 1 a 3 correspondem a Verdadeiro/Falso e valores de 4 a 6 correspondem a Escolha Múltipla. Os números são gerados pelo código; a animação Lottie representa visualmente o lançamento.
- RF04 — Obtenção de perguntas: A aplicação deve solicitar à OpenTDB um bloco de três perguntas compatíveis com a categoria e o formato selecionados. O pedido deve decorrer enquanto a animação dos dados está a ser apresentada. A aplicação só deve abrir o resumo quando tiver recebido três perguntas e as duas animações tiverem

terminado. Se o carregamento falhar, deve informar o jogador e permitir uma nova tentativa. Atualmente, a atividade apresenta uma mensagem e volta a aguardar um novo abanão.

- RF05 — Baralhamento das opções: Antes de apresentar uma pergunta, a aplicação deve juntar a resposta correta às respostas incorretas e baralhar a lista. Deve manter todas as opções recebidas e guardar separadamente a resposta correta. O baralhamento altera a ordem das opções, mas não garante que a resposta correta fique sempre numa posição diferente em cada execução.
- RF06 — Temporização e feedback multissensorial: O resumo dos dados deve permanecer visível durante sete segundos. Cada pergunta deve disponibilizar 15 segundos para resposta. Nos últimos três segundos, se os efeitos estiverem ativos, a aplicação deve emitir um bip e uma vibração curta por segundo. Quando o tempo chegar a zero, deve emitir um alarme e uma vibração mais longa. Depois de uma resposta ou timeout, deve manter o feedback visível durante três segundos antes de avançar.
- RF07 — Validação e feedback da resposta: A aplicação deve comparar a opção selecionada com a resposta correta. Depois da resposta ou do timeout, deve destacar a opção correta a verde e as restantes opções a vermelho. As opções devem ficar desativadas durante o feedback, impedindo uma segunda resposta à mesma pergunta.
- RF08 — Cálculo da pontuação e do streak: Uma resposta correta dentro do tempo deve aumentar o streak e a pontuação. A fórmula atual é: 10 pontos base, mais 15 pontos por cada segundo restante, mais 25 pontos por cada resposta correta consecutiva. Uma resposta errada ou um timeout deve reiniciar o streak. A aplicação deve acompanhar também o maior streak alcançado na partida.
- RF09 — Registo de resultados: No fim de uma partida normal, a aplicação deve guardar localmente, por ID de jogador, a melhor pontuação e o melhor streak. Deve também enviar para o Supabase o ID, o nickname, a pontuação final e o streak máximo. O registo remoto deve ser identificado pelo ID e atualizado sem criar linhas

duplicadas para o mesmo jogador. Os valores artificiais do Modo Deus não devem ser tratados como resultados normais.

- RF10 — Consulta do leaderboard: A aplicação deve apresentar o Top 10 geral, ordenado por pontuação decrescente, seguido do streak máximo e do nickname para desempate. O ranking é all time, não semanal. Deve estar acessível pelas definições e depois de uma partida. No fim do jogo, deve indicar se o jogador está no Top 10 e apresentar a mensagem correspondente. Se o pedido falhar, deve mostrar um erro e permitir tentar novamente.
- RF11 — Apresentação da interface: A aplicação deve apresentar os ecrãs de login, Home, lançamento dos dados, resumo, pergunta e leaderboard com uma identidade visual consistente. O login e o ecrã Home utilizam fundo branco; os ecrãs de jogo utilizam azul escuro. Os botões principais usam azul escuro e texto branco, e o feedback de resposta utiliza verde e vermelho. A aplicação implementa uma paleta fixa; não disponibiliza atualmente a seleção de várias paletas.
- RF12 — Resumo do lançamento: Depois de cada lançamento, o resumo deve apresentar o número e o tipo/categoria associados a cada dado, o progresso da partida — pergunta atual e total —, a pontuação atual e o streak. Este ecrã deve conduzir à pergunta seguinte ao fim dos sete segundos.
- RF13 — Perfil e definições: O painel Settings deve mostrar o nickname, a pontuação máxima e o streak máximo guardados localmente para o jogador atual. Deve permitir ativar ou desativar separadamente os efeitos sonoros, a música ambiente e a vibração. Deve disponibilizar também o acesso ao leaderboard.
- RF14 — Terminar sessão: A aplicação deve pedir confirmação antes de terminar a sessão local. Após confirmação, deve remover o ID e o nickname guardados no dispositivo e regressar ao ecrã de login. Os recordes locais, as definições de áudio e vibração e o registo remoto do jogador devem ser preservados.
- RF15 — Modo Deus para testes: Cinco toques rápidos consecutivos no logótipo devem alternar o Modo Deus. A aplicação deve apresentar uma mensagem ao ativar ou desativar o modo. Uma partida neste

modo deve ter três perguntas e utilizar valores de teste para pontuação e streak, sem guardar ou submeter esses valores como recordes reais.

- RF16 — Mensagens de inatividade: Se o jogador permanecer no ecrã dos dados sem os abanar durante um período definido, a aplicação poderá apresentar uma mensagem motivacional. O código atual não contém esse comportamento.

3.2. Requisitos Não Funcionais (RNF) - Modelo FURPS+:

Enquanto os requisitos funcionais ditam o que o sistema faz, os Requisitos Não Funcionais (RNF) determinam como ou com que qualidade o sistema o faz. Estes são vitais para a aceitação do produto, pois garantem que a aplicação é rápida, segura, imersiva e tolerante a falhas. Para garantir uma análise exaustiva, adotou-se o modelo de classificação FURPS+ (*Usability, Reliability, Performance, Supportability* + *Constraints*), omitindo o "F" (*Functionality*) por já estar coberto na secção anterior.

Usabilidade (Usability):

A usabilidade foca-se na experiência do utilizador e na facilidade de interação com a aplicação.

- RNF01 — Persistência local: O ID e o nickname devem manter-se entre aberturas da aplicação, até o jogador terminar a sessão. Os recordes e as definições devem manter-se associados ao jogador e ao dispositivo.
- RNF02 — Experiência multissensorial: A aplicação deve disponibilizar música ambiente, som dos dados a rolar, som de resposta correta ou errada, bips no fim da contagem e um alarme no timeout. Deve disponibilizar também vibração nos últimos segundos e no timeout. O jogador deve poder desligar efeitos sonoros, música e vibração separadamente. A vibração não é contínua durante o abanão nem acompanha atualmente as respostas certas ou erradas.

- RNF03 — Ritmo dos ecrãs: O resumo deve durar sete segundos, o tempo máximo de resposta deve ser 15 segundos e o feedback da resposta deve permanecer visível durante três segundos.
- RNF04 — Legibilidade e acessibilidade: Os textos, botões e contrastes devem continuar legíveis em diferentes dimensões de ecrã e com as definições de tamanho de texto do dispositivo. Os controlos interativos devem ter áreas de toque adequadas e as imagens funcionais devem possuir descrições acessíveis.

Confiabilidade / Fiabilidade (Reliability):

A fiabilidade diz respeito à robustez do sistema, ao tratamento de exceções e à sua capacidade de recuperar de falhas (como quebras de rede) ou tentativas de uso indevido.

- RNF05 — Comportamento em segundo plano: Se a aplicação sair de primeiro plano durante uma pergunta, deve registar o instante da saída. Ao regressar, se o tempo em segundo plano ultrapassar dez segundos, a pergunta deve ser tratada como timeout. O estado da partida não é guardado como sessão recuperável; se o processo da aplicação terminar, a partida em memória não é retomada.
- RNF06 — Falhas de rede: A aplicação não deve encerrar inesperadamente quando a OpenTDB ou o Supabase não estiverem disponíveis. A falha ao carregar perguntas deve apresentar uma mensagem e permitir um novo abanão. A falha no leaderboard deve apresentar uma mensagem e disponibilizar o botão de nova tentativa.
- RNF07 — Orientação e ciclo de vida: Durante uma partida, a interface deve manter-se na orientação vertical para evitar que a rotação recrie a sessão. Se o ecrã for bloqueado durante a espera pelo abanão ou durante uma animação, o sistema deve evitar iniciar ou concluir operações de jogo inesperadas em segundo plano.

Desempenho (Performance):

Estes requisitos definem as métricas de velocidade, eficiência e tempos de resposta necessários para manter a fluidez do jogo.

- RNF08 — Execução das operações de rede: As operações de rede não devem bloquear a interface. O pedido à OpenTDB utiliza execute(), mas é executado num ExecutorService; os pedidos ao Supabase utilizam enqueue() e callbacks.
- RNF09 — Carregamento por blocos: As perguntas devem ser solicitadas em blocos de três. O pedido deve decorrer em paralelo com as animações dos dados, e o resumo só deve surgir quando ambos tiverem terminado.
- RNF10 — Disponibilidade de serviços externos: A aplicação não pode garantir disponibilidade permanente da OpenTDB ou do Supabase. Deve tratar a indisponibilidade destes serviços e permitir ao jogador recuperar do erro sem perder o controlo da interface.
- RNF11 — Tempo de arranque: A meta de arranque a frio inferior a três segundos só deve ser apresentada como requisito validado depois de ser medida num dispositivo e numa versão Android especificados. O código atual não comprova essa métrica.

Suportabilidade e Restrições (Supportability & +):

Esta secção engloba as restrições tecnológicas, dependências de hardware, normas de segurança e padrões arquiteturais impostos ao projeto.

- RNF12 — Separação de responsabilidades: A interface deve apresentar os ecrãs, a GameSession deve gerir o estado e as regras da partida, e os repositórios devem tratar do acesso aos dados. O código tem esta separação em vários componentes, embora algumas decisões e dependências concretas sejam ainda coordenadas diretamente pela GameActivity.
- RNF13 — Plataforma: A aplicação deve ser nativa para Android e desenvolvida em Java. O minSdk atual é 24. O IntelliJ IDEA é uma

ferramenta de desenvolvimento, não uma característica exigida ao dispositivo que executa a aplicação.

- RNF14 — Dependências de hardware e rede: O fluxo de jogo depende de acelerómetro e ligação à Internet para descarregar perguntas. A vibração é opcional: se o dispositivo não tiver motor de vibração, os restantes elementos do jogo devem continuar disponíveis. Atualmente, a aplicação não apresenta uma alternativa explícita para a ausência de acelerómetro.
- RNF15 — Segurança da comunicação: As comunicações com a OpenTDB e o Supabase devem utilizar HTTPS. A chave pública anon do Supabase não deve ser tratada como segredo ou como prova de identidade. A aplicação não deve incluir uma chave privilegiada de serviço; o controlo de acesso deve ser aplicado no servidor, através das políticas adequadas.
- RNF16 — Integridade dos dados: Os campos enviados devem respeitar os tipos esperados: ID e nickname como texto, pontuação e streak como inteiros. O servidor deve rejeitar valores inválidos e preservar os melhores valores de pontuação e streak por jogador. A validação feita no cliente, por si só, não impede a alteração manual dos valores enviados.
- RNF17 — Controlo de pedidos à OpenTDB: Deve ser feito um pedido por lançamento aceite, para obter um bloco de três perguntas. O estado da sessão impede novos abanões enquanto o bloco está a ser carregado. O código não implementa um limitador temporal global específico para a API.
- RNF18 — Testabilidade e Modo Deus: O Modo Deus deve permitir executar rapidamente um bloco de três perguntas com valores artificiais, sem alterar os recordes reais nem o leaderboard. O código atual ativa-o com cinco toques no logótipo, separados por, no máximo, 900 milissegundos, e guarda a opção localmente. Em versões públicas, o modo de teste deve ser limitado a versões de QA ou desenvolvimento. Os testes unitários atuais cobrem o mapeamento e a composição das opções das perguntas; não comprovam, por si só, todos os fluxos do jogo.

- RNF19 — Cancelamento de lançamentos interrompidos: Se o dispositivo for bloqueado durante uma animação ou um pedido de perguntas, a aplicação deve impedir que o resultado parcial seja apresentado depois de o jogador regressar e deve permitir uma tentativa coerente.

4. Casos de Uso

A modelação através de Casos de Uso (Use Cases) constitui uma ferramenta essencial na Engenharia de Software para traduzir os requisitos funcionais em interações claras entre os intervenientes (atores) e o sistema. Esta abordagem é crítica porque estabelece uma ponte entre as necessidades de negócio e o desenvolvimento técnico, permitindo mapear visual e descritivamente as fronteiras da aplicação, identificar quem interage com ela e detalhar a sequência exata de eventos necessários para atingir um objetivo específico. No projeto do DiceMania, a especificação dos Casos de Uso garante que toda a arquitetura é pensada com foco na perspetiva do utilizador, assegurando que os fluxos lógicos, desde o processo de registo inicial e mecânica central do jogo, até à submissão das pontuações e manutenções de depuração, cumprem as regras estabelecidas antes da implementação final do código.

Como ilustrado visualmente na figura 2, o sistema "Dice Mania" atua como a fronteira central onde decorrem as interações, recebendo os estímulos do "Jogador" e do "Tester". De seguida, apresenta-se o detalhe técnico de cada um destes cenários.

4.1. Especificação dos Casos de Uso:

UC01 - Registar Utilizador

A aplicação deve permitir ao utilizador inserir um "Nickname" e gerar um ID único no servidor para efeitos de identificação.

- Ator Principal: Jogador
- Ator Secundário: Sistema (Supabase)
- Relacionamento: Jogador -> Registar Utilizador
- Fluxo Principal:
    1. O Jogador insere o "Nickname" e clica em registar.
    2. O Sistema envia o pedido para o Servidor (Supabase).
    3. O Servidor devolve um ID único.
    4. O Sistema guarda o Nickname e ID localmente para não voltar a pedir.

UC02 - Lançar Dados e Jogar

O utilizador inicia uma partida através do movimento físico, responde aos blocos de perguntas cronometradas, e a sua pontuação é validada e gravada.

- Ator Principal: Jogador
- Atores Secundários: Sistema (OpenTDB)
- Relacionamento: Jogador -> Lançar Dados e Jogar
- Fluxo Principal:
    1. O Jogador "agita" o telemóvel (interage com os sensores).

2. O Sistema sorteia o Tema e o Tipo e faz o pedido à API Externa (OpenTDB).

3. A API Externa devolve as 3 perguntas.

4. O Sistema inicia os temporizadores (5s espera + 10s resposta) e exibe as perguntas.

5. O Jogador seleciona uma resposta.

6. O Sistema valida a resposta, calcula a pontuação e atualiza o Streak.

7. O loop repete-se até às 15 perguntas.

8. No fim, o Sistema submete a pontuação final de forma invisível para o Servidor (Supabase).

UC03 - Consultar Leaderboard

O utilizador solicita a visualização do ranking global para verificar a sua posição em relação aos restantes jogadores.

- Ator Principal: Jogador

- Ator Secundário: Sistema (Android)

- Relacionamento: Jogador -> Consultar Leaderboard

- Fluxo Principal:

    1. O Jogador clica no botão "Ver Leaderboard".

2. O Sistema faz um pedido ao Servidor (Supabase).

3. O Servidor devolve a lista das melhores pontuações.

4. O Sistema apresenta a tabela ordenada no ecrã.

UC04 - Ativar Ferramentas de Depuração (Debug Mode)

O utilizador privilegiado acede a um menu oculto, através de um atalho não documentado (Easter Egg), para manipular o estado do jogo, facilitando testes e demonstrações em tempo real.

- Ator Principal: Desenvolvedor (ou Tester)

- Ator Secundário: Sistema

- Relacionamento: Tester -> Ativar Ferramentas de Depuração

- Fluxo Principal:
    1. O Desenvolvedor efetua 5 toques rápidos consecutivos num elemento inativo do ecrã principal (ex: logótipo da app).

    2. O Sistema deteta o padrão de toques validando a janela de tempo.

    3. O Sistema sobrepõe um menu oculto com opções de depuração (ex: "Injetar 15.000 Pontos", "Simular Erro de Rede", "Limpar Registo Local").

    4. O Desenvolvedor seleciona a ação pretendida.

    5. O Sistema executa a alteração, contornando as regras normais de jogo, e atualiza o ecrã imediatamente.

UC05 - Consultar Perfil e Estatísticas

O utilizador acede a um ecrã de definições a partir do menu principal para visualizar a sua identidade local e as suas métricas de jogo, servindo também como ponte de navegação para outras áreas.

- Ator Principal: Jogador
- Ator Secundário: Sistema (Memória Local - SharedPreferences)
- Jogador -> Consultar Perfil e Estatísticas -> Sistema
- Fluxo Principal:
    1. O Jogador, no ecrã de menu (onde escolhe começar a jogar), clica no botão de "Definições/Perfil".
    2. O Sistema acede à memória local (SharedPreferences).
    3. O Sistema recupera o nome do jogador, a pontuação máxima e o *streak* máximo.
    4. O Sistema apresenta estes dados no ecrã, disponibilizando botões adicionais (ex: "Ver Leaderboard").

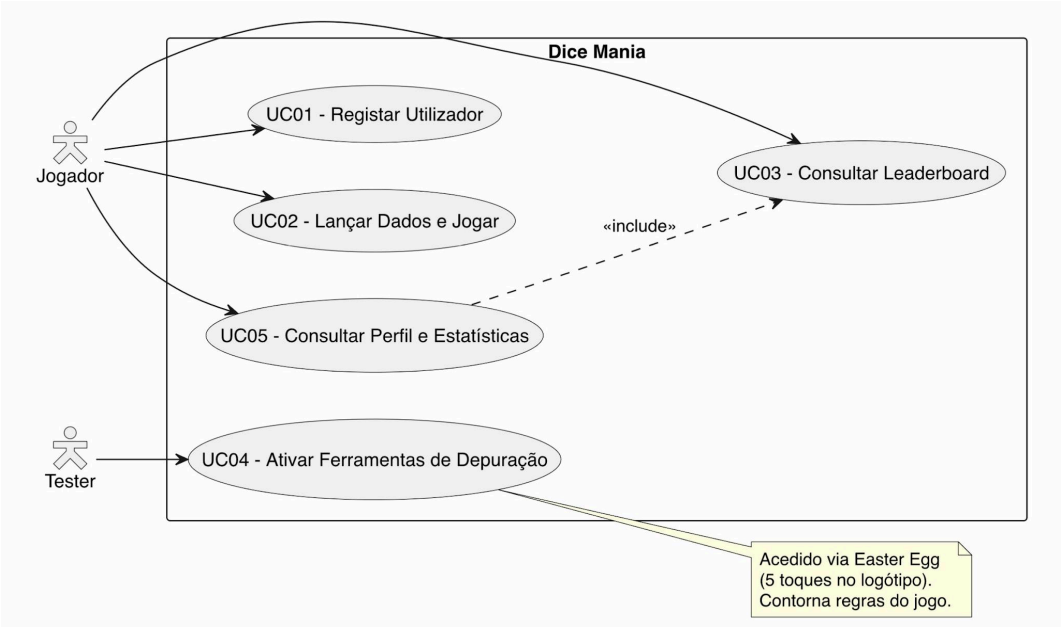


Figura 2 - Diagrama de Casos de Uso

5. Diagramas de Sequência do Sistema (SSD)

Os Diagramas de Sequência de Sistema (SSD) são artefactos fundamentais na engenharia de software que ilustram a sequência cronológica de interações entre os atores externos (como o utilizador) e o sistema em análise. Nesta fase de modelação, o sistema é tratado como uma "caixa negra", o que significa que o diagrama foca-se exclusivamente naquilo que entra (comandos, eventos físicos, dados) e naquilo que sai (feedback visual, alterações de estado), abstraindo-se da complexidade do código e das classes internas que executam essas operações.

A elaboração destes diagramas é de extrema importância, pois serve como ponte entre os Casos de Uso (orientados ao negócio e à experiência do utilizador) e o Design Arquitetural (orientado à programação). Eles permitem identificar com precisão os "Eventos de Sistema", os gatilhos exatos aos quais a aplicação terá de reagir.

Para o DiceMania, foram mapeados os fluxos de eventos dos quatro Casos de Uso principais, garantindo que todas as interações vitais estão devidamente documentadas:

- UC01 - Registar Utilizador: Este diagrama, figura 2, ilustra a interação inicial de onboarding, demonstrando como o sistema reage ao pedido de criação de identidade do jogador, gerindo a comunicação externa e garantindo a persistência local dos dados.
- UC02 - Jogar Partida: Sendo o core da aplicação, este SSD, presente na figura 3, modela um fluxo mais complexo e cíclico. Representa a tradução de uma ação física (agitar o dispositivo) num evento de sistema, seguido por um loop cronometrado de perguntas e respostas, culminando na submissão invisível da pontuação.

- UC03 - Consultar Leaderboard: Demonstra uma interação de consulta (leitura de dados), onde o ator solicita a classificação global e o sistema atua como intermediário na devolução e apresentação estruturada da tabela. Podemos ver este diagrama na figura 4.
- UC04 - Ativar Ferramentas de Depuração (Debug Mode): A figura 5 ilustra a interação de um ator secundário e privilegiado (o Desenvolvedor/Tester). Este diagrama é crucial para evidenciar como o sistema deteta um padrão de inputs ocultos (5 toques) para sobrepor regras de negócio e injetar alterações diretas no estado da aplicação.

5.1. UC01 - Registar Utilizador

Como novo jogador, pretendo inserir um nickname para ser identificado de forma única no sistema e nas tabelas de classificação.

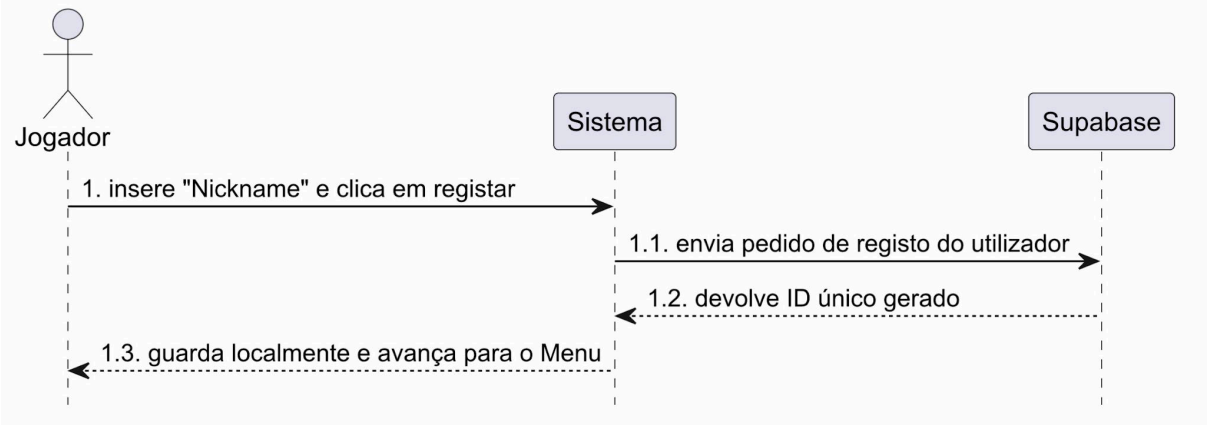


Figura 3 - SSD do UC01

UC01 - Registar utilizador	Registo inicial do jogador na aplicação para geração de credenciais únicas.
Pré-condições	O jogador abre a aplicação pela primeira vez e não tem credenciais guardadas na memória local (SharedPreferences).
Pós-condições	O "Nickname" e o "ID" gerado são guardados localmente; o jogador avança para o Menu Principal.
Cenário Ideal	1. O Jogador insere o "Nickname" e submete o registo. 2. O Sistema comunica com o servidor, recebe um ID, guarda as credenciais localmente e confirma o sucesso da operação.

Tabela 1 - Especificações do SSD de UC01

5.2. UC02 - Jogar Partida

Como jogador, pretendo iniciar uma partida através do movimento físico, responder às questões cronometradas e ver a minha pontuação final submetida.

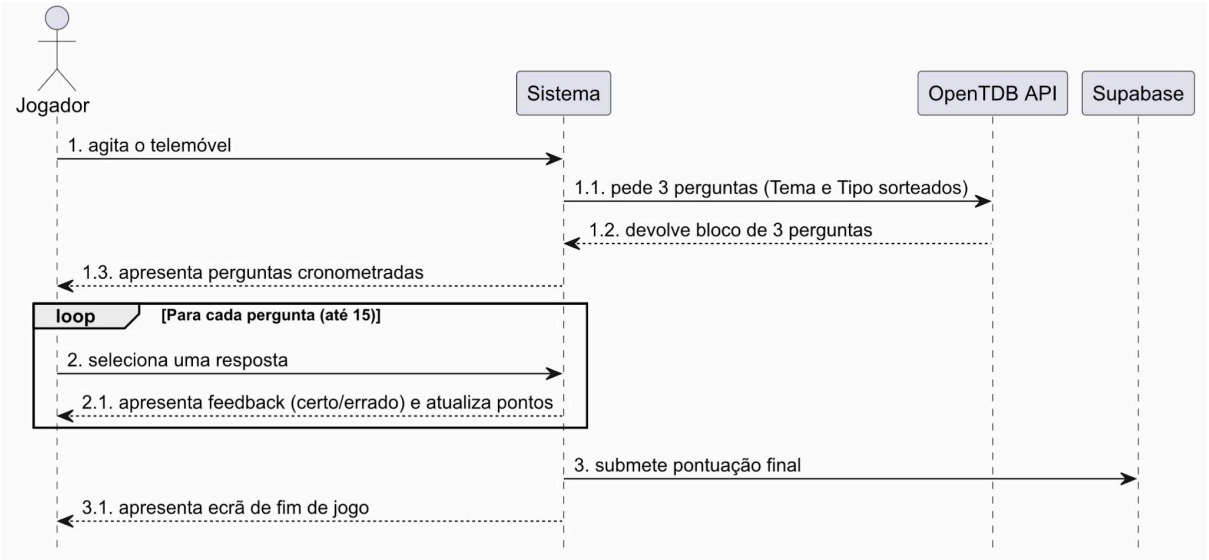


Figura 4 - SSD de UC03

UC02 - Jogar Partida	Fluxo completo de uma partida, desde o lançamento dos dados (agitar) até à submissão da pontuação final.
Pré-condições	O jogador tem de ter o sensor acelerómetro ativo e ligação à internet para descarregar as perguntas.
Pós-condições	A pontuação final é enviada e guardada no servidor (Supabase); o estado do jogo passa a FINALIZADO.

Cenário Ideal	1. O Jogador agita o telemóvel. 2. O Sistema gera um bloco de perguntas e apresenta a primeira após o compasso de espera. 3. O Jogador responde atempadamente à pergunta. 4. O Sistema valida a resposta, fornece feedback visual (verde/vermelho) e atualiza a pontuação e streak. 5. O processo repete-se até à 15ª pergunta. 6. O Sistema submete silenciosamente a pontuação para o servidor no final da partida.

Tabela 2 - Especificações do SSD de UC02

5.3. UC03 - Consultar Leaderboard

Como jogador, pretendo visualizar a tabela global de classificações para verificar a minha posição face aos restantes utilizadores.

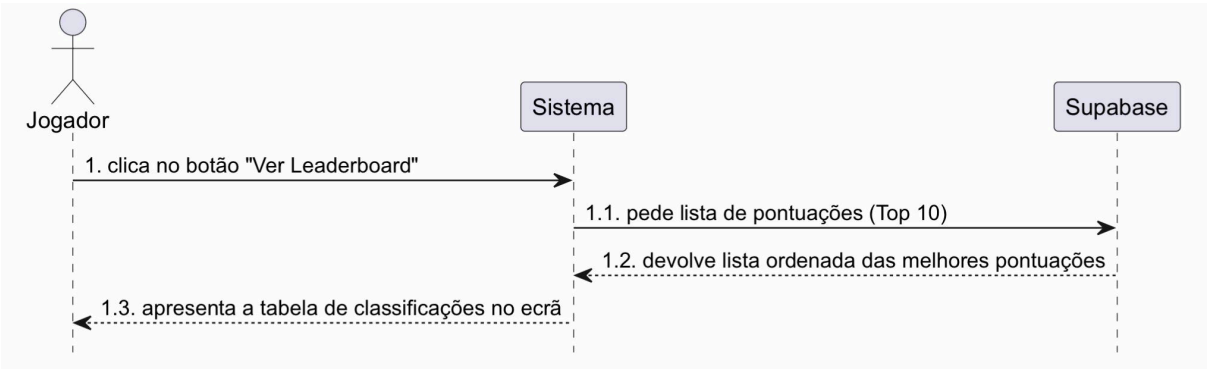


Figura 4 - SSD do UC03

UC03 - Consultar Leaderboard	Apresentação do ranking global com as melhores pontuações do servidor.
Pré-condições	Ligação à internet disponível para efetuar o pedido ao servidor.
Pós-condições	Nenhuma.
Cenário Ideal	1. O Jogador solicita a visualização do Leaderboard. 2. O Sistema descarrega os dados do servidor (Supabase) e apresenta a

	tabela ordenada (Top 10) no ecrã.

Tabela 3 - Especificações do SSD de UC03

5.4. UC04 - Ativar Ferramentas de Depuração (Debug Mode)

Como desenvolvedor, pretendo aceder a um menu oculto para manipular o estado do jogo e facilitar os processos de teste.

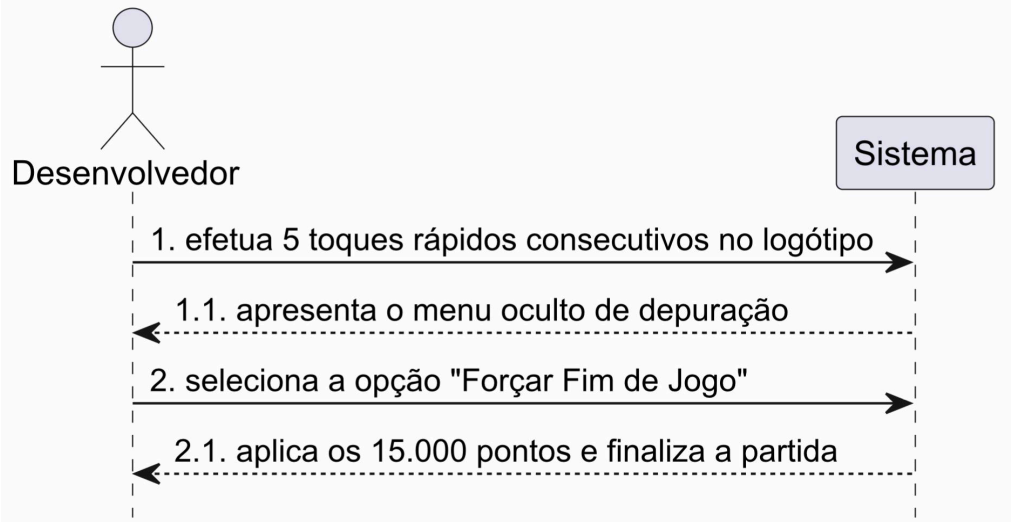


Figura 5 - SSD do UC04

UC04 - Consultar Leaderboard	Acesso a um menu escondido para testes e injeção de estado.
Pré-condições	A aplicação tem de estar aberta no ecrã principal e pronta a detetar toques.
Pós-condições	O estado do sistema é modificado artificialmente (ex: a partida é dada como concluída com pontos máximos).
Cenário Ideal	1. O Desenvolvedor executa o padrão de toques exigido (5 toques no logótipo). 2. O Sistema reconhece o padrão e sobrepõe o menu de Debug. 3. O Desenvolvedor escolhe injetar pontuação. 4. O Sistema atualiza a GameSession instantaneamente, contornando o fluxo normal.

Tabela 4 - Especificações do SSD de UC04

5.5. UC05 - Consultar Perfil e Estatísticas

Como jogador registado, pretendo aceder a um ecrã de perfil para visualizar o meu nome, os meus recordes locais (pontuação e streak) e navegar para a tabela de classificações.

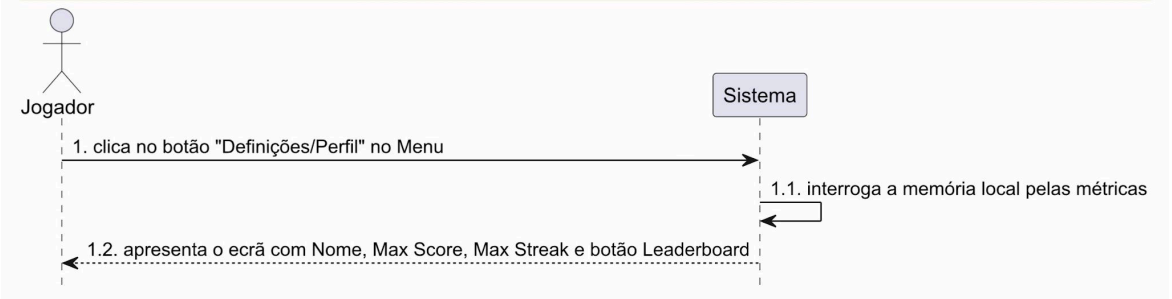


Figura 6 - SSD do UC05

UC05 - Consultar Perfil e Estatística	Apresentação de um ecrã estático de resumo (dashboard) com os dados pessoais e recordes do utilizador, atuando como um hub de navegação secundário.
Pré-condições	O jogador tem de estar registado (possuir credenciais ativas na memória local) e encontrar-se no Menu Principal da aplicação.
Pós-condições	Nenhuma alteração de estado no sistema. O ecrã fica a aguardar uma nova ação (voltar, ou ir para o Leaderboard).

Cenário Ideal	1. O Jogador, perante as opções "Começar Jogo" ou "Definições", seleciona "Definições". 2. O Sistema lê o ficheiro XML do Android em frações de segundo. 3. O Sistema exibe as informações no ecrã.

Tabela 5 - Especificações do SSD de UC05

6. Arquiteturas

Na Engenharia de Software, as arquiteturas servem para documentar e comunicar decisões estruturais do projeto. Elas são necessárias porque o desenvolvimento de um sistema nunca se resume apenas a escrever código; é preciso planear como o software será organizado (lógica) e onde/como ele irá funcionar no mundo real (física).

- A Arquitetura Lógica se foca no "Como" o código-fonte é organizado. Divide o sistema em pacotes e módulos de forma a garantir a manutenção, separação de responsabilidades (por exemplo, a física do sensor longe do visual) e a testabilidade. Segue os princípios de Clean Architecture.
- A Arquitetura Física se foca no "Onde" o sistema corre e de que peças de hardware depende. Descreve a infraestrutura (telemóvel, servidores), as comunicações (Internet, protocolos) e os componentes físicos. É vital para compreender o fluxo de rede, limitações de hardware e segurança (HTTPS).

6.1 Arquitetura Física

Esta imagem, presente na figura 7, serve para mostrar o ecossistema físico onde o teu jogo corre. Mapeia o telemóvel do jogador, os sensores de hardware e as comunicações à distância com o mundo.

- O Nó do Smartphone (Dispositivo Android (Smartphone)):
    - Um bloco 3D que representa o hardware físico do utilizador onde está instalado o ficheiro de execução (DiceMania.apk).
- Os Componentes Internos e as suas Conexões:
    - UI (Apresentação): A interface visual que corre no ecrã. Liga-se à GameSession do jogo através de um canal chamado GameController.
    - Debug manager: O menu oculto. Este liga-se à UI via 5 toques (Ativação) (o Easter Egg) e que, por sua vez, tem um canal direto (DebugController) para a GameSession, permitindo saltar as regras,simular erros de rede ou injetar pontos.
    - Core Business (GameSession): É o cérebro local que gere o loop das 15 perguntas e a máquina de estados (se o jogador está à espera de agitar, a responder, ou no intervalo).
    - Hardware / Sensor: O chip físico (acelerómetro) do telemóvel. Comunica com o motor de jogo usando o evento ShakeSensor quando o movimento atinge os 2.7G de força.
    - Adapter / Network (Retrofit): O módulo de rede. Conecta-se à lógica interna por meio do Network Accesse abre o canal de internet para o exterior do telemóvel.
    - Helper / Local Storage (SharedPreferences): O chip de memória interna que lê e guarda o perfil do jogador.
- O Canal de Comunicação de Rede (Internet (HTTPS)):
    - Verificamos que as linhas de rede que saem do telemóvel rumo à nuvem, protegidas por encriptação segura HTTPS.
- Os Servidores Externos (Nós de Destino):

- Servidor OpenTDB (API Externa): O servidor 3D onde vive o componente de API REST. É a ele a quem o jogo faz os pedidos para sacar as perguntas de forma assíncrona.
- Servidor Supabase (Backend): O servidor 3D da vossa base de dados centralizada que corre o componente BD PostgreSQL. É aqui que se regista o novo utilizador no primeiro arranque e para onde se envia a pontuação final para preencher o leaderboard dos melhores jogadores da semana.

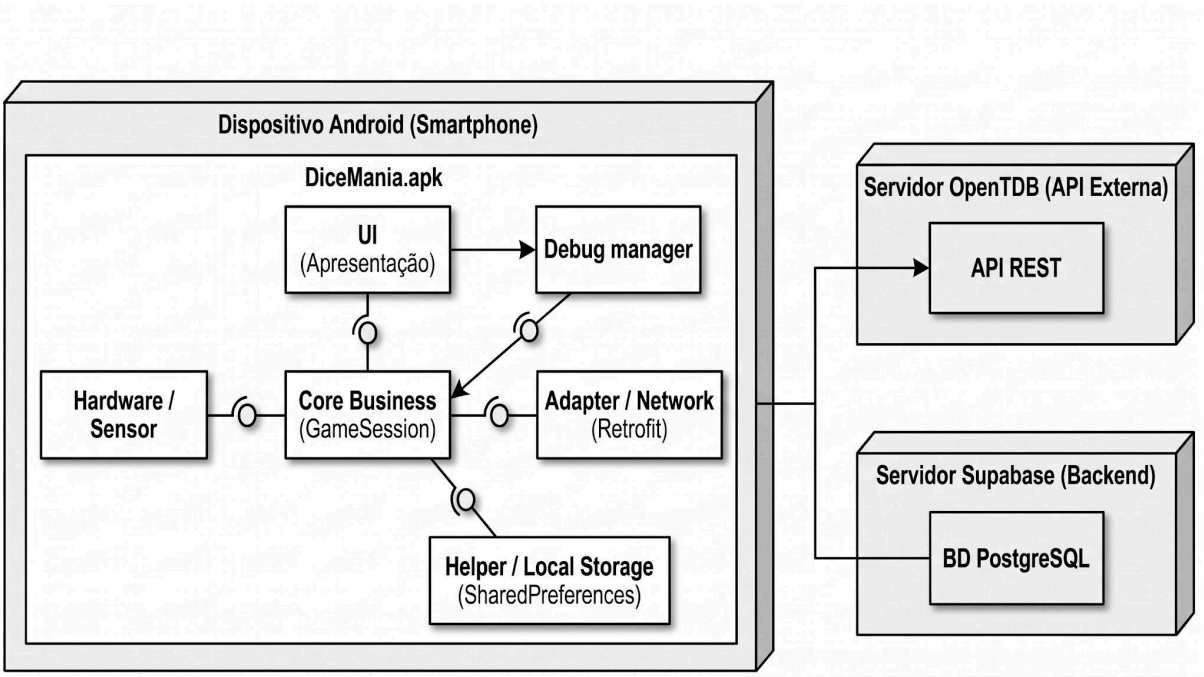


Figura 7 - Arquitetura Física

6.2 Arquitetura lógica

A Arquitetura Lógica da aplicação reflete um padrão de separação de camadas (Clean Architecture), como podemos ver na figura 8. Esta separação metodológica garante um baixo nível de acoplamento, permitindo que

alterações nas camadas tecnológicas de suporte não inflijam danos ou necessitem de refatorização nas regras do jogo. A organização dos pacotes da aplicação materializa este princípio.

A solução assenta em três pilares verticais (conforme o Package Diagram delineado):

1. Camada de Domínio (domain): É o componente nuclear e agnóstico do sistema. Embala as regras estritas da lógica de negócio. Está dividida em Modelos (estrutura do jogador, questões e regras de temporização — como o AntiCheatTimer) e Contratos/Repositórios (Interfaces). O isolamento deste pacote é total: desconhece a existência do Android ou da internet.
2. Camada de Apresentação (presentation / ui): Camada exterior de topo. Gere exclusivamente o ciclo de vida e a interface visual do Android (GameActivity, LeaderboardActivity). Atua de forma passiva: recolhe sinais da interface e reencaminha ordens para os controladores, não guardando dados lógicos, mas tão-somente operando a camada áudio, transições de cor e leitura de botões.
3. Camada de Infraestrutura/Dados (data): É responsável pela comunicação do domínio com o "mundo exterior sujo". Alberga os subpacotes que interagem concretamente com as APIs (network usando a framework genérica do Retrofit), com o hardware mecânico do dispositivo móvel (sensor), e com os mecanismos de armazenamento estático da memória Flash do terminal (local via Android SharedPreferences).

A grande valia desta configuração e organização manifesta-se no padrão de dependências: as setas do design fluem da Apresentação e dos Dados sempre em direção ao Domínio. Desta forma, fica provado conceptualmente que o núcleo da aplicação (regras de jogo) sobrevive a qualquer mudança periférica, como a migração para outra plataforma móvel ou a substituição integral do servidor de base de dados.

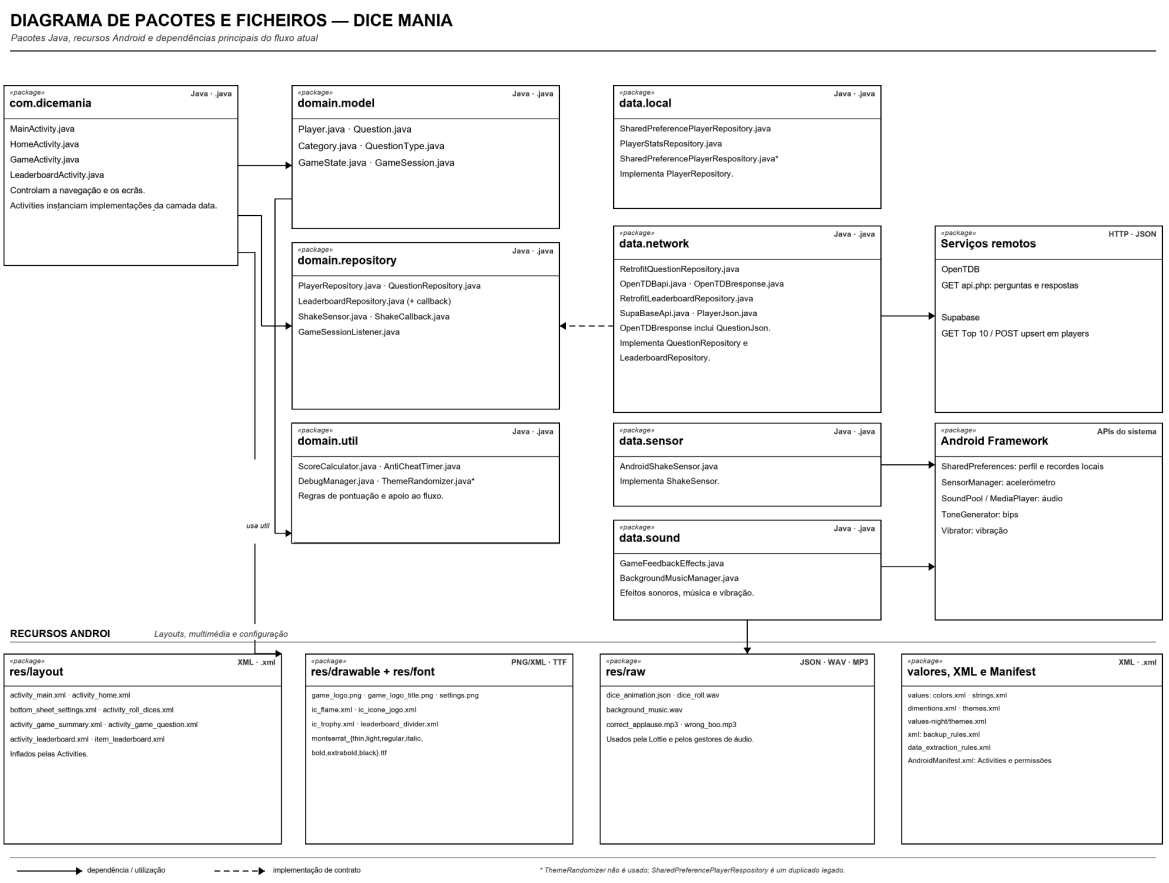


Figura 8 - Arquitetura Lógica

7. Diagrama de Classes

Um modelo de classes, amplamente utilizado no âmbito da Linguagem de Modelação Unificada (UML), é um diagrama visual que descreve a estrutura estática de um sistema de software ao nível da programação orientada a objetos. Em vez de focar nas ações temporais ou nos fluxos de processos do programa, este modelo foca-se nos conceitos centrais do sistema, detalhando as classes existentes, os seus respetivos atributos (como dados ou propriedades) e os seus métodos (as ações ou funções que o programa pode executar). Além disso, estabelece as associações e dependências que ligam essas entidades umas às outras, criando um mapa completo da organização do código.

A principal utilidade do modelo de classes reside no facto de funcionar como uma verdadeira planta arquitetural para o desenvolvimento de software, como verificável nas figuras 9, 10, 11, 12 e 13. Ele permite que os programadores e engenheiros planeiem e organizem logicamente todo o sistema antes da fase de codificação, evitando falhas estruturais e facilitando a comunicação entre os membros da equipa através de um formato padronizado e fácil de compreender. Posteriormente, serve também como documentação essencial para a manutenção e evolução do projeto, além de simplificar a conversão dos conceitos abstratos do negócio para linguagens de programação e para a estrutura de bases de dados.

7.1. Entidades de Dados — domain.model

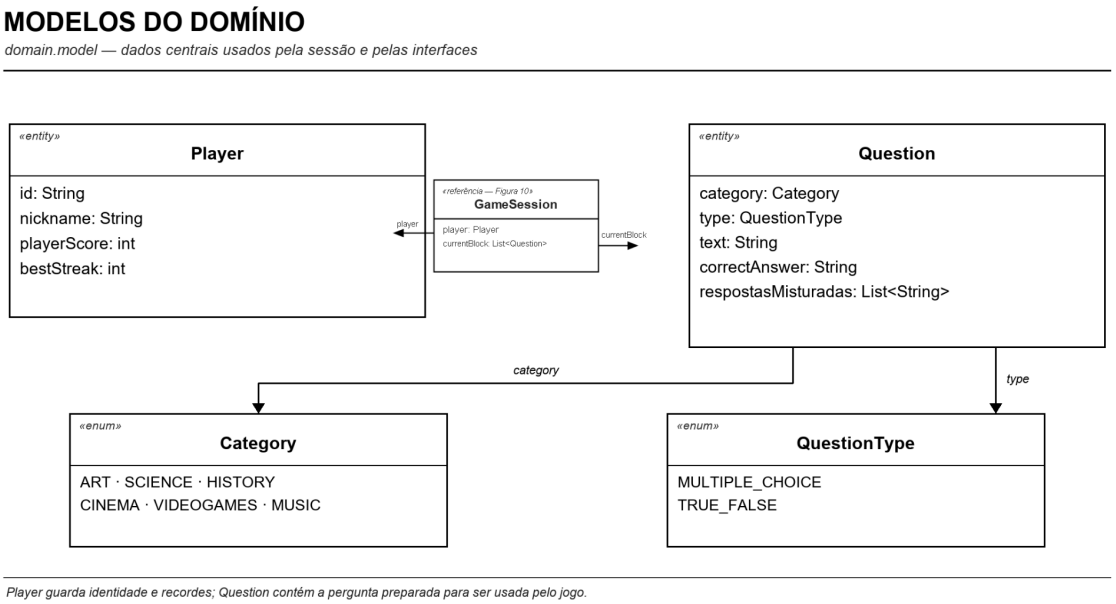


Figura 9 — Entidades de domínio e enumerações usadas pelo jogo.

Esta figura, representa o pacote domain.model contém os objetos que representam os dados usados durante uma partida.

A classe Player representa o perfil de um jogador. Guarda o seu identificador (id), o nickname, a pontuação (playerScore) e o maior streak registado (bestStreak). O identificador é criado localmente pela MainActivity, através de um UUID, e é usado para reconhecer o mesmo jogador quando os dados são enviados para o Supabase. A pontuação da partida em curso é mantida na GameSession; o playerScore pertence ao objeto de jogador usado, por exemplo, pelos dados do leaderboard.

A classe Question representa uma pergunta pronta a ser apresentada. Guarda a categoria, o formato da pergunta, o enunciado, a resposta correta e a lista de respostas misturadas. Essa lista é preparada pelo repositório de perguntas antes de ser entregue à interface.

Category enumera as categorias disponíveis: ART, SCIENCE, HISTORY, CINEMA, VIDEOGAMES e MUSIC. QuestionType limita os formatos a MULTIPLE_CHOICE e TRUE_FALSE.

7.2. Controlador e Estados da Partida — domain.model

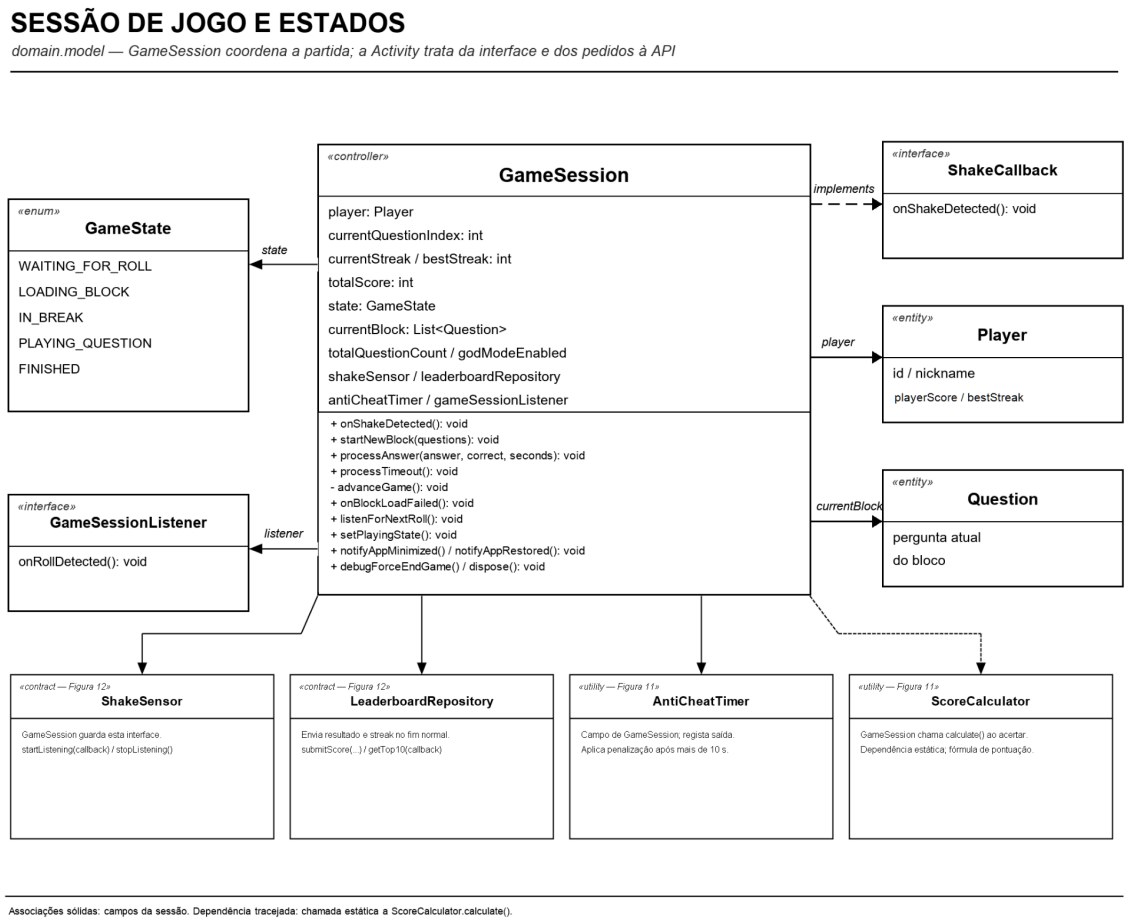


Figura 10 — Controlador da partida, estados e objetos com que se relaciona.

A classe GameSession controla o estado e o progresso de uma partida. Guarda o jogador, o índice da pergunta atual, o streak atual e máximo, a pontuação acumulada, o bloco de perguntas e o estado do jogo. A figura também mantém referências ao sensor, ao repositório do leaderboard, ao temporizador anti batota e ao listener usado para avisar a GameActivity.

A GameSession implementa ShakeCallback. Quando recebe um abanão válido no estado WAITING_FOR_ROLL, muda para LOADING_BLOCK, desliga temporariamente o sensor e informa a GameActivity através de GameSessionListener. A Activity inicia a animação dos dados e pede as perguntas ao repositório. Portanto, a GameSession controla a partida, mas não faz diretamente o pedido à OpenTDB.

Depois de receber três perguntas, a Activity chama startNewBlock(). A sessão passa para IN_BREAK, estado usado durante o resumo entre perguntas. Quando o resumo termina, a Activity chama setPlayingState() e apresenta a pergunta. processAnswer() trata da resposta e atualiza os valores da partida; processTimeout() trata do caso em que o tempo termina. advanceGame() determina se o jogo continua, se é necessário voltar a lançar os dados ou se a partida terminou.

Quando a aplicação passa para segundo plano, notifyAppMinimized() regista a saída se houver uma pergunta em curso e pára o sensor. Ao regressar, notifyAppRestored() verifica se é necessário aplicar a penalização por tempo fora da aplicação. A escuta do sensor é retomada pela GameActivity, através de listenForNextRoll(), apenas quando a sessão está em WAITING_FOR_ROLL. Estes métodos ajustam o comportamento existente da sessão; não introduzem uma nova entidade nem um novo estado.

GameState descreve as fases da partida:

- WAITING_FOR_ROLL: à espera de um abanão.
- LOADING_BLOCK: animação e carregamento das perguntas.
- IN_BREAK: resumo entre perguntas.
- PLAYING_QUESTION: pergunta em curso.
- FINISHED: partida terminada.

Uma partida normal tem 15 perguntas, organizadas em blocos de três. No modo Deus, a sessão é reduzida a três perguntas e utiliza valores artificiais de pontuação e streak para permitir testar rapidamente os ecrãs.

7.3. Sensor Físico e Classes Utilitárias — data.sensor e domain.util

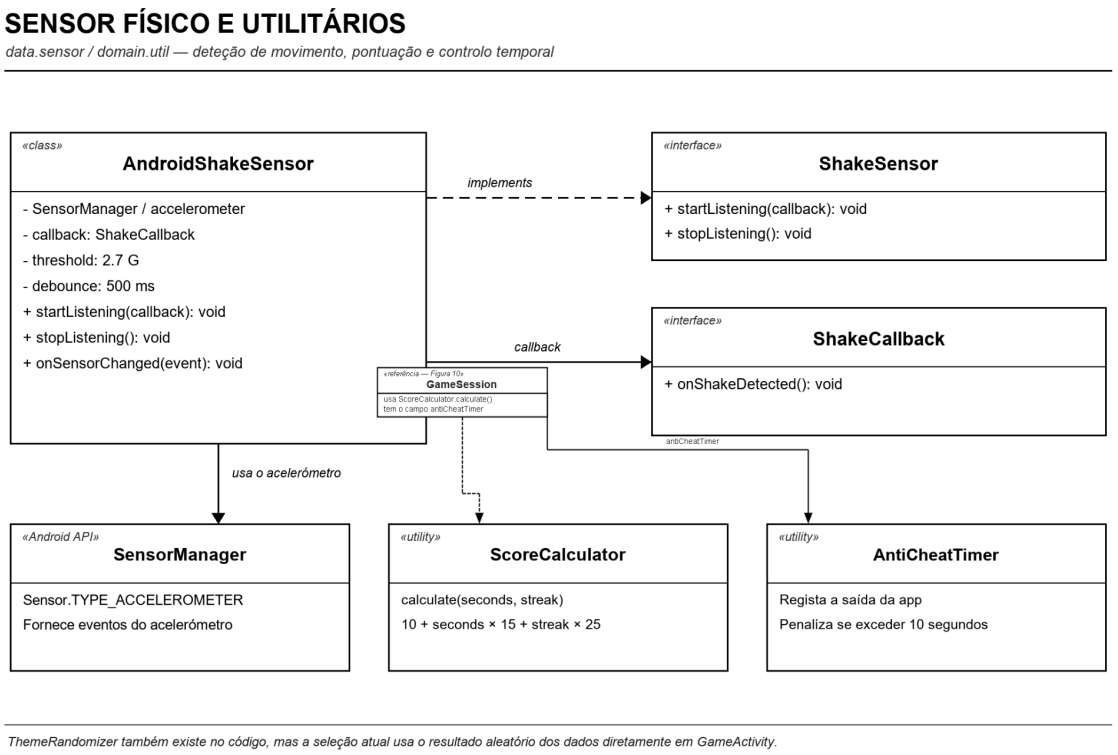


Figura 11 — Deteção do abanão e utilitários de pontuação e controlo temporal.

A figura mostra a classe AndroidShakeSensor que implementa ShakeSensor e usa o acelerómetro do Android. Calcula a força do movimento a partir dos três eixos do sensor e considera válido um abanão acima de 2.7G. O intervalo de 500 milissegundos entre deteções ajuda a evitar que um único movimento seja tratado como vários abanões. Quando deteta um abanão válido, chama onShakeDetected() no callback recebido.

A interface ShakeSensor define as operações para começar e parar a escuta do sensor. A interface ShakeCallback define o aviso enviado quando o abanão é detetado. A interface GameSessionListener permite à sessão avisar a Activity de que pode começar a animação.

ScoreCalculator calcula os pontos de uma resposta correta usando o tempo restante e o streak: 10 + (segundos restantes × 15) + (streak × 25). Se o tempo terminou, o cálculo devolve zero.

AntiCheatTimer regista quando o jogador sai da aplicação durante uma pergunta. Se regressar depois de mais de 10 segundos, a GameSession trata a situação como timeout.

A classe ThemeRandomizer também existe, mas não é usada no fluxo atual. Atualmente, a GameActivity sorteia os resultados dos dados e converte-os diretamente em categoria e formato de pergunta.

7.4. Contratos e Repositórios — domain.repository

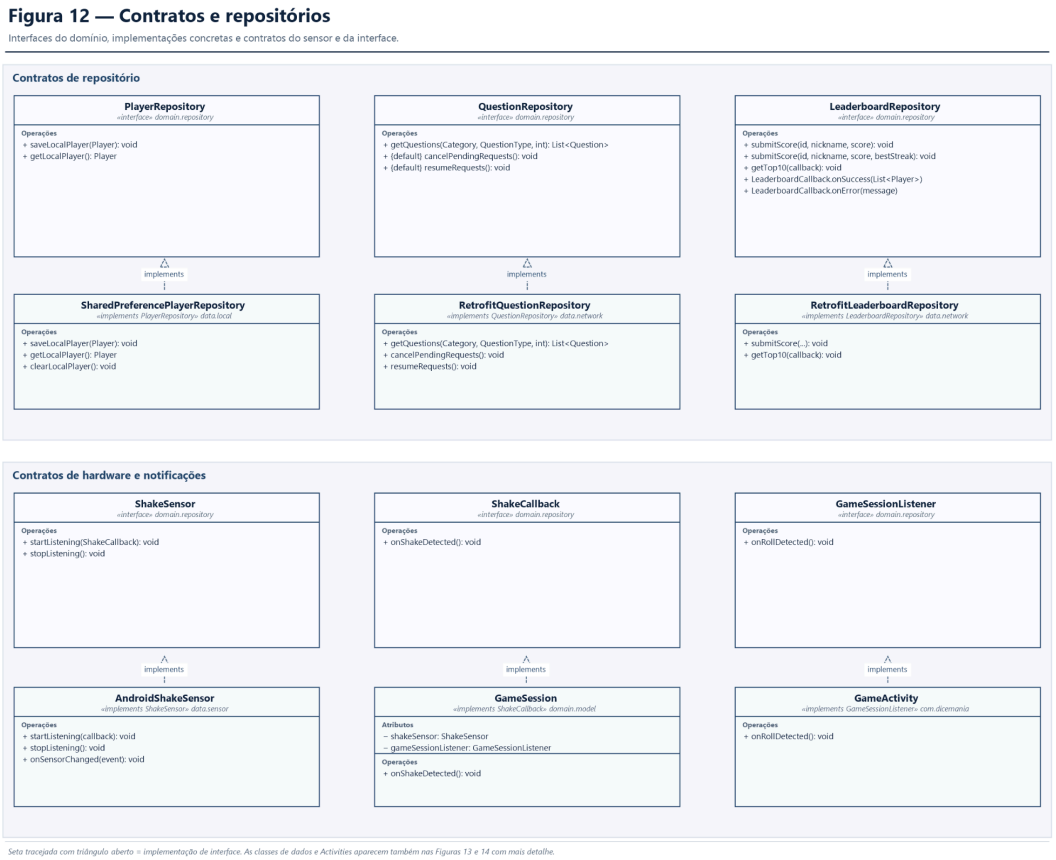


Figura 12 — Interfaces do domínio e respetivas implementações.

As interfaces de domain.repository definem as operações de acesso aos dados, como vemos na figura 12, sem obrigar a camada de domínio a conhecer todos os pormenores da implementação.

PlayerRepository define como guardar e recuperar localmente o jogador. QuestionRepository define como pedir perguntas por categoria, formato e quantidade. Também disponibiliza cancelPendingRequests() e resumeRequests(): o primeiro permite cancelar pedidos pendentes quando o ecrã é interrompido; o segundo permite aceitar novos pedidos quando a

Activity retoma a partida. Estas operações têm implementações vazias por omissão, para que repositórios sem pedidos canceláveis não tenham de acrescentar lógica desnecessária.

LeaderboardRepository define como enviar os resultados e carregar os dez melhores jogadores. LeaderboardRepository.LeaderboardCallback devolve o resultado da leitura do ranking: uma lista de jogadores em caso de sucesso ou uma mensagem de erro.

ShakeSensor, ShakeCallback e GameSessionListener definem os contratos usados na comunicação entre o sensor, a sessão e a interface.

As implementações concretas — SharedPreferencePlayerRepository, RetrofitQuestionRepository e RetrofitLeaderboardRepository — encontram-se nas camadas de dados. Isto permite que o código que usa os contratos não dependa diretamente dos detalhes de armazenamento ou de comunicação de rede.

7.5. Rede, DTOs e Persistência Local — data.network e data.local

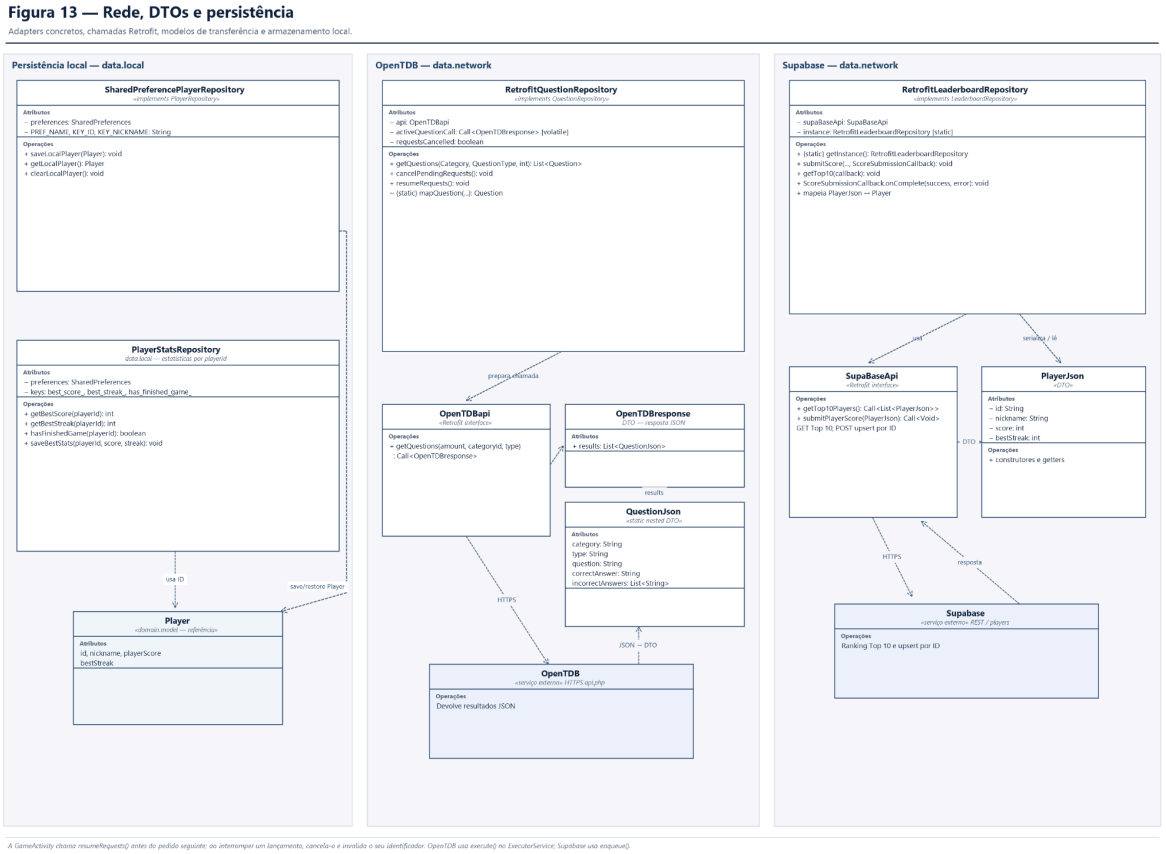


Figura 13 — Percurso dos dados entre as APIs, os DTOs, os modelos e o armazenamento local.

Na figura, o RetrofitQuestionRepository implementa QuestionRepository. Comunica com a OpenTDB através da interface OpenTDBapi, converte os dados recebidos em objetos Question e baralha as respostas antes de as entregar à partida. A resposta bruta é representada por OpenTDBresponse e pela classe interna QuestionJson.

Para permitir que um pedido seja interrompido, RetrofitQuestionRepository mantém uma referência ao pedido ativo (activeQuestionCall) e um indicador de cancelamento (requestsCancelled). cancelPendingRequests() ativa esse indicador e cancela a chamada HTTP em curso, se existir. resumeRequests() limpa o indicador para permitir novos pedidos. Antes de iniciar o pedido, o repositório verifica se este já foi cancelado.

O pedido à OpenTDB usa uma chamada síncrona através de execute(). No fluxo atual, a GameActivity executa esse pedido num ExecutorService, fora da thread da interface, para evitar bloquear o ecrã enquanto as perguntas são carregadas. Se o pedido for interrompido, a Activity também invalida o identificador desse lançamento; assim, uma resposta antiga não é apresentada quando o jogador regressa.

RetrofitLeaderboardRepository implementa LeaderboardRepository e comunica com o Supabase através de SupaBaseApi. A classe PlayerJson é o DTO usado para converter os dados do jogador entre JSON e Java. A API permite enviar ou atualizar um jogador pelo seu ID e pedir o Top 10, ordenado por pontuação, streak máximo e nickname.

SharedPreferencePlayerRepository guarda localmente o ID e o nickname do jogador. PlayerStatsRepository guarda os recordes locais — melhor pontuação, melhor streak e indicação de que já terminou uma partida — associados ao ID do jogador.

7.6. Activities, Vistas e Serviços de Apoio — com.dicemania

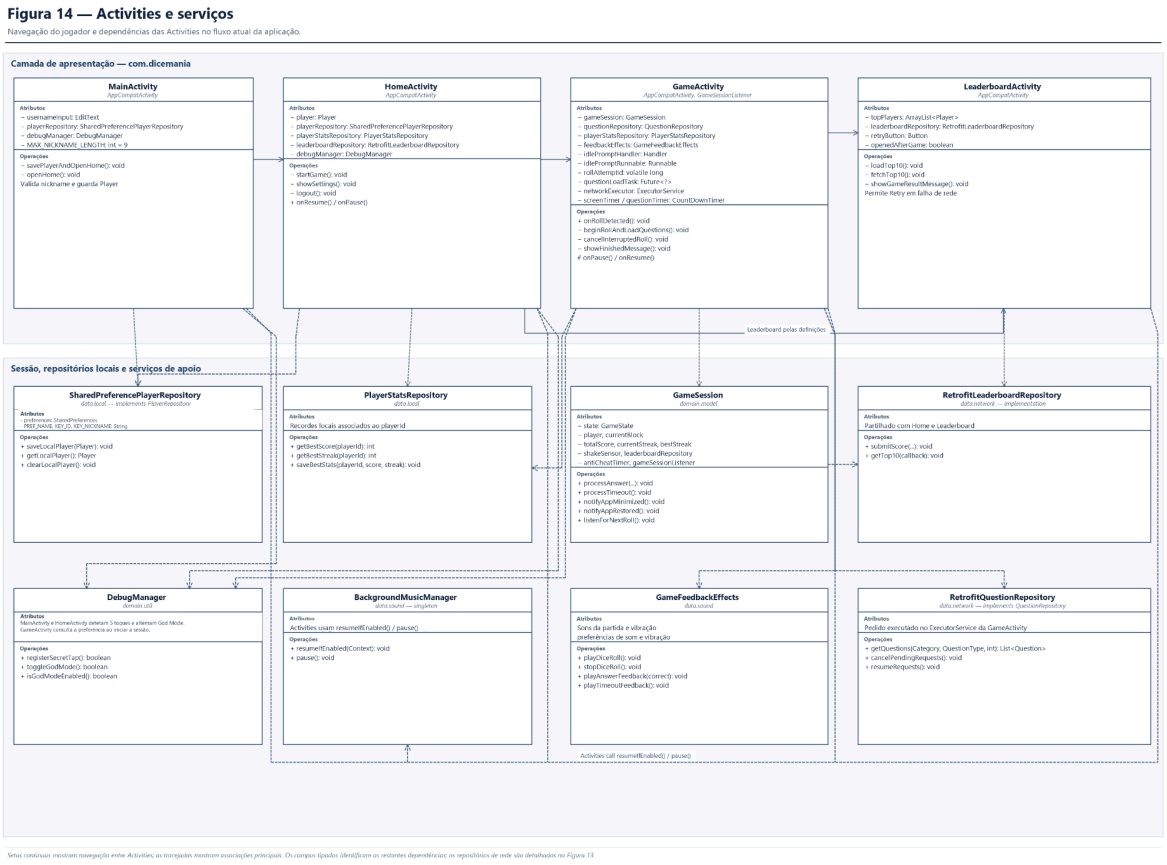


Figura 14 — Navegação entre ecrãs e serviços usados pelas Activities.

Esta figura retrata várias coisas. A MainActivity trata da entrada do jogador, valida o nickname, cria ou recupera o ID local e abre a HomeActivity. HomeActivity apresenta o botão para começar a partida e as definições do jogador, incluindo os recordes e o acesso ao leaderboard.

A GameActivity controla visualmente os dados, o resumo e as perguntas. Estes ecrãs não são Activities separadas: a mesma Activity troca entre os layouts activity_roll_dices.xml, activity_game_summary.xml e

activity_game_question.xml. Também coordena a animação, o carregamento das perguntas, os temporizadores e o feedback às respostas.

A Activity contém um Handler (idlePromptHandler) e um Runnable (idlePromptRunnable) para apresentar uma mensagem motivacional se o jogador permanecer 15 segundos à espera de abanar os dados. O campo rollAttemptId identifica cada lançamento. Quando um lançamento é interrompido, esse identificador é alterado para que os resultados de animações ou pedidos antigos sejam ignorados. questionLoadTask, do tipo Future<?>, guarda a tarefa de carregamento das perguntas executada pelo ExecutorService.

Ao sair do ecrã durante o carregamento de um bloco, a GameActivity cancela a tarefa, pede ao repositório que cancele a chamada à OpenTDB, interrompe as animações e descarta as perguntas parciais. Ao regressar, apresenta um ecrã seguro; se a sessão estiver à espera de um novo lançamento, volta a ativar o sensor.

LeaderboardActivity carrega e apresenta o Top 10. Quando é aberta no final de uma partida, também apresenta a mensagem correspondente ao resultado do jogador no ranking.

As classes auxiliares complementam este fluxo. DebugManager gere a sequência de toques no logo que ativa o modo Deus. PlayerStatsRepository guarda os recordes locais. GameFeedbackEffects gere os sons e a vibração da partida; BackgroundMusicManager gere a música ambiente e a sua pausa e retoma.

8. Diagramas de Sequência

Os Diagramas de Sequência são artefactos comportamentais da linguagem UML (Unified Modeling Language) que representam a ordem das interações entre os participantes de um sistema ao longo do tempo. A leitura faz-se de cima para baixo: cada mensagem representa uma chamada, uma resposta ou uma notificação entre partes da aplicação.

Enquanto o Modelo de Classes apresenta a estrutura estática do sistema, os Diagramas de Sequência mostram como essa estrutura participa nos principais casos de uso. Neste projeto, ajudam a acompanhar a comunicação entre o jogador, as Activities, a sessão de jogo, os sensores do dispositivo, os repositórios e os serviços externos.

Os diagramas documentam o fluxo implementado. Ajudam a verificar a divisão de responsabilidades entre os componentes, mas, por si só, não provam que a aplicação segue integralmente uma arquitetura específica. Por exemplo, a GameActivity coordena o ecrã e cria componentes concretos, como o sensor e os repositórios.

8.1. Elementos Essenciais dos Diagramas

Para interpretar os diagramas apresentados nesta secção, consideram-se os seguintes elementos:

- Ator: representa uma entidade externa que interage com o sistema. Nestes diagramas, é normalmente o jogador.
- Participante e linha de vida: cada caixa no topo identifica uma classe, serviço ou componente envolvido. A linha vertical tracejada indica a sua existência durante a sequência representada.
- Caixa de ativação: o retângulo sobre uma linha de vida assinala um período em que o participante está a executar uma operação.
- Chamada síncrona: a seta contínua com ponta preenchida representa uma chamada direta. O fluxo de execução aguarda a conclusão dessa operação.

- Mensagem assíncrona: representa uma notificação ou chamada cujo resultado chega mais tarde, sem bloquear o participante que a iniciou. É o caso de callbacks do sensor e das chamadas Retrofit com enqueue().
- Mensagem de retorno: a seta tracejada representa um resultado devolvido por uma chamada.
- Fragmento alt: representa alternativas condicionais, como os ramos de um if/else.
- Fragmento opt: representa uma operação que só acontece quando uma determinada condição se verifica.
- Fragmento loop: representa uma repetição, como o ciclo de perguntas ou os vários lançamentos dos dados.
- Fragmento par: representa operações que decorrem em paralelo. No carregamento de um bloco, a animação dos dados e o pedido de perguntas avançam em paralelo.

Os diagramas representam a ordem e as condições principais das interações; não pretendem especificar cada operação ao milissegundo. Os intervalos apresentados — como os 15 segundos para responder ou os 3 segundos de feedback — correspondem às durações definidas para os ecrãs.

8.1.1. UC01 — Registar Jogador

Esta figura, apresenta um diagrama representa o primeiro acesso à aplicação e a criação do perfil local. O registo não cria uma conta remota: o identificador do jogador é gerado no dispositivo e os dados de identificação são guardados em SharedPreferences.

Quando a aplicação abre, a MainActivity pede ao SharedPreferencePlayerRepository o jogador guardado localmente. Se o repositório encontrar um identificador e um nickname, a aplicação encaminha o jogador para o ecrã principal. Caso contrário, apresenta o formulário de entrada.

Ao carregar para iniciar, a aplicação valida o nickname. Se estiver vazio, mostra uma mensagem de erro. Se for válido, cria um identificador com

UUID.randomUUID(), cria o objeto Player, guarda-o localmente e abre a HomeActivity.

Partes essenciais do diagrama:

- Verificação do perfil local: a MainActivity chama getLocalPlayer(). O repositório procura o ID e o nickname nas preferências do Android.
- Alternativa — jogador existente: o repositório devolve um objeto Player, e a aplicação abre o ecrã principal.
- Alternativa — novo jogador: a aplicação apresenta o formulário. Depois da validação, cria o ID localmente, instancia Player e chama saveLocalPlayer(player).
- Persistência: o repositório grava o ID e o nickname em SharedPreferences, usando apply() para aplicar as alterações sem bloquear a interface.
- Validação do nickname: um nickname vazio mantém o jogador no formulário e apresenta uma indicação para o preencher.

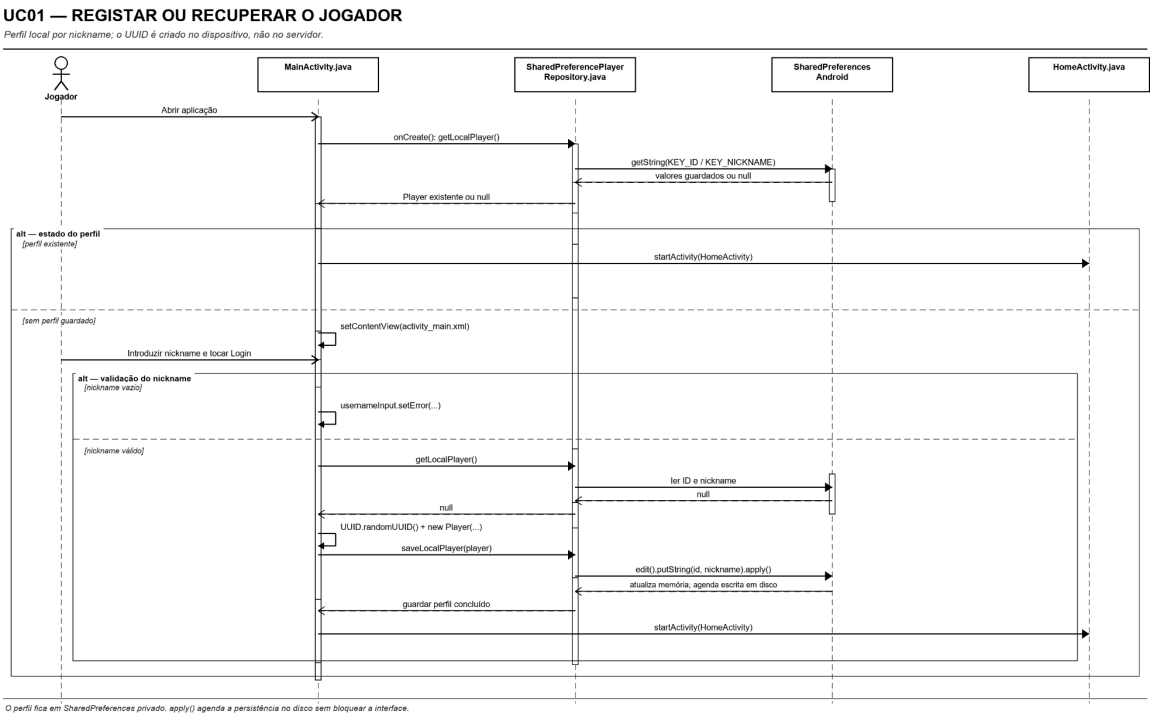


Figura 15 — Diagrama de sequência do UC01

8.1.2. UC02 — Jogar Partida

Para representar o fluxo sem sobrecarregar uma única imagem, o UC02 está dividido em quatro diagramas. O UC02-A mostra a deteção do abanão e o carregamento das perguntas. O UC02-A2 documenta a mensagem de inatividade e o cancelamento de um lançamento interrompido. O UC02-B representa as respostas, a pontuação e o fim da partida. O UC02-B2 mostra o comportamento da sessão quando a aplicação é pausada durante uma pergunta.

8.1.2.1. UC02-A — Abanar os dados e carregar um bloco

A figura 16 mostra que quando a GameActivity inicia uma partida, cria a GameSession e fornece-lhe o sensor físico e o listener da interface. A sessão começa no estado WAITING_FOR_ROLL e pede ao AndroidShakeSensor que comece a ouvir o acelerómetro.

Quando o jogador abana o dispositivo, o sensor calcula a força resultante do movimento. O abanão só é aceite quando a força excede 2,7 G, decorreu o intervalo mínimo de 500 ms desde o último abanão aceite e a sessão está em WAITING_FOR_ROLL. Se estas condições forem cumpridas, o sensor chama onShakeDetected(). A sessão muda para LOADING_BLOCK, desliga temporariamente o sensor e notifica a GameActivity.

A atividade sorteia os resultados visuais dos dois dados e determina a categoria e o formato da pergunta. Em seguida, inicia as animações Lottie e o som dos dados, e pede três perguntas à OpenTDB. O pedido é executado num ExecutorService: embora o método Retrofit execute() seja síncrono, é executado numa thread de trabalho e não bloqueia a interface. A animação e o pedido de rede decorrem em paralelo.

O RetrofitQuestionRepository transforma os registos recebidos em objetos Question e mistura as opções de resposta. A atividade só inicia o bloco quando recebeu exatamente três perguntas e terminaram as animações dos dois dados. Nesse momento, chama startNewBlock(questions), a sessão passa a IN_BREAK e a atividade apresenta o resumo durante 7 segundos antes de mostrar a pergunta.

Se o pedido falhar ou não devolver um bloco utilizável, a atividade mostra a mensagem “Opsss! The Dices fell of the Table. Try Again!”, repõe a sessão em WAITING_FOR_ROLL e reativa o sensor para permitir novo lançamento.

Partes essenciais do diagrama:

- Inicialização do sensor: a GameSession chama startListening() no AndroidShakeSensor.

- Validação do movimento: o sensor verifica a força G, o intervalo de debounce e o estado da sessão.
- Notificação do abanão: onShakeDetected() muda o estado para LOADING_BLOCK, desliga o sensor e notifica a GameActivity.
- Execução em paralelo: a atividade inicia as animações e o som enquanto o ExecutorService pede as perguntas à OpenTDB.
- Transformação da resposta: o repositório cria objetos Question e mistura as opções.
- Conclusão do bloco: startNewBlock(questions) só é chamado depois de receber as três perguntas e concluir as duas animações.
- Falha: a atividade apresenta a mensagem de erro e permite um novo lançamento.

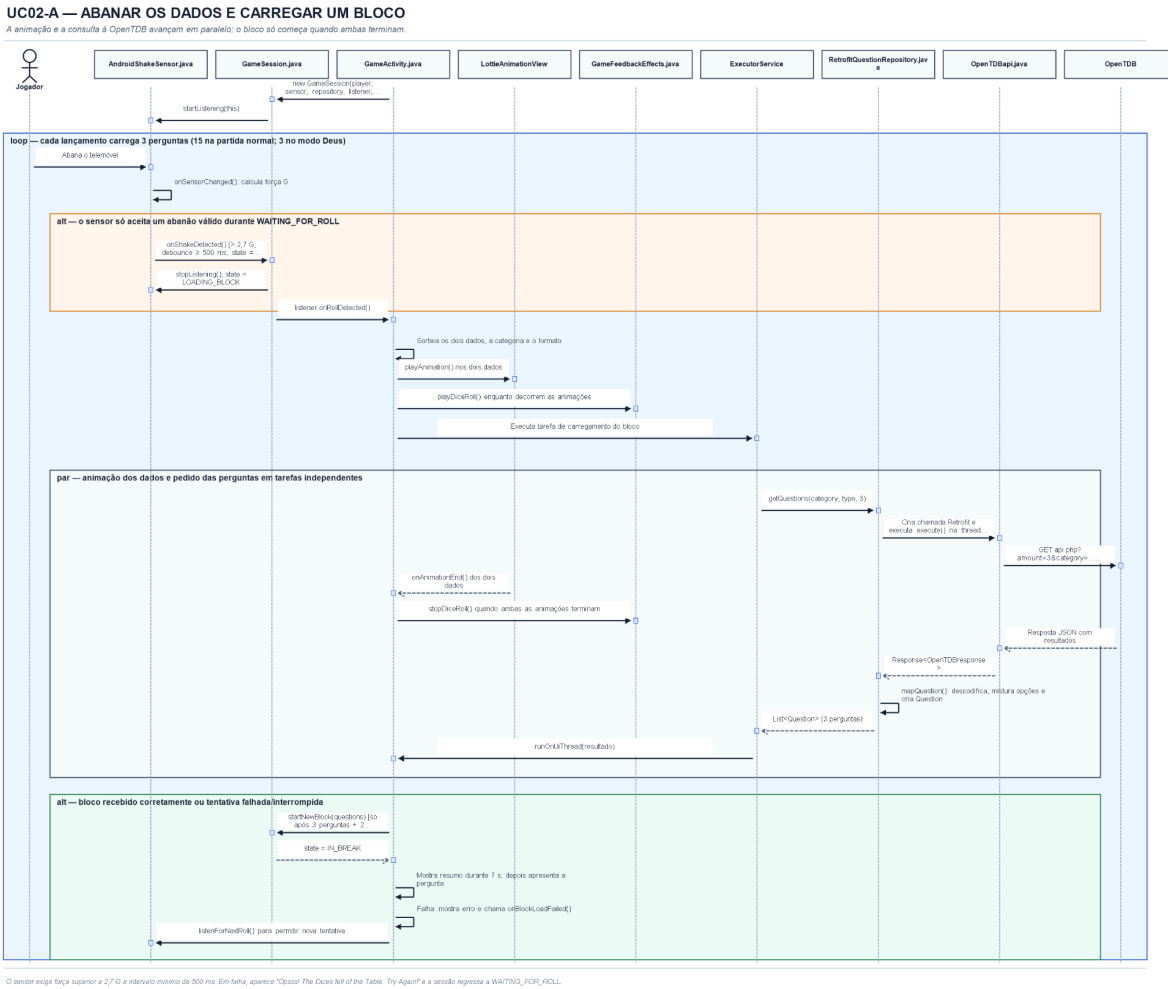


Figura 16 — Diagrama de sequência do UC02-A

8.1.2.2. UC02-A2 — Inatividade e interrupção no ecrã dos dados

Por outro lado, a figura 17, quando o ecrã dos dados é apresentado, a GameActivity agenda uma mensagem através de um Handler. Se passarem 15 segundos sem um lançamento e a atividade continuar visível com a sessão em WAITING_FOR_ROLL, a instrução é substituída por “Come on, roll it! So

you can become a Dice Master!". Se o jogador abanar os dados antes desse prazo, a atividade cancela a mensagem agendada.

Se a atividade for pausada enquanto a sessão está em LOADING_BLOCK, a tentativa é interrompida. A atividade invalida o identificador da tentativa, cancela a tarefa de trabalho e o pedido Retrofit, cancela as animações dos dois dados, pára o som e limpa os resultados parciais. Depois chama onBlockLoadFailed(), que permite à sessão voltar a WAITING_FOR_ROLL.

Ao regressar, a atividade apresenta novamente o ecrã dos dados e reativa o sensor. Se chegar entretanto um callback de uma tentativa antiga, o identificador inválido faz com que esse resultado seja descartado. Assim, uma resposta parcial de uma tentativa interrompida não é apresentada ao jogador.

Se a atividade for pausada enquanto a sessão já está em WAITING_FOR_ROLL, não existe um carregamento parcial para cancelar; o sensor é reativado quando o jogador regressa.

Partes essenciais do diagrama:

- Mensagem de inatividade: o Handler aguarda 15 segundos e só apresenta a mensagem se a sessão continuar à espera de um lançamento.
- Cancelamento do prompt: um abanão antes do prazo remove a mensagem pendente.
- Pausa da atividade: onPause() cancela temporizadores e impede a sessão de continuar a receber movimentos.
- Cancelamento do lançamento: se o estado for LOADING_BLOCK, são cancelados a tarefa, o pedido de rede, as animações e o som.
- Proteção contra callbacks antigos: uma resposta cuja tentativa já foi invalidada é descartada.
- Regresso ao jogo: em WAITING_FOR_ROLL, a atividade volta a apresentar o ecrã dos dados e reativa o sensor.

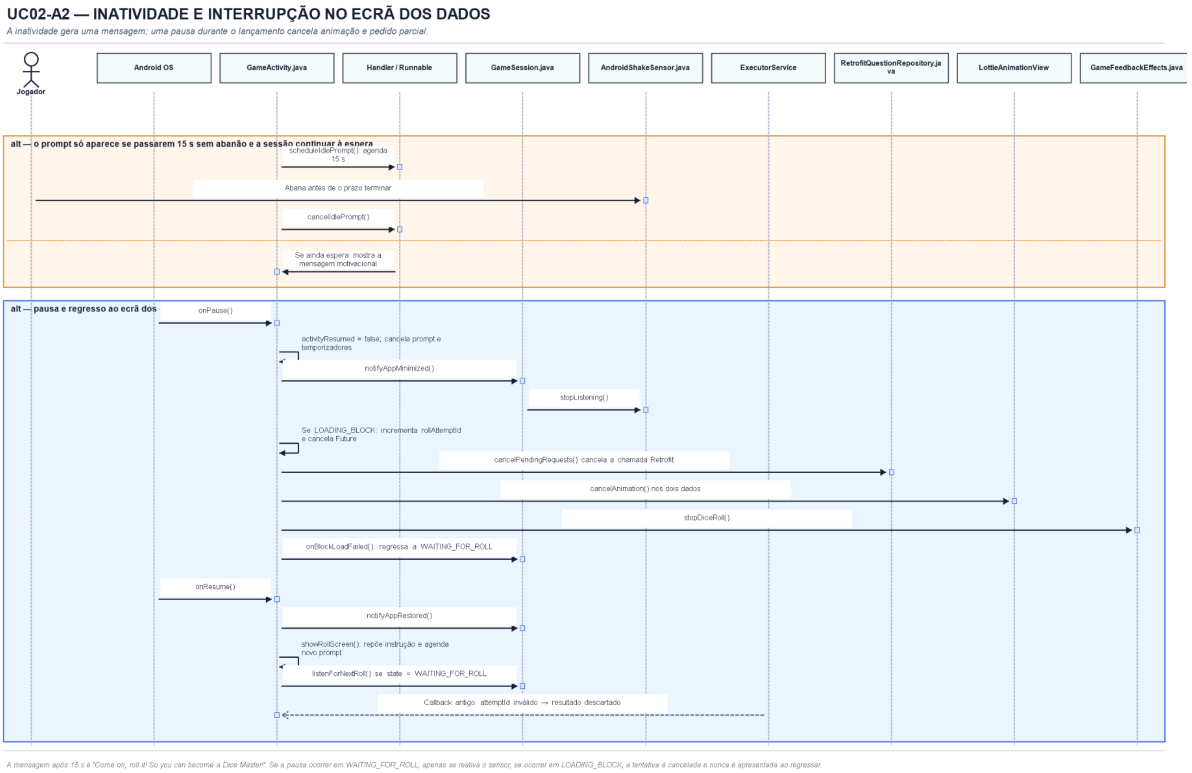


Figura 17 — Diagrama de sequência do UC02-A2

8.1.2.3. UC02-B — Responder às perguntas e terminar a partida

Quando uma pergunta é apresentada,a figura 18mostra a GameActivitya a iniciar um temporizador de 15 segundos. Se o jogador selecionar uma resposta antes do fim do tempo, a atividade chama processAnswer() na GameSession, enviando a resposta escolhida, a resposta correta e o tempo restante. Se o temporizador chegar a zero, chama processTimeout().

Numa partida normal, uma resposta correta dada antes de o tempo terminar aumenta o currentStreak, atualiza o bestStreak e acrescenta pontos calculados

pelo ScoreCalculator. A fórmula implementada é 10 + segundos restantes × 15 + streak atual × 25. Uma resposta errada ou um timeout repõe o streak atual a zero.

A atividade apresenta o feedback visual durante 3 segundos: destaca a resposta correta a verde e as incorretas a vermelho. Também pode emitir o efeito sonoro e a vibração correspondentes, conforme as definições do jogador. Depois desse intervalo, a atividade avança de acordo com o estado definido pela sessão.

A cada três perguntas, se ainda houver perguntas na partida, a sessão alterna entre IN_BREAK e WAITING_FOR_ROLL. No estado IN_BREAK, a atividade apresenta o resumo durante 7 segundos. No estado WAITING_FOR_ROLL, regressa ao ecrã dos dados e reativa o sensor. A partida normal termina após 15 perguntas.

No fim normal, a GameSession inicia um envio da pontuação para o repositório. Depois do período de feedback, a GameActivity guarda localmente a melhor pontuação e o melhor streak, e abre a LeaderboardActivity com os dados do resultado.

O modo Deus termina após 3 perguntas. A sessão mantém os valores de teste — pontuação de 999999 e streak de 15 — mas não guarda esses valores como recordes reais nem os submete como resultado normal.

Partes essenciais do diagrama:

- Alternativa entre resposta e timeout: o fluxo chama processAnswer(...) quando há uma resposta e processTimeout() quando o relógio termina.
- Cálculo da pontuação: só uma resposta correta com tempo restante positivo usa o ScoreCalculator.
- Atualização do streak: respostas erradas e timeouts interrompem o streak normal.
- Modo Deus: os valores de teste mantêm-se fixos durante as três perguntas.

- Feedback: a atividade apresenta cores, som e vibração durante 3 segundos.
- Progressão: advanceGame() decide entre IN_BREAK, WAITING_FOR_ROLL e FINISHED.
- Fim normal: guarda os recordes locais e encaminha o jogador para o leaderboard.
- Fim do modo Deus: abre o leaderboard com a indicação de modo Deus, sem guardar ou submeter os valores artificiais.

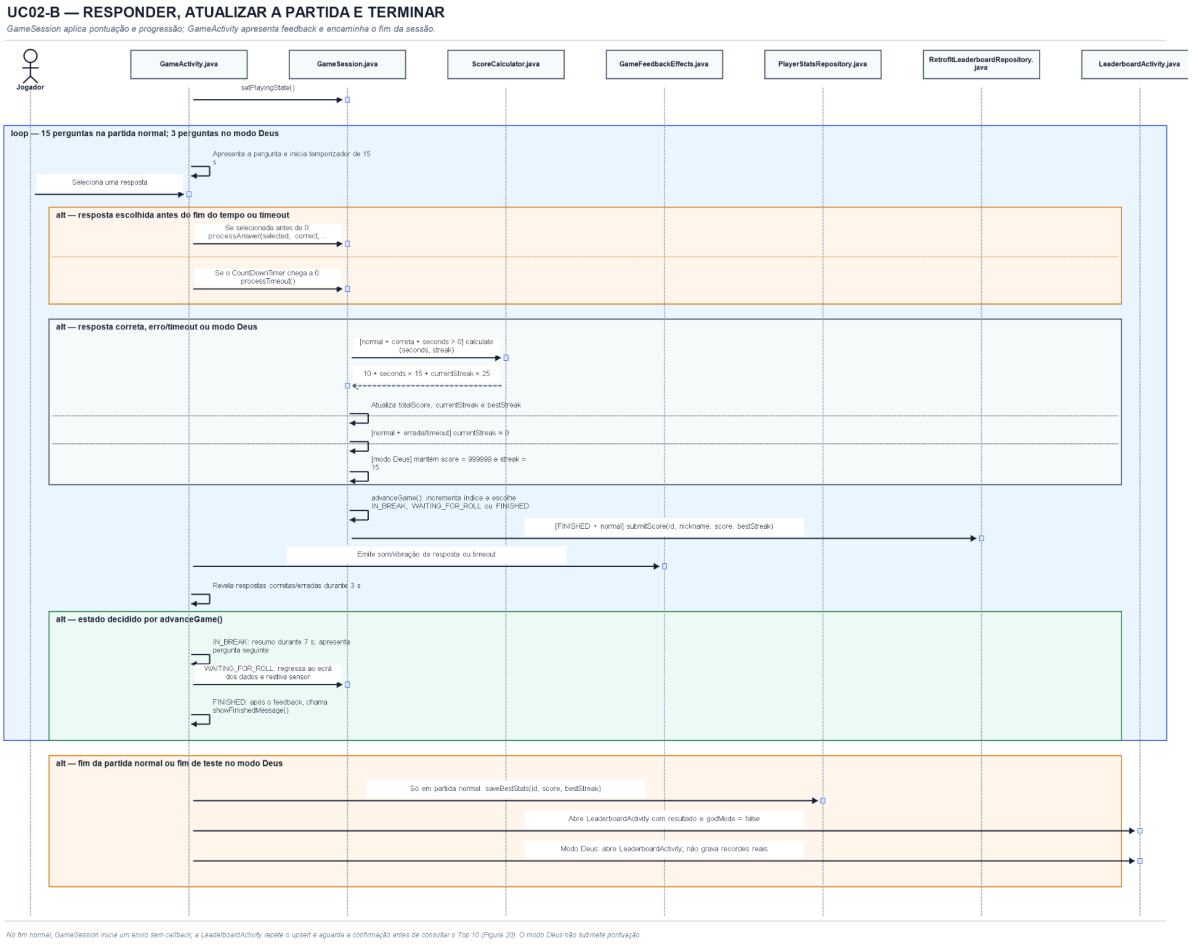


Figura 18 — Diagrama de sequência do UC02-B

Nota sobre o envio do resultado: no fluxo atual, a GameSession inicia um envio da pontuação ao terminar a partida. Depois, ao abrir o leaderboard após uma partida normal, a LeaderboardActivity também envia o resultado e espera a resposta antes de consultar o Top 10. Podem, portanto, ocorrer duas chamadas de envio para a mesma partida. O upsert usa o ID do jogador e não cria duas linhas para o mesmo ID, mas não elimina a segunda chamada HTTP. Este segundo envio está representado no UC03.

8.1.2.4. UC02-B2 — Pausa e regresso durante uma pergunta

A orientação da GameActivity está fixada em modo vertical no Manifest, evitando que a rotação recrie o ecrã durante a partida. Nesta figura, se a aplicação for pausada durante uma pergunta, a atividade cancela o CountDownTimer e regista o tempo restante. A GameSession assinala a pausa no AntiCheatTimer.

Quando a atividade regressa, chama notifyAppRestored(). Se o jogador esteve fora da aplicação durante mais de 10 segundos, a sessão aplica processTimeout(), quebra o streak normal e avança o estado da partida. A atividade reconstrói o ecrã correspondente ao novo estado.

Se o tempo fora não exceder 10 segundos, a atividade desconta do relógio os segundos decorridos, arredondados para cima. Se ainda restar tempo, reinicia o temporizador. Se o tempo chegar a zero, executa o timeout normal, apresenta o feedback e avança depois de 3 segundos.

Partes essenciais do diagrama:

- Pausa: a atividade cancela o temporizador e guarda o tempo restante.
- Registo anti-cheat: a sessão assinala a hora a que a aplicação foi minimizada.
- Saída superior a 10 segundos: a sessão aplica a penalização de timeout e avança o estado.
- Saída até 10 segundos: a atividade desconta o tempo decorrido e retoma o relógio.

- Relógio esgotado: é executado o timeout normal, com os efeitos de alarme e feedback.

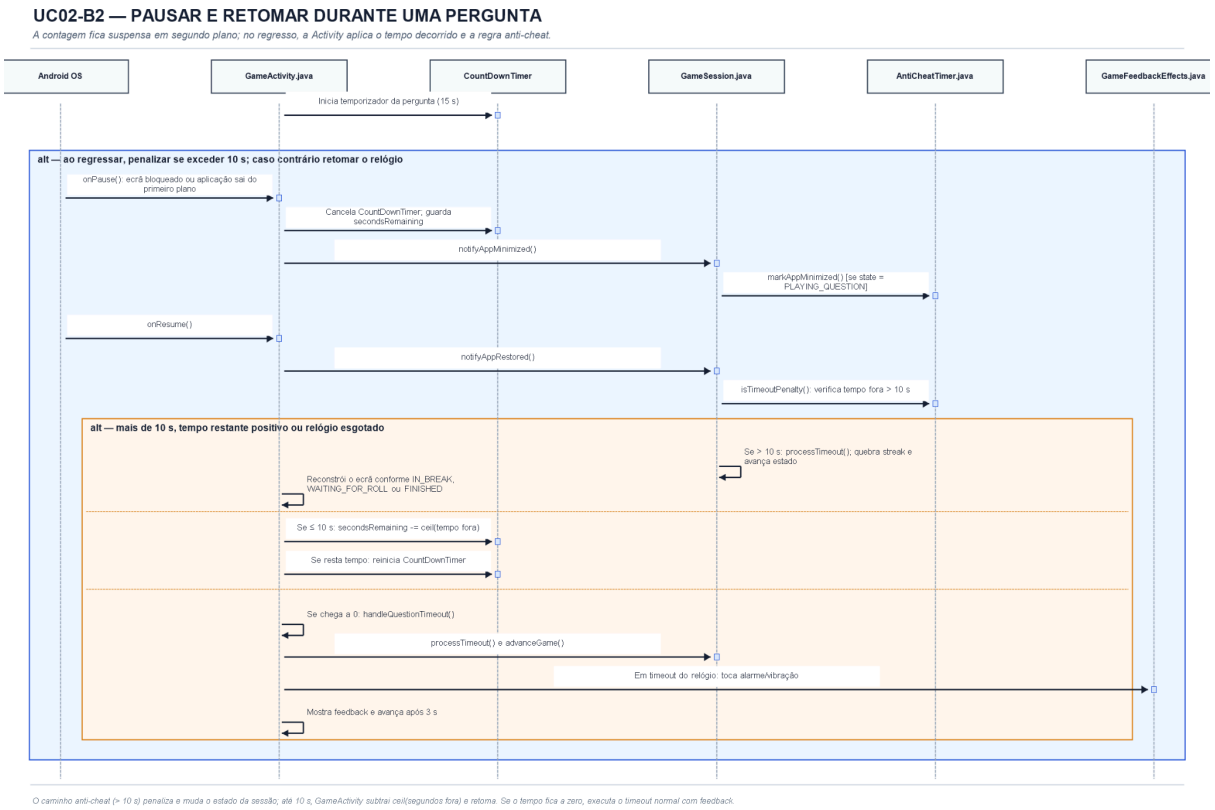


Figura 19 — Diagrama de sequência do UC02-B2

8.1.3. UC03 — Consultar o Leaderboard

Este diagrama, figura 20, representa a consulta do Top 10 global all-time. Não existe filtro semanal. A consulta devolve o ID, o nickname, a pontuação e o melhor streak. Os resultados são ordenados por pontuação decrescente; em caso de empate, o melhor streak e o nickname são usados como critérios seguintes, e a consulta está limitada a dez jogadores.

Quando o jogador abre o leaderboard a partir das definições, a HomeActivity inicia a LeaderboardActivity sem dados de uma partida. A atividade pede diretamente o Top 10 ao repositório.

Quando é aberta no fim de uma partida normal, a GameSession já iniciou um envio sem callback. A LeaderboardActivity faz também um upsert do resultado e espera a resposta. Só depois de o envio ter sucesso pede o Top 10. Se o envio falhar, apresenta a mensagem de erro e o botão para tentar novamente. No modo Deus, não é enviado um resultado de teste; a atividade consulta diretamente a tabela.

O RetrofitLeaderboardRepository usa enqueue() para executar os pedidos sem bloquear a interface. No envio, cria um PlayerJson e chama a SupaBaseApi, que faz um upsert por ID. Na leitura, recebe uma lista de PlayerJson, converte-a numa lista de objetos de domínio Player e comunica o resultado através de um callback.

Quando recebe a lista, a LeaderboardActivity atualiza o adaptador e, se tiver sido aberta depois de uma partida, verifica se o ID do jogador está entre os dez primeiros. Se estiver, apresenta “You are a Dice Master!”; caso contrário, apresenta “Keep it up, you have to roll your Dices better!”. Se a consulta ou o envio falhar, mostra “The Positions for the Dice Master's are not Available. Try Again!” e disponibiliza uma nova tentativa.

Partes essenciais do diagrama:

- Origem do ecrã: definições, fim de uma partida normal ou fim do modo Deus.

- Envio do resultado: depois de uma partida normal, a atividade aguarda o callback do upsert antes de consultar o ranking.
- Pedido assíncrono: getTop10(callback) usa Retrofit e enqueue() para não bloquear a interface.
- Consulta remota: a SupaBaseApi pede ao Supabase os dez primeiros registos.
- Conversão dos DTOs: o repositório converte PlayerJson em Player, incluindo o melhor streak.
- Atualização da interface: a atividade atualiza a lista e verifica se o ID do jogador está no Top 10.
- Erro e repetição: o botão Try again repete o envio se este falhou; se o envio já teve sucesso e só falhou a leitura, repete apenas a consulta.

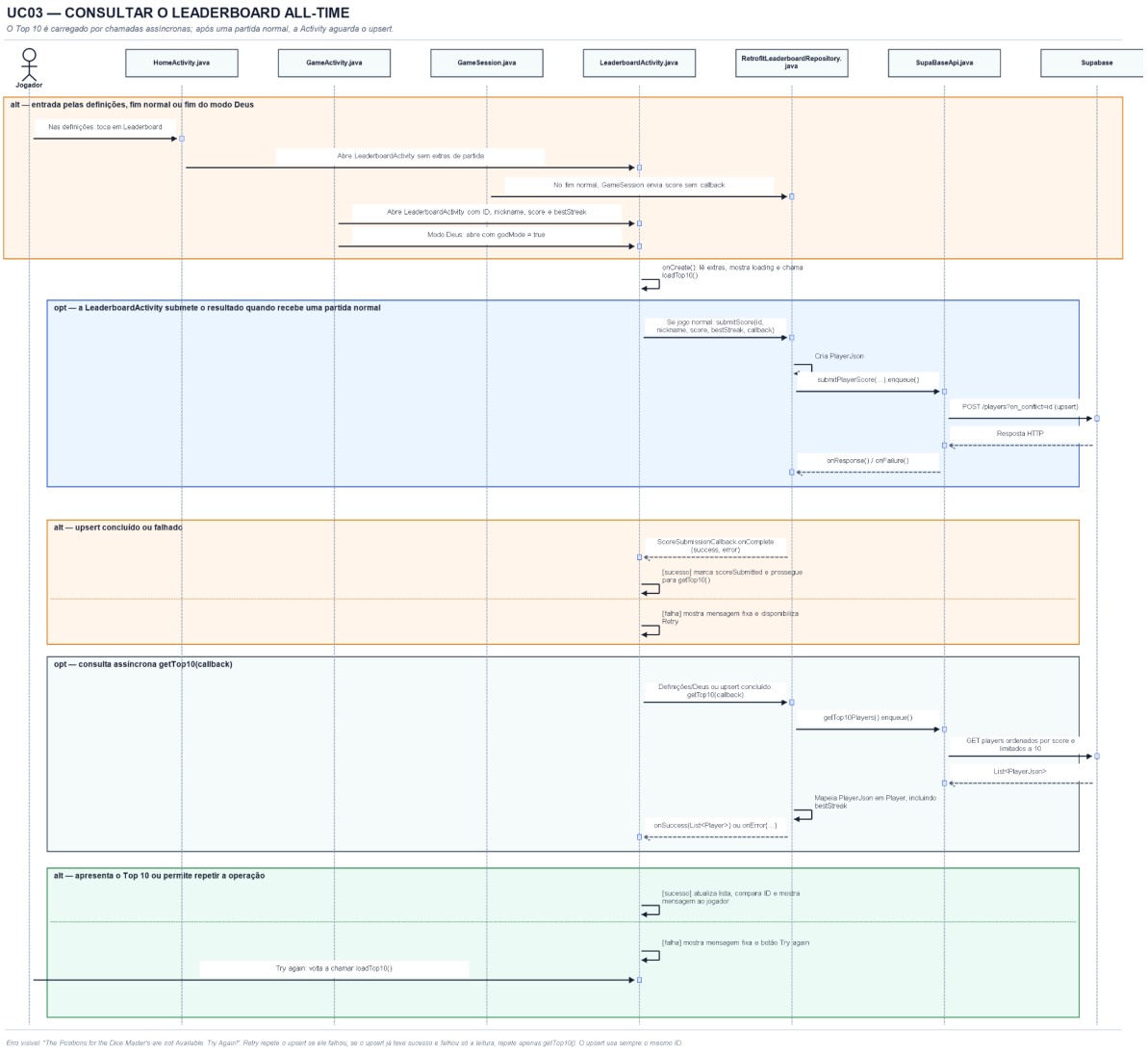


Figura 20 — Diagrama de sequência do UC03

8.1.4. UC04 — Ativar ou desativar o Modo Deus

O Modo Deus é ativado ou desativado através de cinco toques consecutivos no logótipo. O padrão é reconhecido pelo DebugManager, que verifica o intervalo entre toques — até 900 ms entre cada toque — e guarda o estado do modo nas preferências locais. A interface mostra "You are now the God Dice!" ao ativá-lo e "You are a Normal Dice!" ao desativá-lo.

Na figura, quando uma partida é iniciada, a GameActivity lê essa preferência e passa o estado do modo à GameSession. No Modo Deus, a sessão reduz a partida para um bloco de três perguntas e atribui valores de teste à pontuação e ao streak. O resultado não é guardado como recorde local nem submetido como resultado normal ao leaderboard.

Este fluxo não chama debugForceEndGame(): os cinco toques alternam a preferência do modo, e a partida curta decorre pelo fluxo normal de perguntas.

Partes essenciais do diagrama:

- Registo dos toques: a interface entrega cada toque no logótipo ao DebugManager.
- Validação do padrão: o gestor confirma que foram registados cinco toques dentro dos intervalos permitidos.
- Alternativa — padrão incompleto: a aplicação não altera o modo.
- Alternativa — padrão completo: o gestor alterna a preferência persistente e a interface mostra a mensagem correspondente.
- Configuração da partida: ao iniciar uma partida, a GameActivity lê a preferência e configura a GameSession.
- Partida de teste: a sessão executa um bloco de três perguntas com pontuação e streak de teste, sem registar o resultado como pontuação normal.

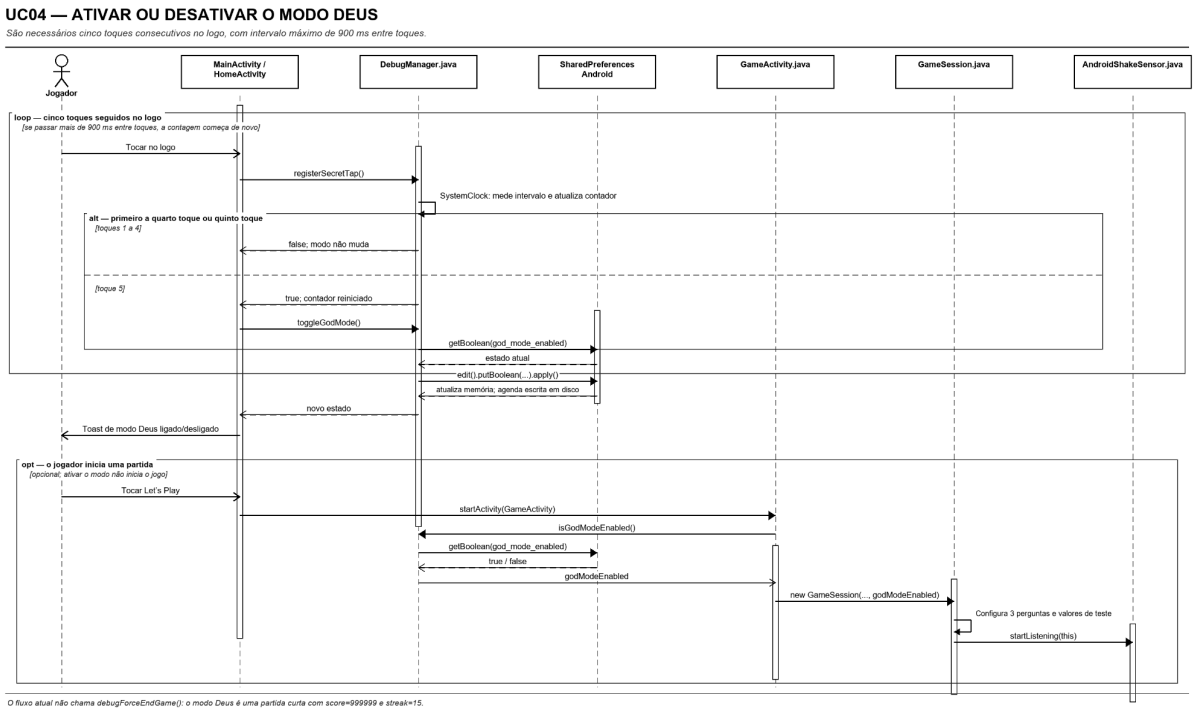


Figura 21 — Diagrama de sequência do UC04

8.1.5. UC05 — Consultar perfil, definições e terminar sessão

O perfil e as definições são apresentados no painel lateral da HomeActivity; não existe uma ProfileActivity separada. Mostra-se, na figura 22, que o nickname é obtido a partir do jogador carregado localmente. A pontuação máxima e o streak máximo são consultados no PlayerStatsRepository, que mantém esses valores localmente, associados ao ID do jogador.

O painel também apresenta opções para controlar os sons, a música e a vibração. Estas preferências são guardadas no dispositivo e aplicadas pelos

componentes correspondentes. O botão do leaderboard abre a LeaderboardActivity.

Ao escolher terminar sessão, a aplicação pede confirmação. Se o jogador confirmar, o SharedPreferencePlayerRepository remove apenas o ID e o nickname guardados localmente. A aplicação abre então a MainActivity e limpa a tarefa anterior, fazendo regressar o jogador ao ecrã de início de sessão. Os recordes locais, as preferências de áudio e vibração e o registo remoto no Supabase não são apagados por este fluxo.

Partes essenciais do diagrama:

- Leitura do perfil: a HomeActivity usa o objeto Player carregado localmente para apresentar o nickname.
- Leitura dos recordes: o PlayerStatsRepository lê a pontuação máxima e o streak máximo, guardados separadamente e associados ao ID do jogador.
- Leitura das definições: as opções de som, música e vibração são obtidas das preferências locais.
- Alteração de uma definição: a nova opção é guardada e aplicada pelo componente de áudio ou vibração correspondente.
- Consulta do leaderboard: o botão inicia a LeaderboardActivity.
- Terminar sessão: após confirmação, a aplicação remove o ID e o nickname locais e abre a MainActivity, que volta a apresentar o formulário de início de sessão.

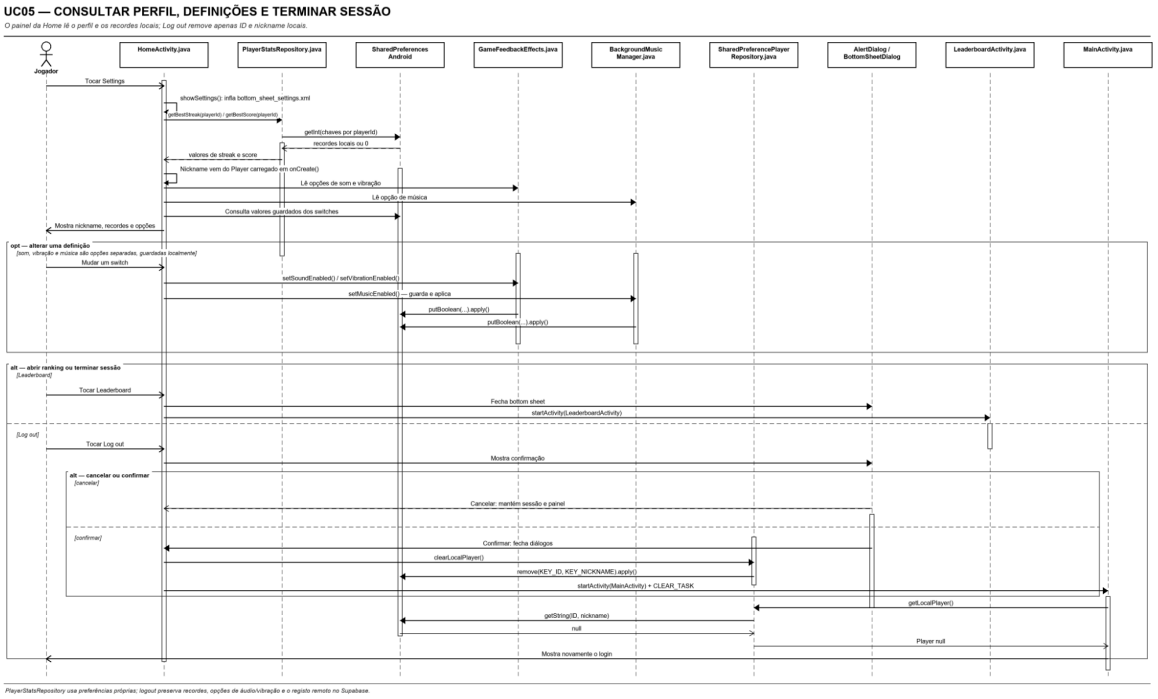


Figura 22 — Diagrama de sequência do UC05

9. Interface de Utilizador - Mock Ups (UI)

Os mockups são representações visuais da interface de uma aplicação. Permitem visualizar a organização dos elementos de cada ecrã — como textos, botões, imagens e indicadores — antes ou durante a implementação. Ao contrário da aplicação funcional, um mockup é estático: não executa ações nem contém a lógica do jogo.

A sua utilização ajuda a planear a navegação, a verificar se a informação está organizada de forma clara e a manter uma aparência coerente entre os diferentes ecrãs. Também facilita a identificação de problemas de disposição e a discussão de alterações antes de estas serem aplicadas no código. No caso da

Dice Mania, os mockups permitem apresentar o percurso principal do jogador, desde a entrada na aplicação até à consulta do ranking.

Foram considerados seis ecrãs principais:

9.1. Login:

Permite ao jogador introduzir o nickname necessário para iniciar a utilização da aplicação. A interface deve tornar esta ação evidente e indicar quando o campo está vazio.

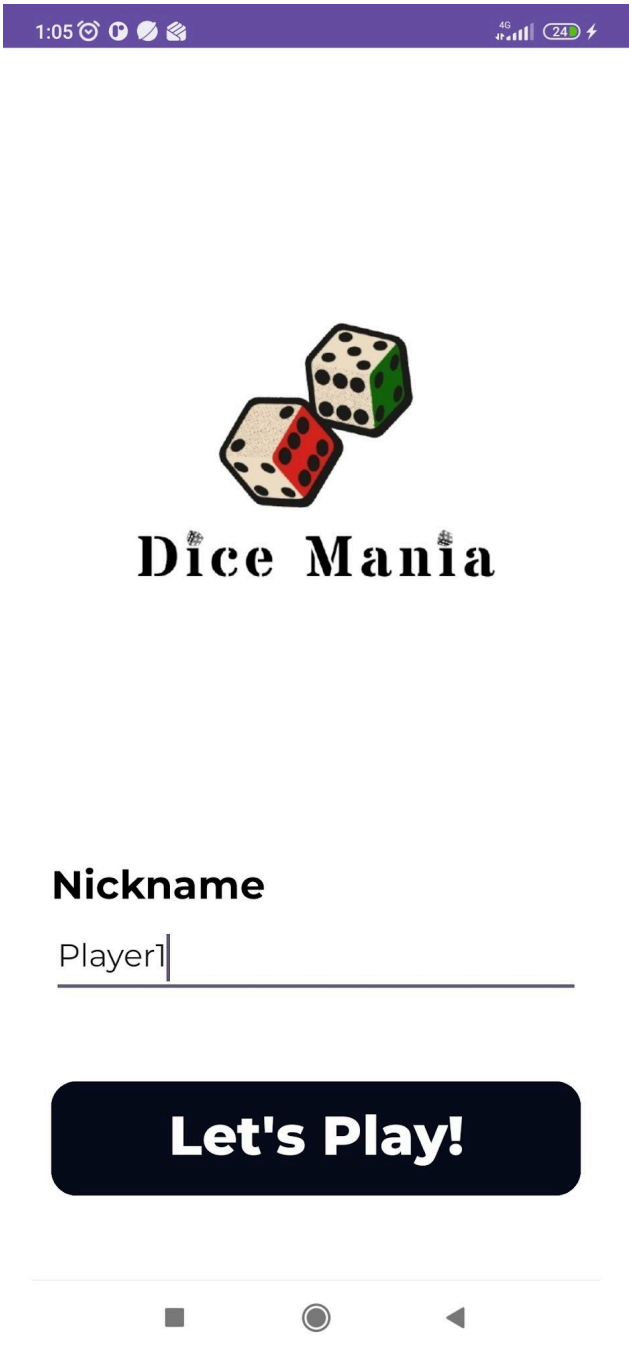


Figura 23 - Ecrã de Log in

9.2. Home:

Apresenta a identidade visual da DiceMania e o botão Let's Play, que inicia uma partida. Inclui também o acesso às definições, onde o jogador pode consultar o seu perfil e os recordes, ajustar as opções de som e vibração e abrir o leaderboard.

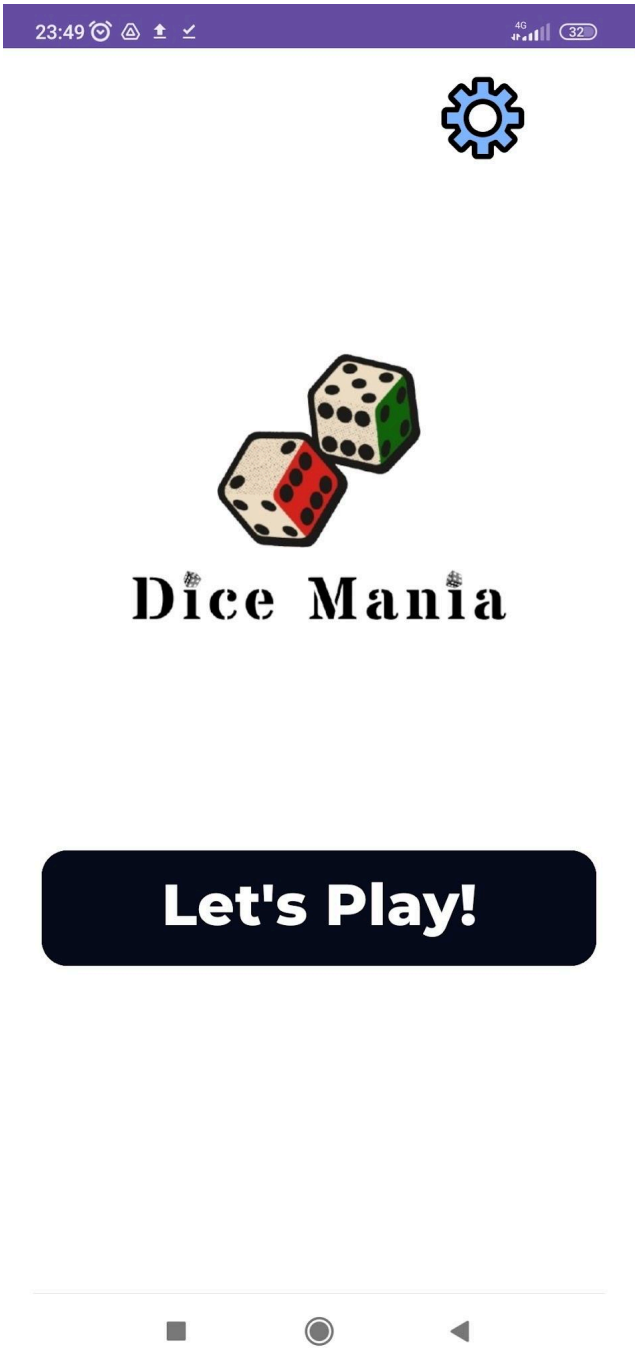


Figura 24 - Ecrã Home

9.3. Lançamento dos dados:

Convida o jogador a abanar o telemóvel. Os dados animados tornam visível a seleção da categoria e do formato das perguntas, enquanto a aplicação carrega o bloco seguinte.

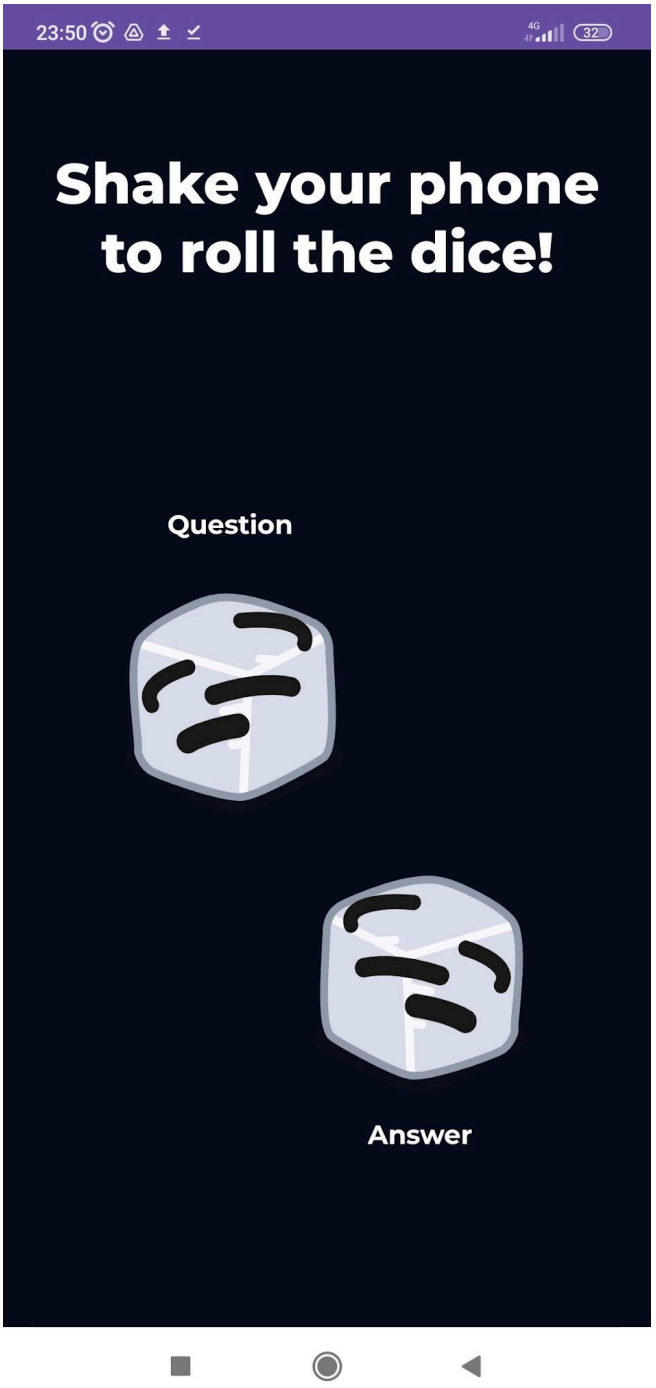


Figura 25 - Ecrã de Lançamento dos Dados

9.4. Resumo da ronda:

Apresenta o resultado dos dados e as informações da ronda, como o tipo de pergunta, o tipo de resposta, o progresso, a pontuação e o streak. Este ecrã prepara o jogador para a pergunta seguinte.

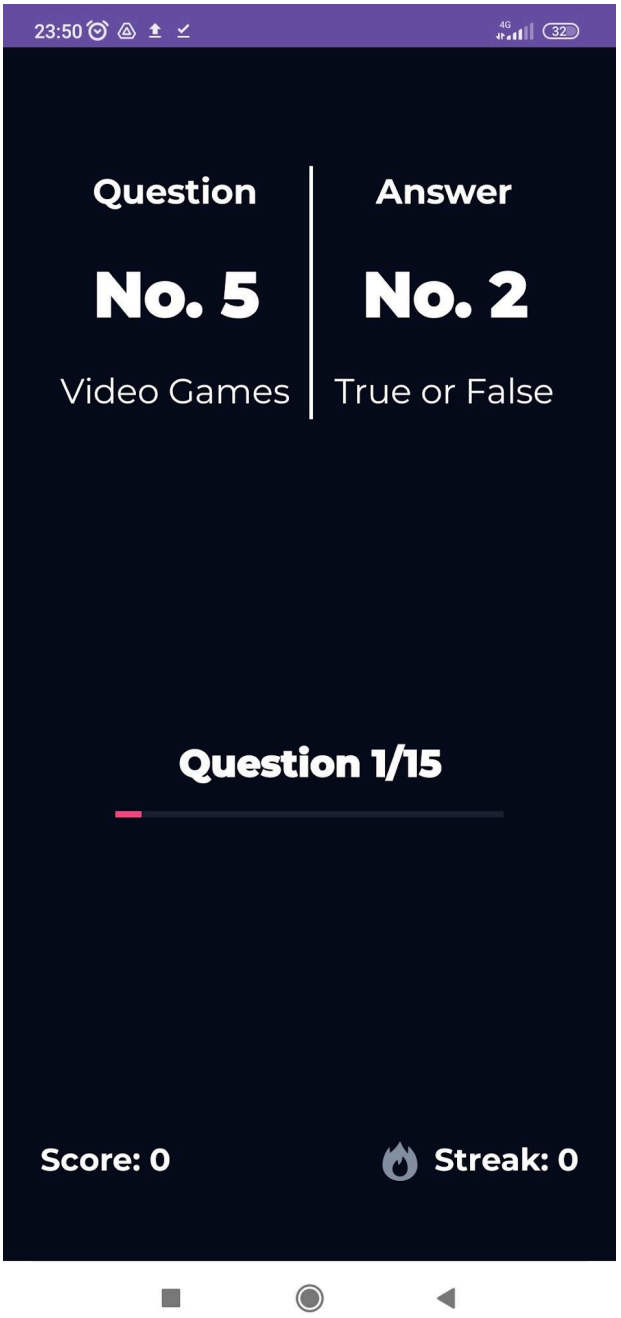


Figura 26 - Ecrã de Resumo

9.5. Pergunta:

Mostra o enunciado, o tempo disponível e as opções de resposta. A disposição deve facilitar a leitura e a seleção rápida. Depois da resposta, as opções corretas e incorretas são destacadas visualmente.

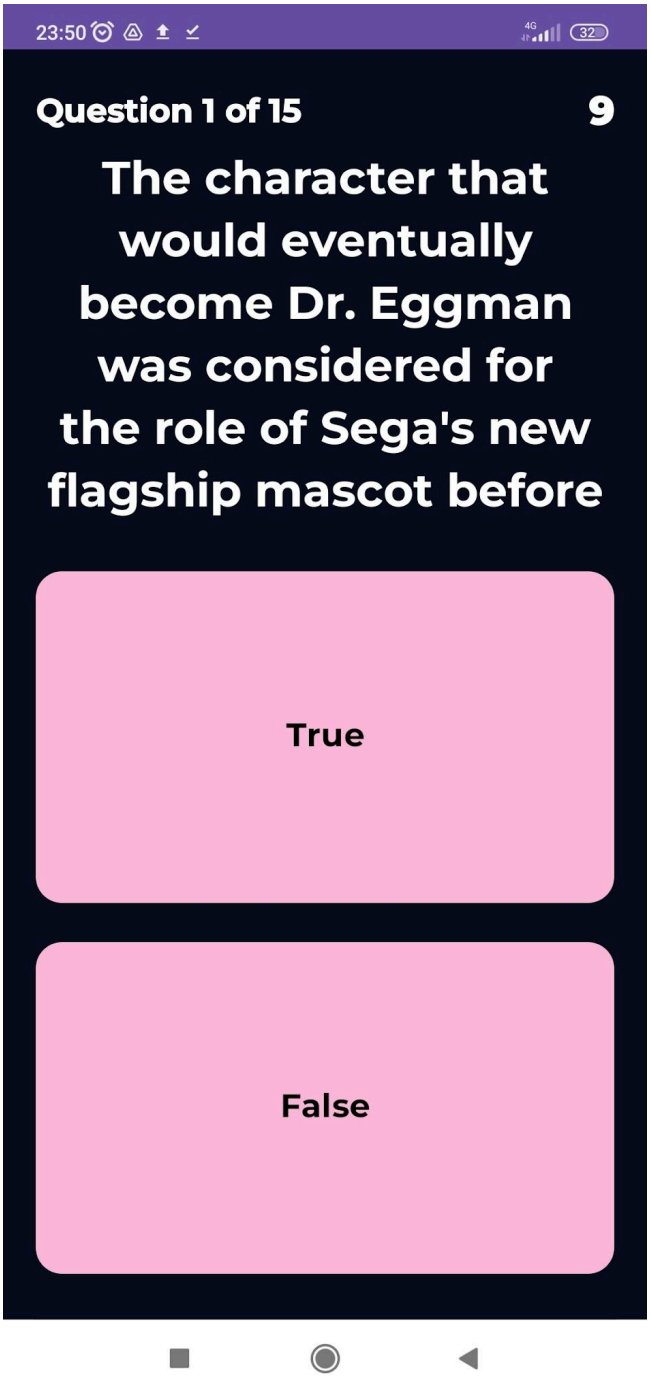


Figura 27 - Ecrã de Pergunta

9.6. Leaderboard:

Apresenta os dez melhores resultados, com a posição, o nickname e a pontuação de cada jogador. No final de uma partida, também informa o jogador se o seu resultado aparece no Top 10.

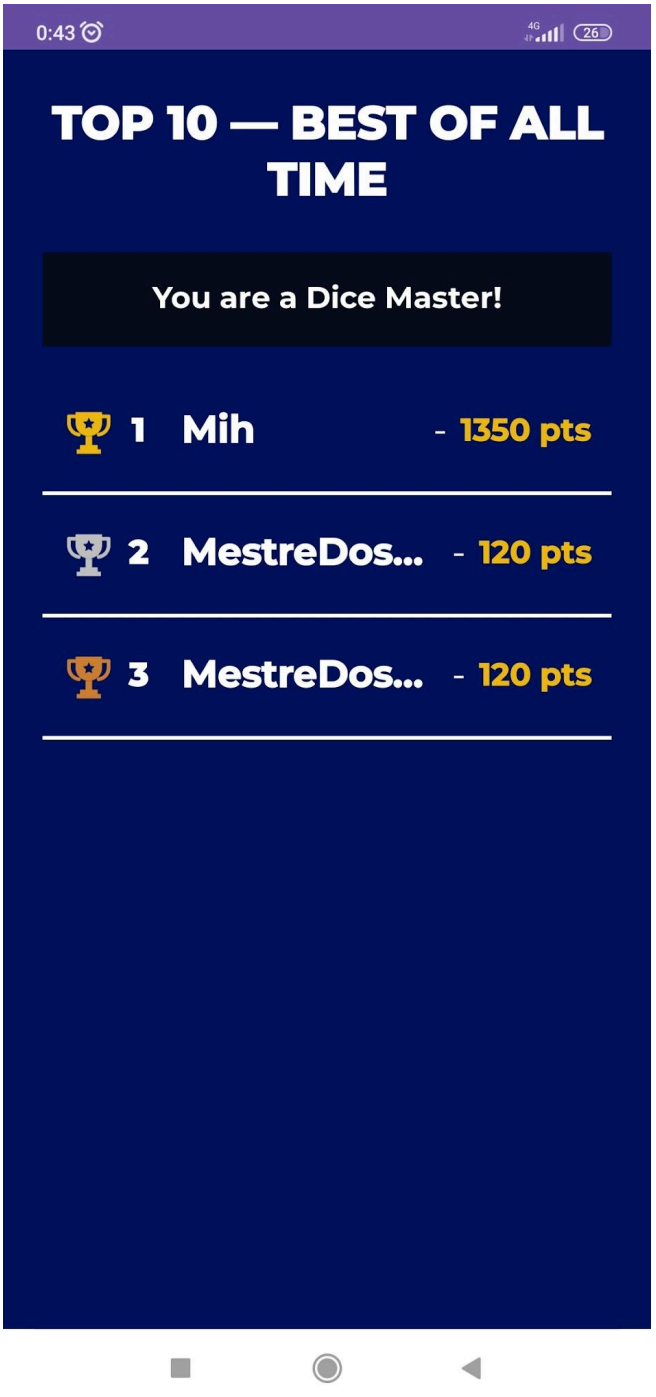


Figura 28 - Ecrã de Leaderboard

10. Implementação

Nesta secção apresentam-se alguns aspetos práticos e algoritmos utilizados na aplicação, com exemplos da deteção de movimentos, da conversão das perguntas recebidas da OpenTDB e da comunicação com o Supabase.

Os fragmentos de código ilustram três responsabilidades distintas: interagir com o acelerómetro, transformar dados externos em objetos do domínio e comunicar com o servidor sem bloquear a interface. Os exemplos correspondem ao código atual da aplicação.

10.1. Deteção do abanão do dispositivo

A classe AndroidShakeSensor comunica com o acelerómetro através das APIs do Android. O método onSensorChanged() recebe as medições dos eixos X, Y e Z, calcula a aceleração resultante e aplica um limiar de ativação e um intervalo mínimo entre eventos consecutivos.

Código 1 — Cálculo da aceleração e validação do abanão:

```
1 | @Override

2 | public void onSensorChanged(SensorEvent event)
{

3 |     if (callback == null) {

4 |         return;

5 |     }

6 |
```

```
7 |     float x = event.values[0];

8 |     float y = event.values[1];

9 |     float z = event.values[2];

10 |

11 |     float gX = x /
SensorManager.GRAVITY_EARTH;

12 |     float gY = y /
SensorManager.GRAVITY_EARTH;

13 |     float gZ = z /
SensorManager.GRAVITY_EARTH;

14 |

15 |     float gForce = (float) Math.sqrt(

16 |             gX * gX + gY * gY + gZ * gZ

17 |     );

18 |

19 |     if (gForce > SHAKE_THRESHOLD_GRAVITY) {

20 |         final long now =
System.currentTimeMillis();

21 |

22 |         if (shakeTimeStamp +
SHAKE_SLOP_TIME_MS > now) {

23 |             return;

24 |         }
```

```
25 |
26 |             shakeTimeStamp = now;
27 |             callback.onShakeDetected();
28 |         }
29 | }
```

Análise da implementação

- O método recebe um SensorEvent fornecido pelo Android. Se não existir um ShakeCallback registado, termina imediatamente sem processar o evento.
- Os valores dos três eixos do acelerómetro são divididos por SensorManager.GRAVITY_EARTH, convertendo-os numa medida relativa à gravidade terrestre.
- A expressão com Math.sqrt() calcula a magnitude da aceleração resultante nos três eixos. Esta magnitude é comparada com a constante SHAKE_THRESHOLD_GRAVITY, definida como 2.7F.
- Depois de ultrapassado o limiar, o método compara o instante atual com o último abanão aceite. Se ainda não tiver passado o intervalo definido por SHAKE_SLOP_TIME_MS — atualmente, 500 milissegundos — o evento é ignorado. Este intervalo funciona como um mecanismo de debounce, limitando eventos repetidos; não é um filtro de histerese.
- Se o movimento for aceite, o método atualiza shakeTimeStamp e chama callback.onShakeDetected().
- A sessão recebe o callback e verifica se está no estado WAITING_FOR_ROLL. Se estiver, muda para LOADING_BLOCK, interrompe temporariamente o sensor e informa a GameActivity de que o lançamento foi aceite.

O sensor é registado com a frequência SENSOR_DELAY_UI. Quando deixa de ser necessário, stopListening() remove o listener do SensorManager e limpa o callback.

10.2. Conversão das perguntas recebidas da OpenTDB

A classe RetrofitQuestionRepository é responsável por pedir perguntas à OpenTDB e converter os objetos recebidos em formato JSON para objetos Question, utilizados pelo jogo. A conversão combina a resposta correta com as respostas incorretas e baralha a ordem das opções.

Código 2 — Construção das perguntas do domínio:

```
1 | Call<OpenTDBresponse> chamada =

2 |         api.getQuestions(amount, categoryId,
typeString);

3 |

4 | Response<OpenTDBresponse> resposta =
chamada.execute();

5 |

6 | if (resposta.isSuccessful() && resposta.body()
!= null) {

7 |     for (OpenTDBresponse.QuestionJson Json

8 |             : resposta.body().results) {

9 |

10 |         List<String> respostasMisturadas = new
ArrayList<>();
```

```
11 |
respostasMisturadas.add(Json.correctAnswer);
12 |
13 |         if (Json.incorrectAnswers != null) {
14 |
respostasMisturadas.addAll(Json.incorrectAnswers);
15 |         }
16 |
17 |
Collections.shuffle(respostasMisturadas);
18 |
19 |         Question perguntaLimpa = new Question(
20 |                 category,
21 |                 type,
22 |                 Json.question,
23 |                 Json.correctAnswer,
24 |                 respostasMisturadas
25 |         );
26 |
27 |         perguntasProntas.add(perguntaLimpa);
28 |     }
29 | }
```

```
30 |

31 | return perguntasProntas;
```

Análise da implementação

- Antes do pedido, o repositório converte a categoria do jogo no identificador esperado pela OpenTDB e o tipo da pergunta para o formato da API: "multiple" ou "boolean".
- A chamada api.getQuestions(...) cria o pedido Retrofit. A chamada execute() aguarda pela resposta e, dentro deste método, é síncrona.
- O repositório percorre a lista results recebida no objeto OpenTDBresponse. Cada item é um DTO QuestionJson, cujos campos são preenchidos a partir do JSON.
- Para cada pergunta, o código cria uma lista de opções, adiciona a resposta correta e, se existirem, as respostas incorretas.
- Collections.shuffle() altera aleatoriamente a ordem das opções. Isto evita que a resposta correta apareça sempre na mesma posição; não constitui, por si só, uma proteção completa contra batota.
- O construtor de Question recebe a categoria, o tipo, o enunciado, a resposta correta e a lista baralhada. Assim, o resto do jogo utiliza o modelo do domínio, sem depender diretamente do DTO da API.
- Se ocorrer uma exceção durante o pedido ou a conversão, o repositório apanha-a e devolve a lista disponível, que pode estar vazia.

Embora execute() seja síncrono, a GameActivity chama getQuestions() dentro de um ExecutorService de thread única. Por isso, a espera pela resposta da OpenTDB decorre fora da thread da interface. Quando o resultado está pronto, a atividade usa runOnUiThread() para continuar a atualização do ecrã. Esta distinção é importante: o método do repositório usa execute(), enquanto é a atividade que o coloca numa thread de trabalho.

10.3. Envio assíncrono de resultados para o Supabase

A classe RetrofitLeaderboardRepository implementa o envio dos resultados e a consulta do Top 10. O envio atual inclui o ID e o nickname do jogador, a pontuação final e o streak máximo. Os dados são preparados num objeto PlayerJson e enviados através de enqueue(), que executa o pedido sem obrigar a interface a aguardar pela resposta.

Código 3A — Campos do DTO `PlayerJson`:

```
1 | @SerializedName("id")
2 | private String id;
3 |
4 | @SerializedName("nickname")
5 | private String nickname;
6 |
7 | @SerializedName("score")
8 | private int score;
9 |
10 | @SerializedName("best_streak")
11 | private int bestStreak;
```

Código 3B — Envio do resultado ao Supabase:

```
1 | public void submitScore(
2 |         String playerId,
3 |         String nickname,
4 |         int finalScore,
5 |         int bestStreak,
6 |         ScoreSubmissionCallback
submissionCallback
```

```
7 | ) {
8 |     PlayerJson playerJson = new PlayerJson(
9 |             playerId,
10 |             nickname,
11 |             finalScore,
12 |             bestStreak
13 |     );
14 |
15 |     supaBaseApi.submitPlayerScore(playerJson)
16 |             .enqueue(new Callback<Void>() {
17 |                 @Override
18 |                 public void onResponse(
19 |                         Call<Void> call,
20 |                         Response<Void>
response
21 |                 ) {
22 |                     if
(response.isSuccessful()) {
23 |                         if (submissionCallback
!= null) {
24 |
submissionCallback.onComplete(true, null);
25 |                         }
26 |                     } else {
27 |                         String errorMessage =
28 |                                 "HTTP " +
response.code();
29 |
30 |                         if (submissionCallback
!= null) {
31 |
submissionCallback.onComplete(
32 |                                         false,
33 |
errorMessage
34 |                                 );
35 |                         }
36 |                     }
```

```
37 |                 }
38 |
39 |                 @Override
40 |                 public void onFailure(
41 |                         Call<Void> call,
42 |                         Throwable t
43 |                 ) {
44 |                     if (submissionCallback !=
null) {
45 |
submissionCallback.onComplete(
46 |                                     false,
47 |                                     t.getMessage()
48 |                             );
49 |                     }
50 |                 }
51 |             });
52 | }
```

Análise da implementação

- PlayerJson é um DTO usado para transportar os dados entre a aplicação e o Supabase. A anotação @SerializedName("best_streak"), por exemplo, permite que o Gson associe a propriedade Java bestStreak à coluna best_streak.
- O repositório cria um PlayerJson com o ID, o nickname, a pontuação final e o streak máximo do jogador.
- supaBaseApi.submitPlayerScore(playerJson) prepara o pedido HTTP. A chamada enqueue() inicia-o de forma assíncrona e permite que a aplicação continue a responder enquanto aguarda pelo servidor.
- onResponse() é executado quando chega uma resposta HTTP. A resposta pode representar sucesso ou erro; por isso, o método verifica response.isSuccessful(). Se houver um erro HTTP, o callback recebe o respetivo código.

- onFailure() trata falhas na execução ou no transporte do pedido, por exemplo, problemas de ligação. A mensagem é devolvida ao callback.
- ScoreSubmissionCallback permite que o componente que iniciou o envio saiba quando este terminou. A LeaderboardActivity pode usar esse resultado para carregar a tabela depois de um envio bem-sucedido ou apresentar uma opção de nova tentativa em caso de erro.
- A API identifica o jogador pelo ID e envia o pedido para inserir ou atualizar o respetivo registo. A preservação dos valores máximos de pontuação e streak depende também das regras configuradas na base de dados do Supabase.
- A consulta do Top 10 também usa enqueue(). Quando recebe a resposta, o repositório converte cada PlayerJson num objeto Player e entrega a lista à interface através do callback do leaderboard.

11. Testes

Com o intuito de garantir a robustez técnica, a integridade da Clean Architecture e o cumprimento dos Requisitos Não Funcionais (FURPS+), o projeto DiceMania foi sujeito a protocolos de testes estruturados, divididos entre Testes Unitários isolados e Testes Funcionais Sistémicos.

11.1. Testes Unitários (Unit Tests)

Os testes unitários (recorrendo ao framework nativo JUnit) focaram-se na validação matemática e temporal do coração da aplicação (camada de Domínio e Utilitários), garantindo que os algoritmos respondem de forma previsível e determinística a todos os vetores de entrada, sem necessidade de carregar os aspetos visuais do Android (Activities).

11.1.1. Validação do Anti-Batota (AntiCheatTimer):

- Propósito: Garantir o cumprimento do RNF04.
- Método Testado: Invocou-se o método markAppMinimized() para simular a minimização do jogo e gerou-se artificialmente um atraso na Thread superior a 10.000 milissegundos (10 segundos).
- Resultado: Ao invocar subsequentemente isTimeoutPenalty(), o utilitário devolveu estritamente o valor booleano true, validando a expulsão correta do jogador perante manipulação temporal no background.

```
CÓDIGO:
public class AntiCheatTimerTest {

    @Test
    public void
testBatota_Ultrapassa10Segundos_AplicaPenalizacao()
throws InterruptedException {
        // 1. Arrange: Instanciar a ferramenta
```

```
        AntiCheatTimer timer = new
AntiCheatTimer();

        // 2. Act: Simular a minimização da app
pelo jogador
        timer.markAppMinimized();

        // Simular o tempo a passar no background
(10.050 milissegundos)
        // Usamos Thread.sleep para atrasar a
execução do teste
        Thread.sleep(10050);

        // 3. Assert: Verificar se o sistema deteta
a infração
        boolean isPenalty =
timer.isTimeoutPenalty();

        assertTrue("O jogador deveria ser
penalizado por demorar mais de 10 segundos",
isPenalty);
    }
}
```

Confirmaçao:

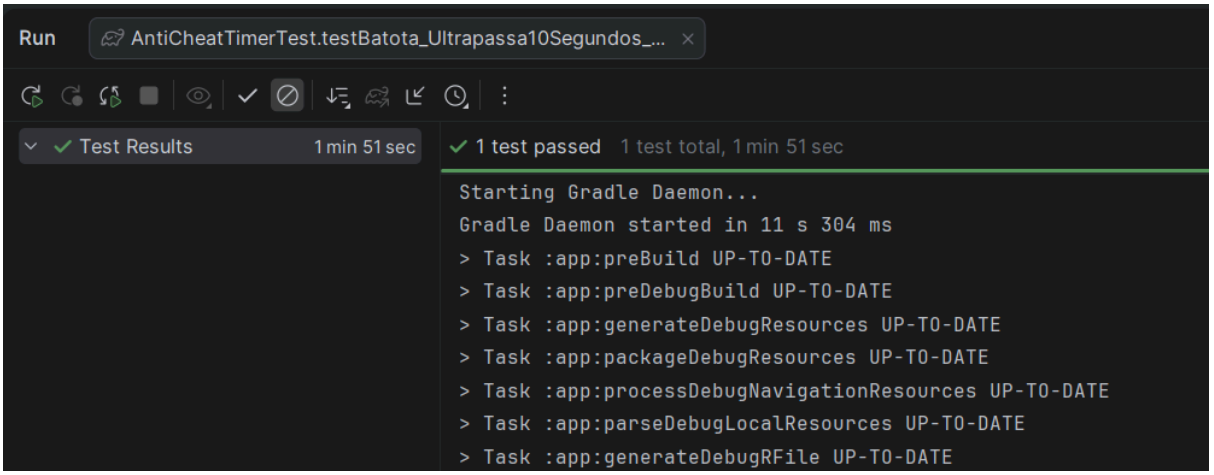


Figura 29 - Confirmação de sucesso do teste da Anti-Batota

11.1.2. Validação do Hardware Matemático (AndroidShakeSensor):

- Propósito: Validar o teorema de Pitágoras e a Histerese temporal.
- Método Testado: Injetou-se manualmente vetores flutuantes de força (X,Y,Z) na fórmula de cálculo (gForce = Math.sqrt($gX^2 + gY^2 + gZ^2$)). Foram forçados disparos simultâneos.
- Resultado: O sistema bloqueou eficazmente as chamadas sequenciais antes do limite predefinido de 500ms (SHAKE_SLOP_TIME_MS), provando que a bateria e o ciclo HTTP ficam blindados a spam de sensores.

CÓDIGO:

```
public class SensorMathTest {

    @Test
    public void testGForceCalculation_ValidShake()
{
```

```
        // 1. Arrange: Simular valores recebidos de
um abanão forte (ex: Eixo X muito elevado)
        float x = 30.0f;
        float y = 2.0f;
        float z = 1.0f;
        float gravityEarth = 9.8f; //
SensorManager.GRAVITY_EARTH

        // 2. Act: Executar a lógica matemática
presente no AndroidShakeSensor
        float gX = x / gravityEarth;
        float gY = y / gravityEarth;
        float gZ = z / gravityEarth;

        float gForce = (float) Math.sqrt(gX * gX +
gY * gY + gZ * gZ);

        // 3. Assert: Garantir que o resultado
supera o limiar de 2.7G para iniciar o jogo
        assertTrue("A força G (calculada: " +
gForce + ") deve ser maior que o limiar de 2.7",
gForce > 2.7f);
    }
}
```

Confirmaçao:

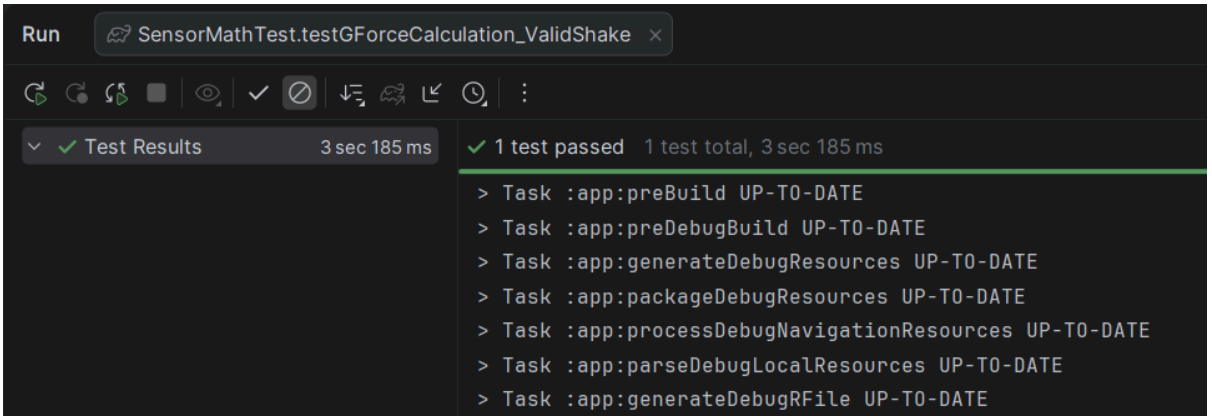


Figura 30- Confirmação de sucesso do teste do Hardware Matemático

11.1.3. Teste do mapeamento do DTO e da permutação das respostas

O teste verifica a conversão de uma pergunta recebida no formato JSON para um objeto Question do domínio. Para isso, usa o Gson para preencher o DTO QuestionJson a partir de um JSON de exemplo e chama a mesma função mapQuestion() utilizada pelo RetrofitQuestionRepository.

A validação não depende da ordem final das respostas, porque essa ordem é aleatória. Em vez disso, confirma que todas as opções esperadas continuam presentes exatamente uma vez. O teste cobre perguntas de escolha múltipla, verdadeiro/falso e o caso em que a lista de respostas incorretas é nula. Não é necessário contactar a OpenTDB para executar estes testes.

Código de produção chamado pelo teste

Excerto do ciclo de getQuestions():

```
1 | for (OpenTDBresponse.QuestionJson Json :
resposta.body().results) {
2 |     Question perguntaLimpa =
mapQuestion(category, type, Json);
3 |     perguntasProntas.add(perguntaLimpa);
4 | }
```

Função de conversão e baralhamento:

```
1 | static Question mapQuestion(
2 |         Category category,
3 |         QuestionType type,
4 |         OpenTDBresponse.QuestionJson Json
5 | ) {
6 |     List<String> respostasMisturadas =
```

```
new ArrayList<>();
7 |
respostasMisturadas.add(Json.correctAnswer);
8 |
9 |     if (Json.incorrectAnswers != null) {
10 |
respostasMisturadas.addAll(Json.incorrectAnsw
ers);
11 |     }
12 |
13 |
Collections.shuffle(respostasMisturadas);
14 |
15 |     return new Question(
16 |             category,
17 |             type,
18 |             Json.question,
19 |             Json.correctAnswer,
20 |             respostasMisturadas
21 |     );
22 | }
```

Código de teste — `RetrofitQuestionRepositoryTest`:

Os números à esquerda servem para identificar as linhas no relatório; remove-os se copiares o código para um ficheiro Java.

```
1 | package com.dicemania.data.network;
2 |
3 | import
com.dicemania.domain.model.Category;
4 | import
```

```
com.dicemania.domain.model.Question;
5 | import
com.dicemania.domain.model.QuestionType;
6 | import com.google.gson.Gson;
7 |
8 | import org.junit.Test;
9 |
10 | import java.util.Collections;
11 | import java.util.List;
12 |
13 | import static
org.junit.Assert.assertEquals;
14 |
15 | public class
RetrofitQuestionRepositoryTest {
16 |
17 |     private final Gson gson = new
Gson();
18 |
19 |     @Test
20 |     public void
mapsMultipleChoiceDtoAndPreservesAllFourOptio
ns() {
21 |         String json = "{"
22 |                 + ""question":"What is 2
+ 2?","
23 |                 +
""correct_answer":"4","
24 |                 +
""incorrect_answers":[\"1\",\"2\",\"3\"]"
25 |                 + "}";
26 |
```

```
27 |         OpenTDBresponse.QuestionJson dto
= gson.fromJson(
28 |                 json,
29 |
OpenTDBresponse.QuestionJson.class
30 |         );
31 |
32 |         Question pergunta =
RetrofitQuestionRepository.mapQuestion(
33 |                 Category.ART,
34 |
QuestionType.MULTIPLE_CHOICE,
35 |                 dto
36 |         );
37 |
38 |         assertEquals(Category.ART,
pergunta.getCategory());
39 |
assertEquals(QuestionType.MULTIPLE_CHOICE,
pergunta.getType());
40 |         assertEquals("What is 2 + 2?",
pergunta.getText());
41 |         assertEquals("4",
pergunta.getCorrectAnswer());
42 |
43 |         List<String> respostasMisturadas
= pergunta.getRespostasMisturadas();
44 |         assertEquals(4,
respostasMisturadas.size());
45 |
assertOccursOnce(respostasMisturadas, "4");
46 |
```

```
assertOccursOnce(respostasMisturadas, "1");
47 |
assertOccursOnce(respostasMisturadas, "2");
48 |
assertOccursOnce(respostasMisturadas, "3");
49 |     }
50 |
51 |     @Test
52 |     public void
mapsTrueFalseDtoAndKeepsBothOptions() {
53 |         String json = "{"
54 |                 + ""question":"The sky
can appear blue?","
55 |                 +
""correct_answer":"True","
56 |                 +
""incorrect_answers":[\"False\"]"
57 |                 + "}";
58 |
59 |         OpenTDBresponse.QuestionJson dto
= gson.fromJson(
60 |                 json,
61 |
OpenTDBresponse.QuestionJson.class
62 |         );
63 |
64 |         Question pergunta =
RetrofitQuestionRepository.mapQuestion(
65 |                 Category.SCIENCE,
66 |                 QuestionType.TRUE_FALSE,
67 |                 dto
68 |         );
```

```
69 |
70 |         List<String> respostasMisturadas
= pergunta.getRespostasMisturadas();
71 |         assertEquals(2,
respostasMisturadas.size());
72 |         assertEquals("True",
pergunta.getCorrectAnswer());
73 |
assertOccursOnce(respostasMisturadas,
"True");
74 |
assertOccursOnce(respostasMisturadas,
"False");
75 |     }
76 |
77 |     @Test
78 |     public void
mapsDtoWhenIncorrectAnswersAreNull() {
79 |         String json = "{"
80 |                 + ""question":"Question
without incorrect options","
81 |                 +
""correct_answer":"Correct","
82 |                 +
""incorrect_answers":null"
83 |                 + "}";
84 |
85 |         OpenTDBresponse.QuestionJson dto
= gson.fromJson(
86 |                 json,
87 |
OpenTDBresponse.QuestionJson.class
```

```
88 |         );
89 |
90 |         Question pergunta =
RetrofitQuestionRepository.mapQuestion(
91 |                 Category.HISTORY,
92 |
QuestionType.MULTIPLE_CHOICE,
93 |                 dto
94 |         );
95 |
96 |         assertEquals(1,
pergunta.getRespostasMisturadas().size());
97 |         assertEquals(
98 |
Collections.singletonList("Correct"),
99 |
pergunta.getRespostasMisturadas()
100 |         );
101 |     }
102 |
103 |     private void assertOccursOnce(
104 |             List<String> respostas,
105 |             String respostaEsperada
106 |     ) {
107 |         assertEquals(1,
Collections.frequency(respostas,
respostaEsperada));
108 |     }
109 | }
```

Análise da implementação e dos testes

- getQuestions() percorre os resultados da OpenTDB e chama mapQuestion() para converter cada DTO.
- mapQuestion() reúne a resposta correta e as respostas incorretas, verifica se a lista incorreta existe, baralha as opções e cria um objeto Question.
- O teste de escolha múltipla confirma que existem quatro opções e que cada resposta esperada aparece exatamente uma vez.
- O teste verdadeiro/falso confirma que são mantidas as duas opções esperadas.
- O terceiro teste verifica que uma lista de respostas incorretas nula não provoca uma exceção.
- Como Collections.shuffle() é aleatório, os testes confirmam a presença e a quantidade das opções, sem exigir uma ordem específica.

Confirmaçao:

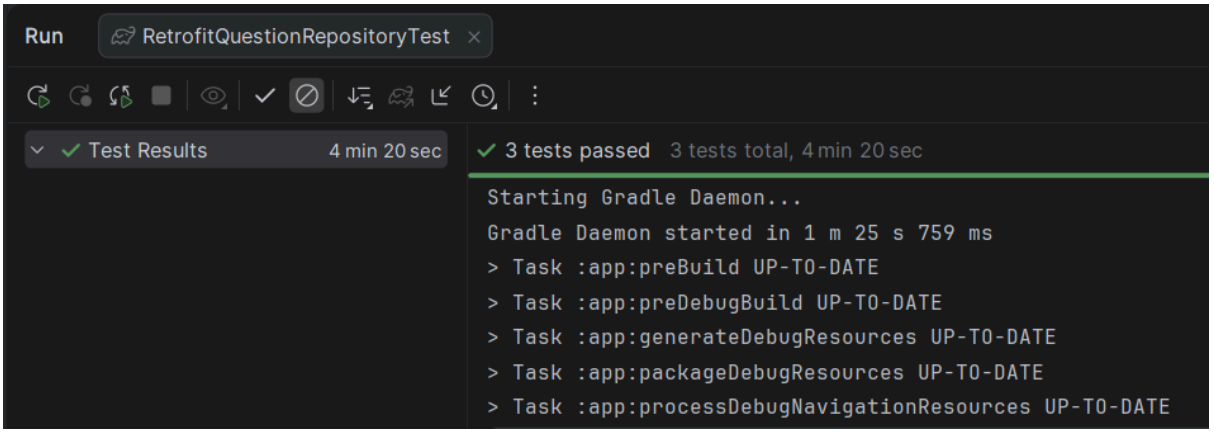


Figura 31- Confirmação de sucesso do teste de DTOs e Permutação

11.2. Testes Funcionais / End-to-End (E2E Tests)

Os testes funcionais validaram a orquestração do sistema como um todo (da UI à Base de Dados), mapeando o estado de aprovação perante os Casos de Uso originais.

Caso de Uso	Fluxo Testado	Resultado Observado	Estado de Validação
UC01 - Registar	Abrir a aplicação sem um jogador guardado e inserir um nickname válido. Depois, fechar e voltar a abrir a aplicação.	A aplicação valida o nickname, gera localmente um ID e guarda o ID e o nickname no `SharedPreferences`. Na abertura seguinte, a `MainActivity` recupera o jogador e encaminha-o para o ecrã Home, sem voltar a pedir o nickname.	Aprovado
UC02 - Jogar Partida	Iniciar uma partida, abanar os dados e responder às perguntas até ao fim.	A partida percorre 15 perguntas, distribuídas por blocos de três. O fluxo passa por `WAITING_FOR_ROLL`, `LOADING_BLOCK`, `IN_BREAK` e `PLAYING_QUESTION`, regressando à espera de um novo lançamento entre blocos. A sessão termina em `FINISHED`. O pedido à OpenTDB corre numa	Aprovado

		thread de trabalho, sem bloquear a interface.	
UC03 - Leaderboard	Abrir o leaderboard sem ligação à Internet; repetir também com uma resposta HTTP de erro do servidor	Perante uma falha de ligação ou timeout, o Retrofit chama `onFailure`; perante uma resposta HTTP não sucedida, o erro é tratado em `onResponse`. Em ambos os casos, a aplicação apresenta uma mensagem de erro e permite tentar novamente, sem encerrar inesperadamente.	Aprovado
UC04 - Debug Mode	Tocar cinco vezes seguidas no logótipo, iniciar uma partida de teste e, depois, tocar mais cinco vezes para desativar o modo.	O modo Deus alterna após cada sequência válida de cinco toques e apresenta a mensagem correspondente. A partida de teste termina após três perguntas e apresenta valores artificiais de pontuação e streak. Esse resultado não é guardado como recorde local nem	Aprovado

		enviado como resultado normal para o leaderboard.	
UC05 - Consultar Perfil e Estatísticas	Com um jogador já registado, abrir o painel Settings a partir do ecrã Home.	O painel apresenta o nickname do jogador e consulta no PlayerStatsRepository a pontuação máxima e o streak máximo associados ao ID desse jogador. Se ainda não existirem recordes guardados, apresenta os valores iniciais (0)	Aprovado

Tabela 6 - Testes Funcionais

11.2.1. UC01:

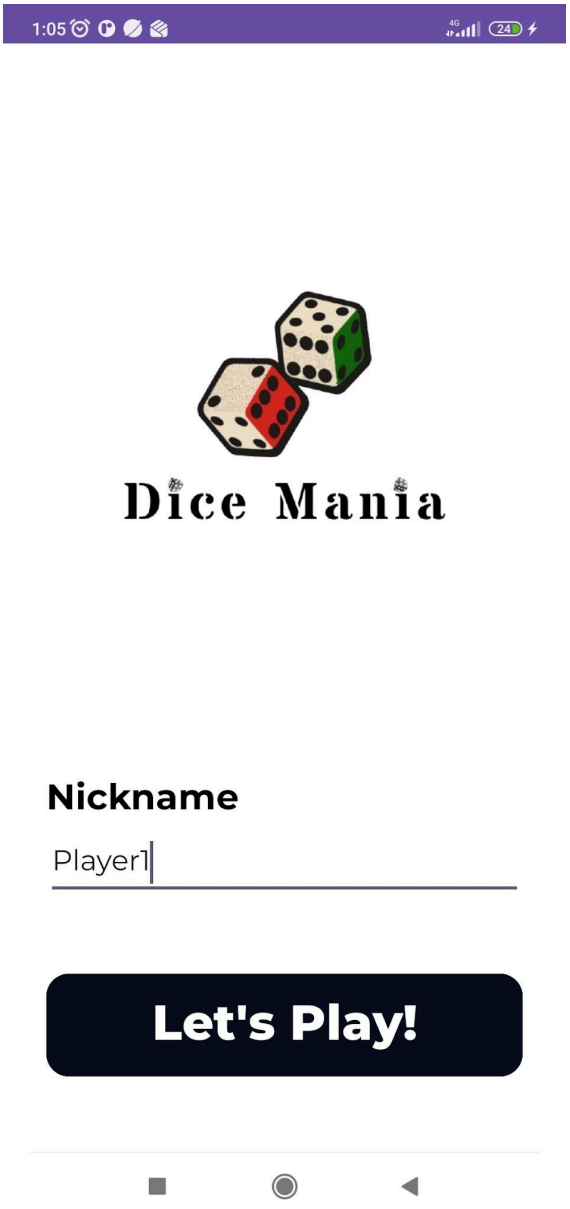


Figura 23 - Ecrã de Log in

- Temos o ecrã inicial, o de log in

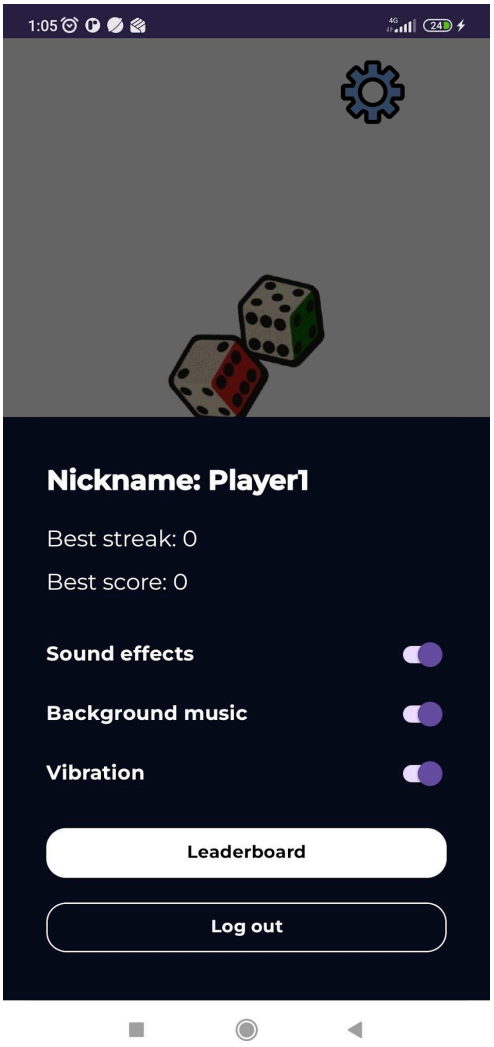


Figura 32 - Ecrã de Definições

- Ao carregarmos em “Let’s Play” aparecemos no menu dos jogadores registados e ao carregarmos no botão de definições, podemos ver as nossas informações novas: streak, pontuação, efeitos de som e vibração, Leaderboard e o Log out.

11.2.2. UC02:

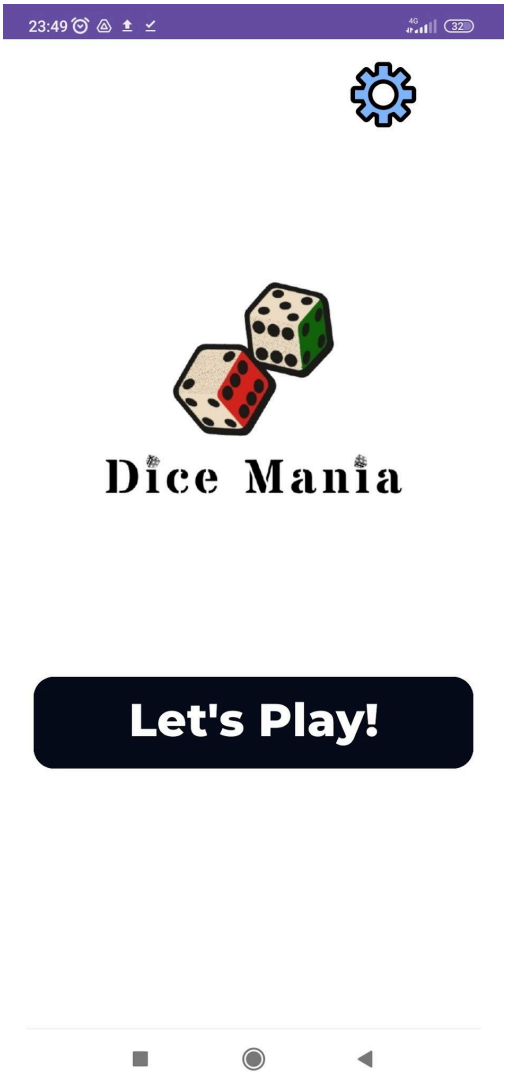


Figura 23 - Ecrã Home

- Vemos o ecrã principal quando se faz login (Home).

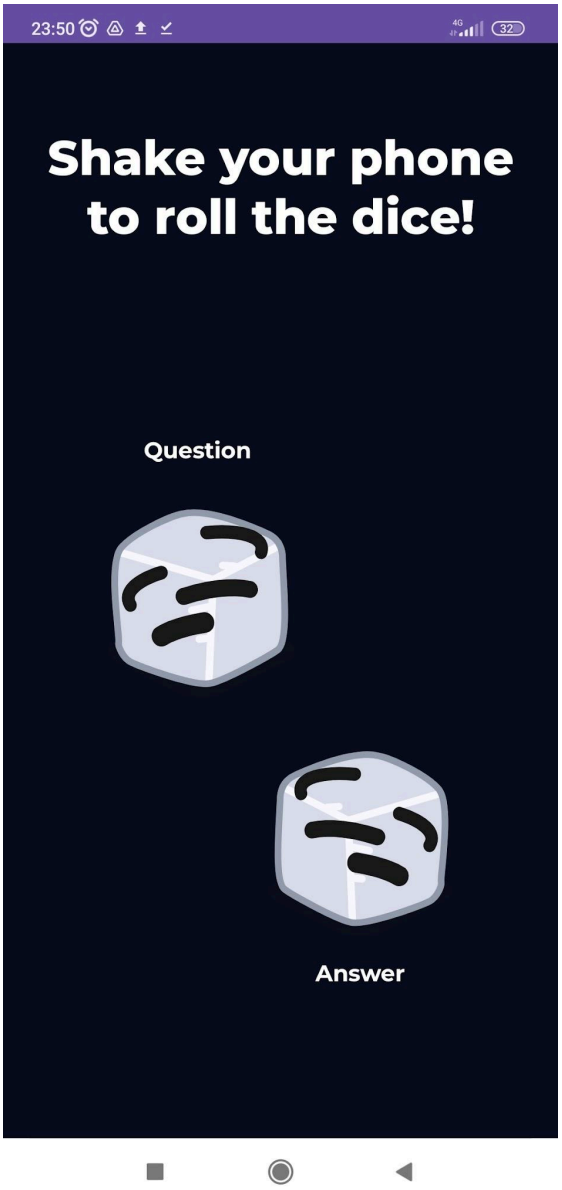


Figura 25 - Ecrã de Lançamento dos Dados

- Carregou-se em “Let’s Play!” e aparece o ecrã de Lançamento dos Dados.

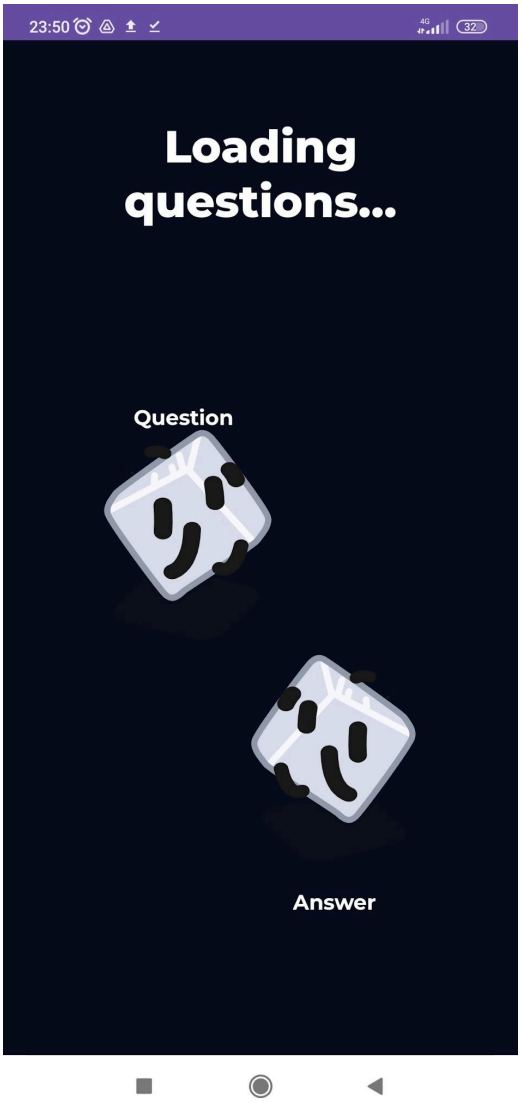


Figura 33 - Ecrã de Lançamento de Dados (Animação)

- Abanou-se o telemóvel e os dados começaram a rolar sendo feito o pedido à OpenTDB enquanto ocorre a animação.

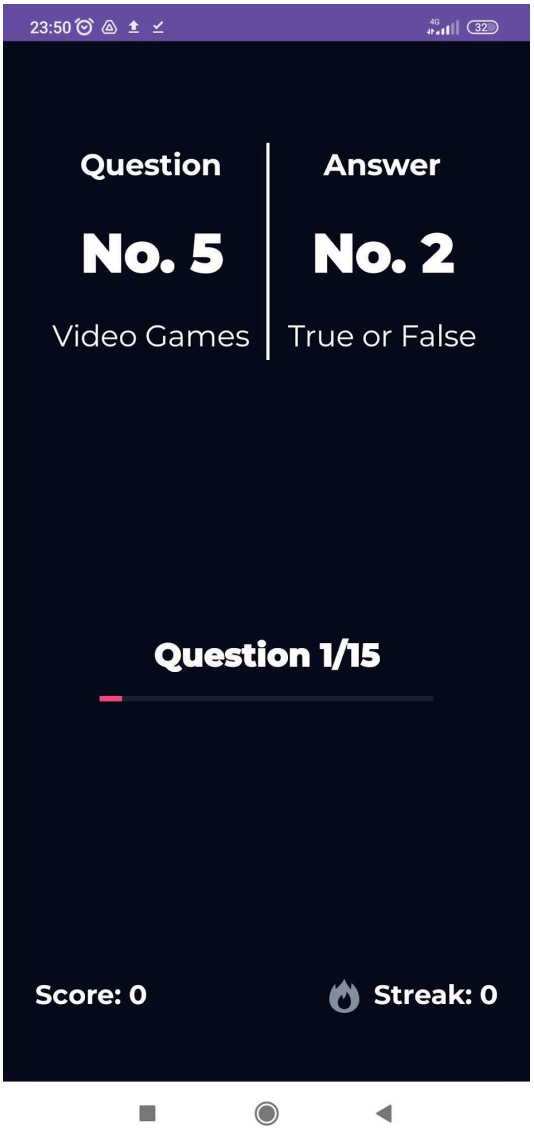


Figura 26 - Ecrã de Resumo

- Entramos no ecrã de Resumo da partida. Vemos o tipo de pergunta e resposta escolhidos pelo lançamento do dado, a pergunta atual e o máximo de perguntas a responder, a pontuação e streak do jogador.

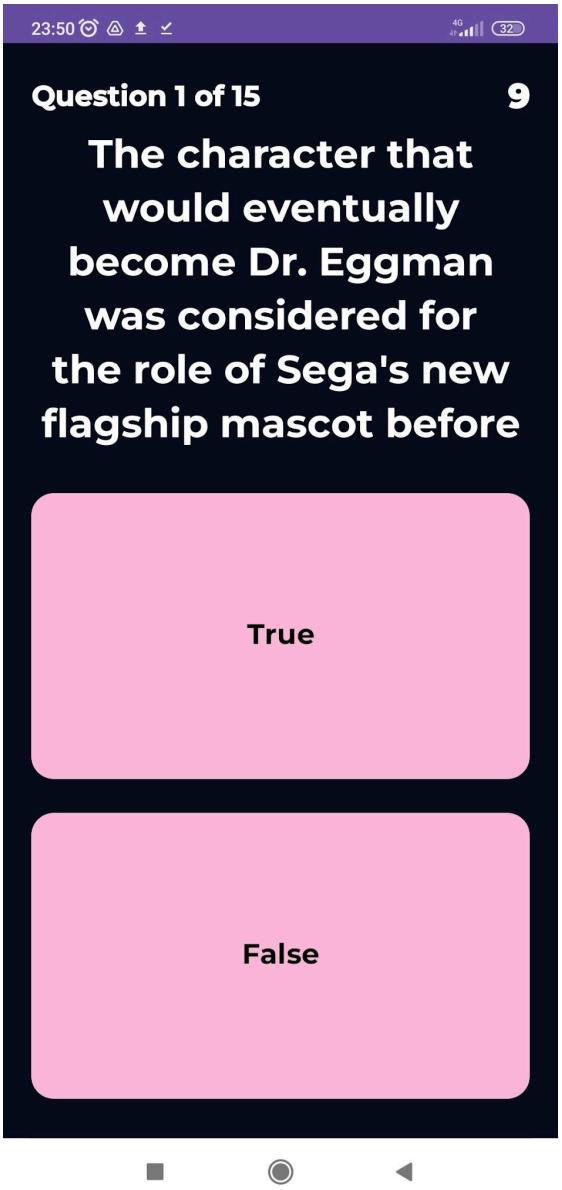


Figura 27 - Ecrã de Pergunta

- Passamos de seguida ao ecrã de Pergunta, sendo aqui a resolução do jogo. O jogador responderá as questões para poder avançar até responder 15 questões.

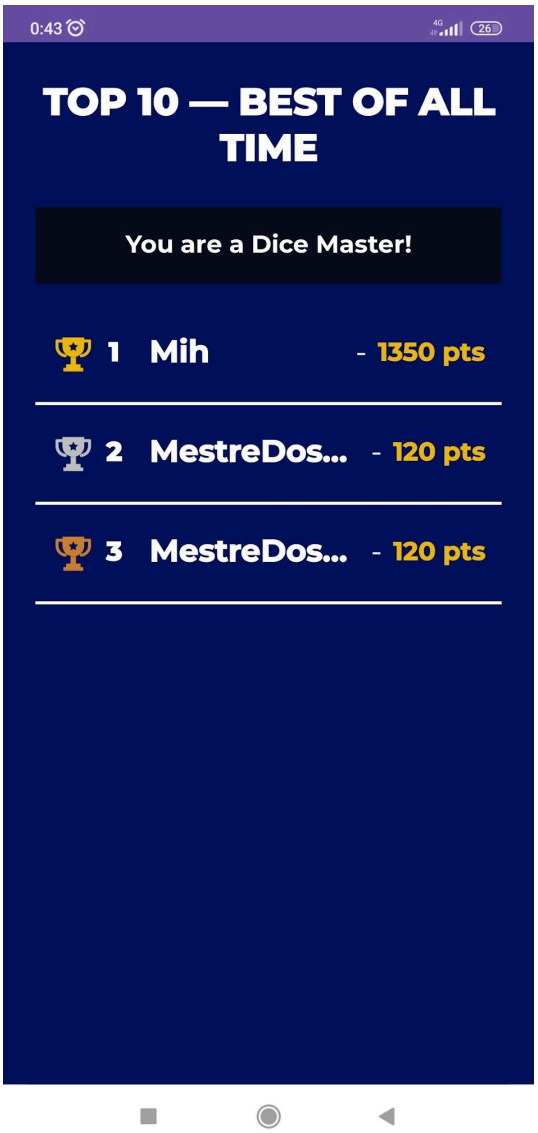


Figura 28 - Ecrã de Leaderboard

- No final das 15 questões, aparecerá o ecrã de Leaderboard e caso o jogador entre no Top 10 aparecerá a frase: “You are a Dice Master!”, caso contrário: “Keep it up! You have to roll the Dices better”

11.2.3. UC03:

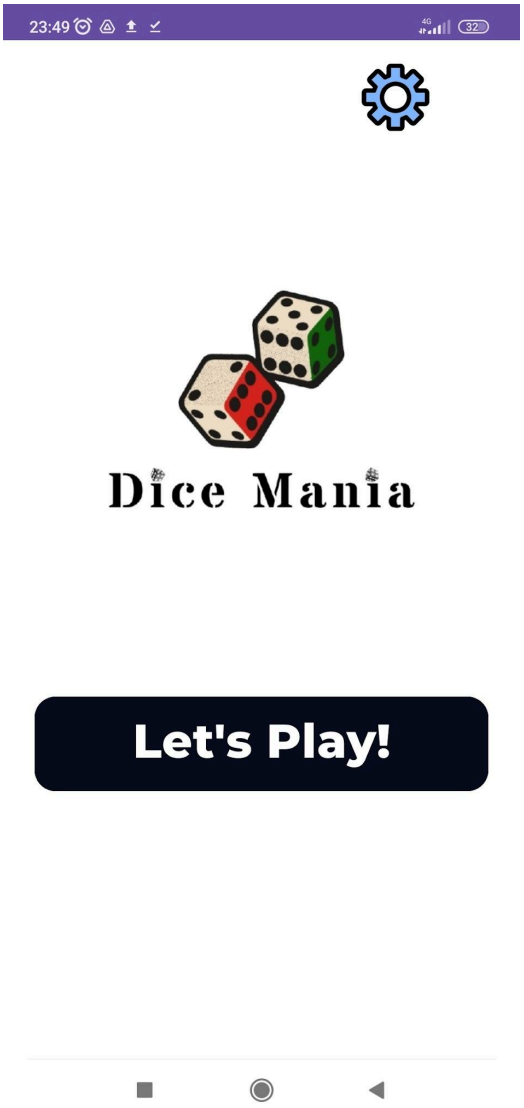


Figura 24 - Ecrã Home

- Estamos no ecrã principal quando se faz login (Home).

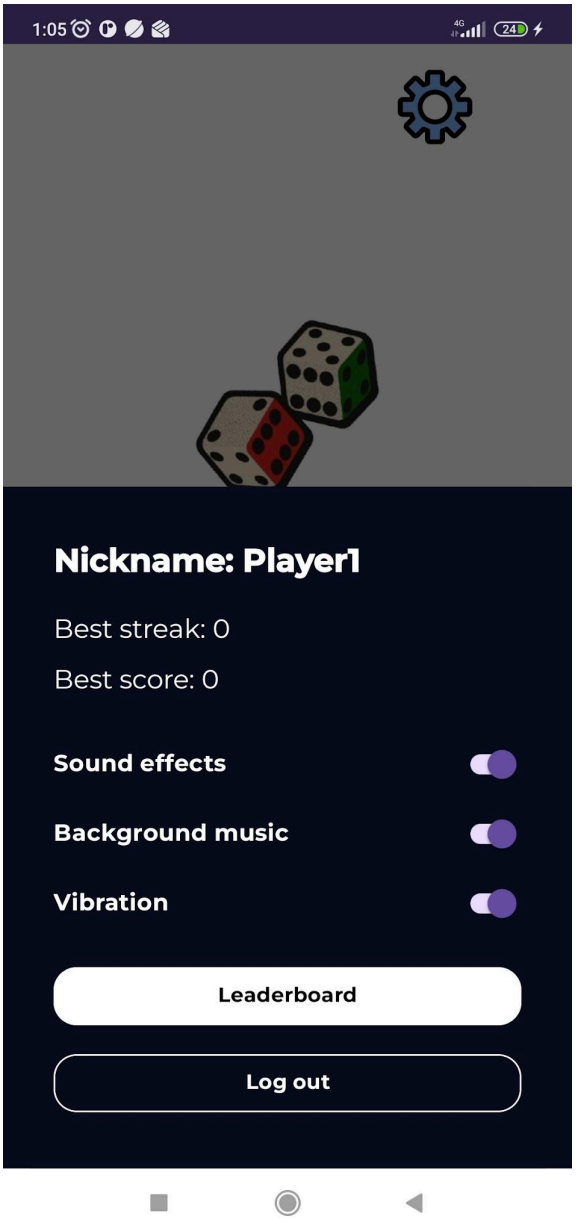


Figura 32 - Ecrã de Definições

- Carregamos nas definições e vai ser aberta esta meia janela, em que aparecem atributos do jogador, efeitos de som e vibração, o Leaderboard e o Log out.

11.2.4. UC04:

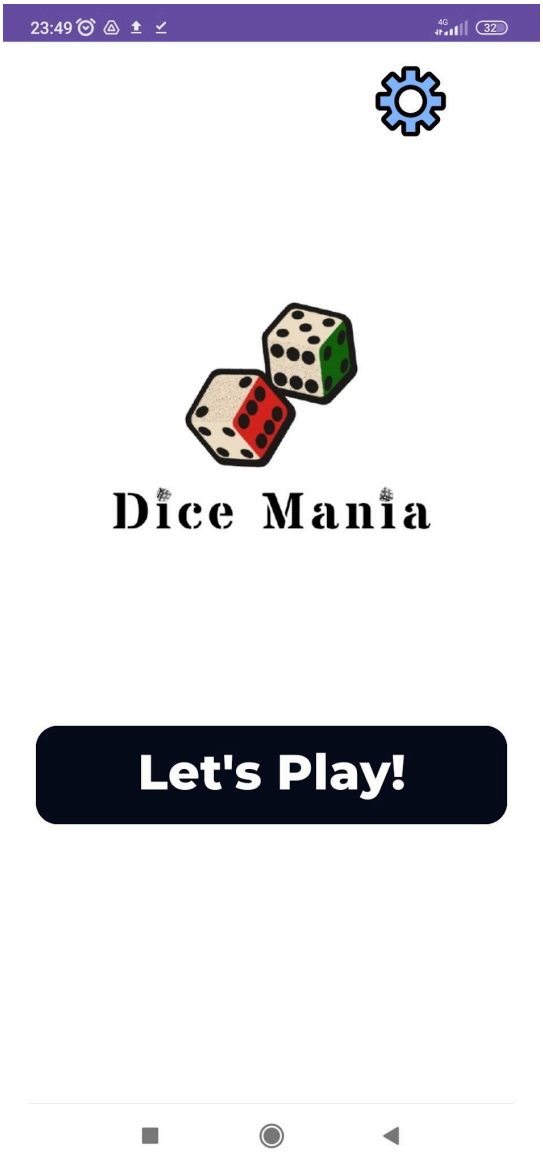


Figura 24 - Ecrã Home

- Vemos o ecrã principal quando se faz login (Home).

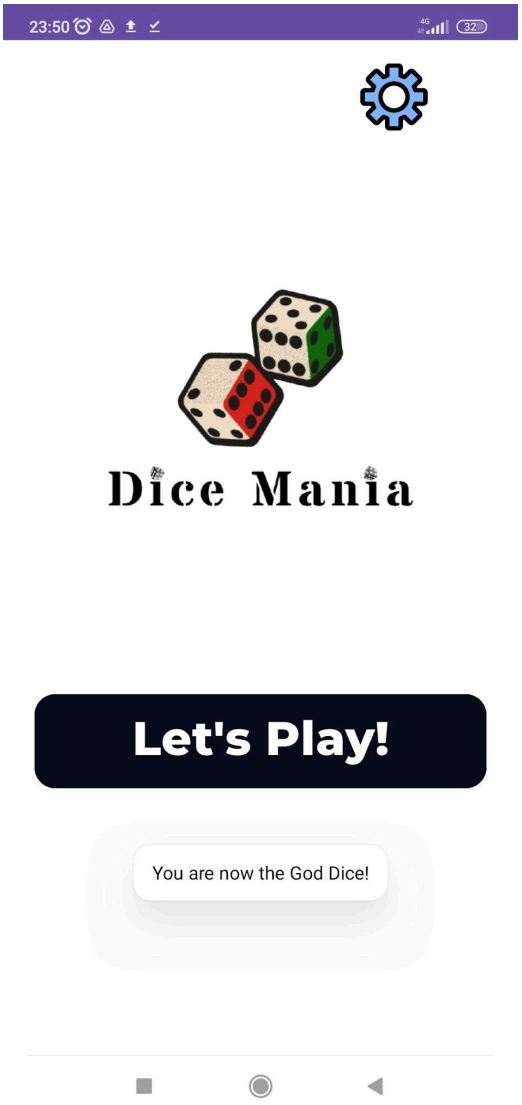


Figura 34 - Ecrã Home (Com DebugMode)

- Realizamos 5 toques seguidos na imagem dos dados e entramos no DebugMode (Modo Deus)

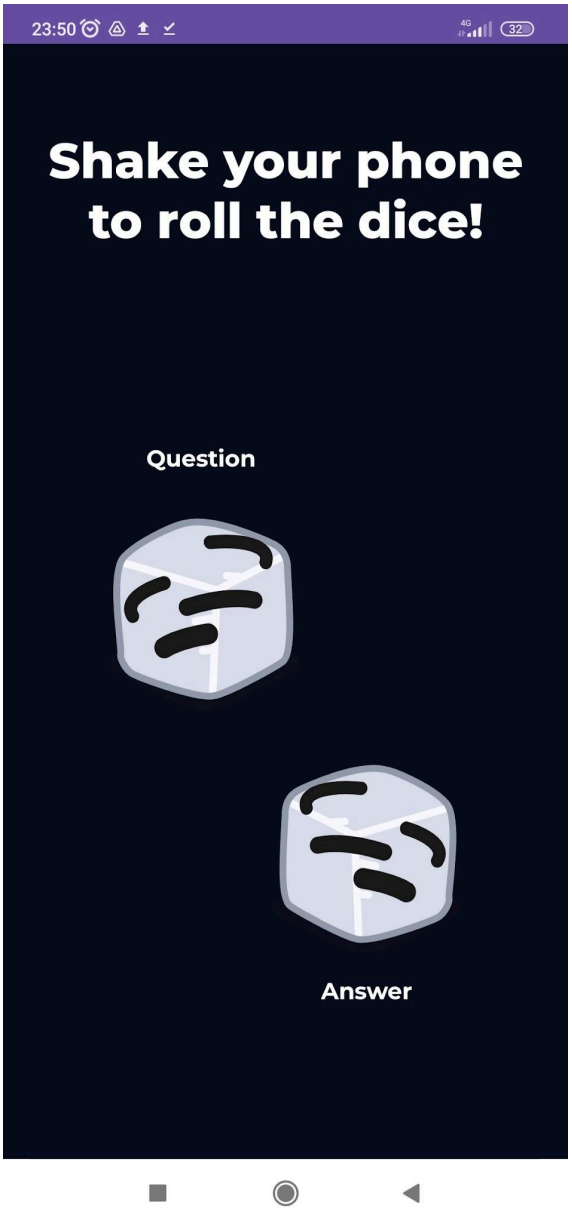


Figura 25 - Ecrã de Lançamento dos Dados

- Ao carregar em “Let’s Play”, passamos ao ecrã de Lançamento dos Dados

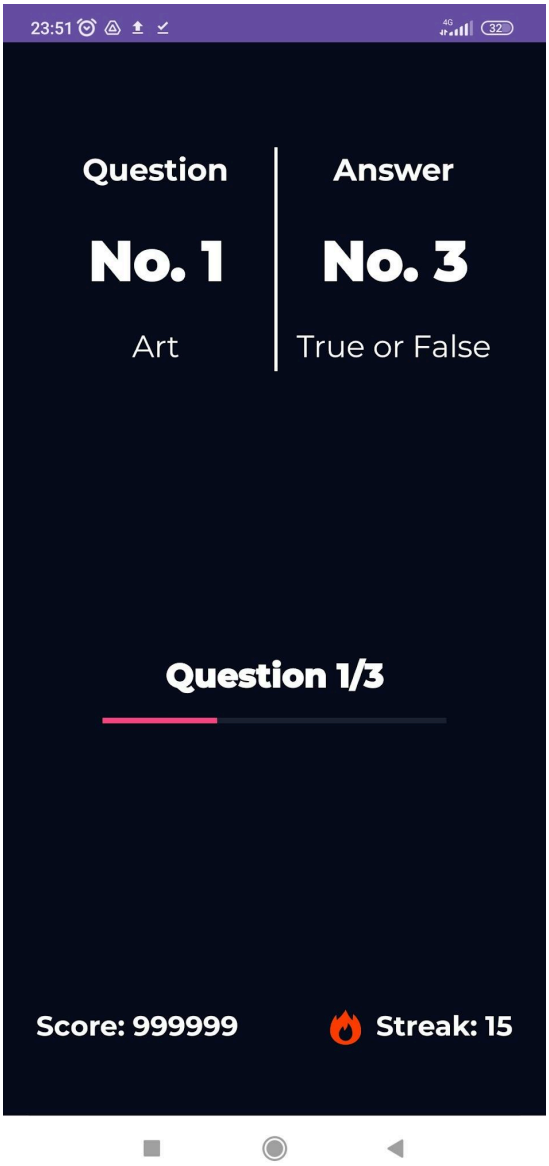


Figura 35 - Ecrã de Resumo (Com DebugMode)

- Passamos ao ecrã de resumo, em que podemos ver todas as propriedades do DebugMode: Pontuação e Streak máximos e só um bloco de perguntas (3 questões) para facilitar a amostragem do ciclo de uma partida.

11.2.5. UC05:

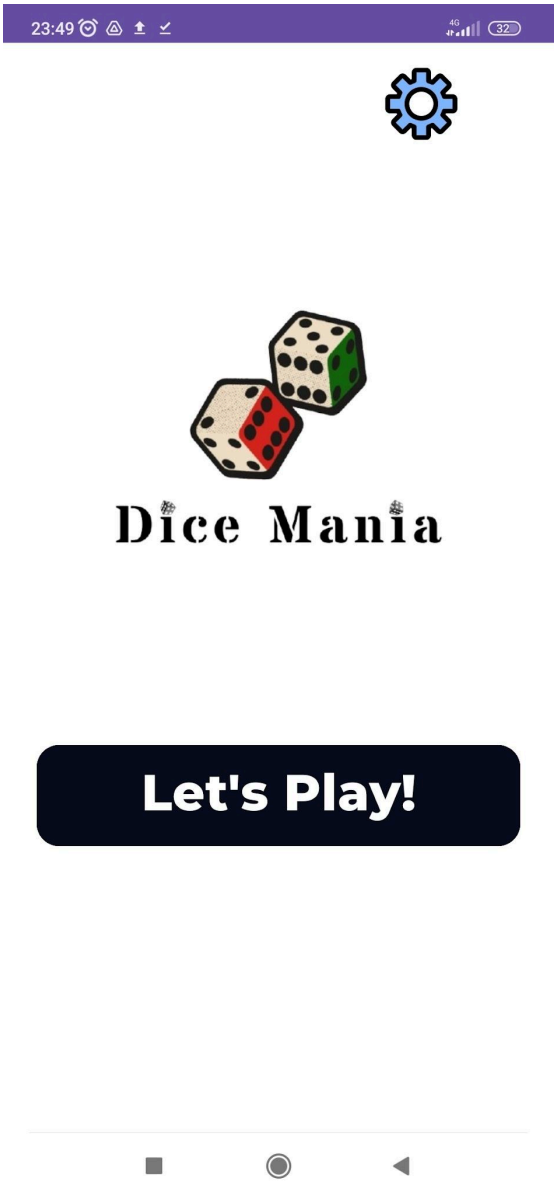


Figura 22 - Ecrã Home

- Vemos o ecrã principal quando se faz login (Home).

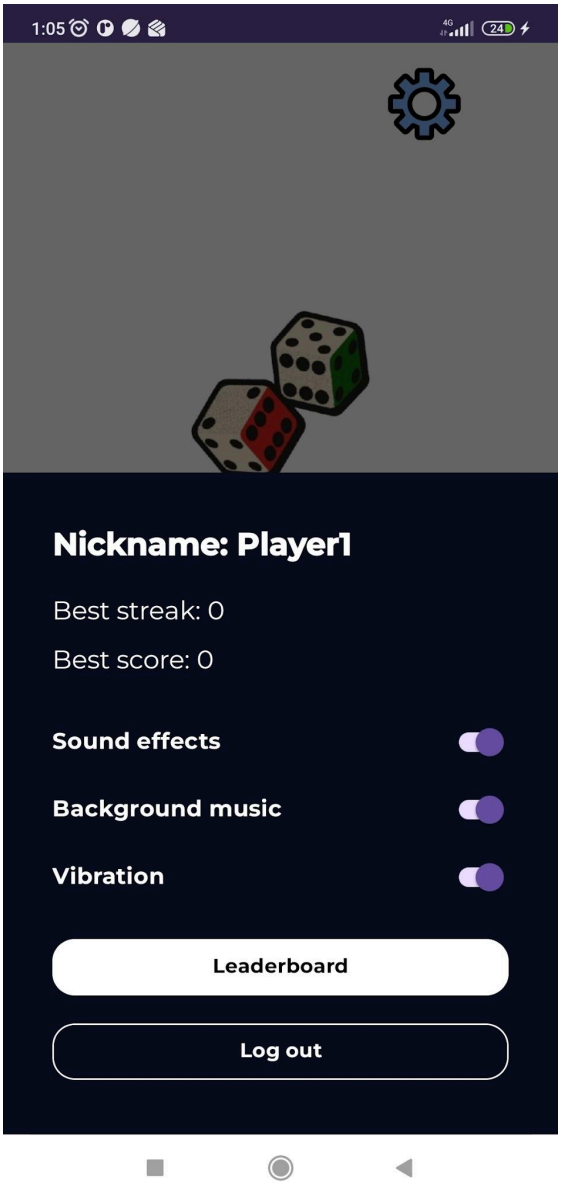


Figura 32 - Ecrã de Definições

- Carregamos nas definições e vai ser aberta esta meia janela, em que aparecem atributos do jogador, efeitos de som e vibração, o Leaderboard e o Log out.

12. Conclusão

O presente relatório documenta a conceção, o desenho arquitetural e a implementação técnica da aplicação móvel DiceMania, desenvolvida no contexto da unidade curricular de Desenvolvimento de Software e Sistemas Móveis (DSSMV). O principal objetivo do projeto envolve a criação de um jogo interativo de conhecimentos gerais que simultaneamente utilize a componente de software e hardware do dispositivo, o que foi plenamente atingido.

12.1. Estado do Sistema

No que concerne ao estado operacional, a aplicação encontra-se totalmente funcional e cumpre com rigor os requisitos funcionais e não-funcionais estipulados. O sistema foi edificado sobre o paradigma de Clean Architecture, e o resultado prático provou a eficácia deste modelo:

- A leitura de hardware (via SensorManager) deteta com precisão a ação mecânica de agitar o telemóvel,atuando como o gatilho central do jogo.

- A integração com a OpenTDB API (através do Retrofit) permite o consumo dinâmico de blocos de perguntas, sendo o tratamento de DTOs e a aleatoriedade das opções processados de forma limpa.

- A persistência de dados local (SharedPreferences) garante uma retenção contínua da sessão do utilizador, enquanto a persistência remota (Supabase) assegura a submissão correta e a consulta assíncrona do Leaderboard global.

- Todos os fluxos detalhados nos Casos de Uso (UC01 a UC05) ocorrem sem bloqueios na interface (graças ao uso de threads secundárias) e com total integridade de dados.

12.2. Desafios e Dificuldades Técnicas (Difficulties)

- Apesar do desfecho positivo, o ciclo de desenvolvimento foi pautado por desafios de engenharia complexos que exigiram pesquisa, refactorização e soluções algorítmicas robustas:
- Calibração e Histerese do Hardware (Sensores): Um dos primeiros obstáculos foi o tratamento do ruído gerado pelo Acelerómetro. A simples leitura dos eixos (X, Y, Z) gerava múltiplos eventos simultâneos, o que causava sobrecarga de pedidos à rede. A solução exigiu a implementação de um cálculo matemático vetorial (para isolar a Força G acima de 2.7) aliada a um mecanismo temporal de debounce (uma janela de slop time de 500ms), impedindo leituras repetidas e poupando a bateria do dispositivo.

- Assincronismo e Ciclo de Vida do Android: Garantir que as chamadas HTTP lentas ao Supabase e à OpenTDB não bloqueavam a Main Thread da UI revelou-se um desafio arquitetural. A transição para chamadas assíncronas (usando o método .enqueue()) resolveu o congelamento do ecrã, mas levantou problemas de sincronização (ex: o jogador submeter uma resposta antes de os dados estarem totalmente carregados). Isto obrigou à criação de uma "Máquina de Estados" sólida na GameSession (ex:WAITING_FOR_ROLL, IN_BREAK), blindando a interação do utilizador durante períodos críticos.

- Segurança e Prevenção de Batota (Gestão de Estado): O requisito de temporização (10 segundos por resposta) levantou uma falha inicial grave: a suspensão do temporizador caso a aplicação fosse minimizada (background). Para resolver esta vulnerabilidade, abandonou-se a contagem regressiva da interface gráfica em prol de um AntiCheatTimer alojado na camada de Domínio, que compara instantes absolutos do relógio do sistema (timestamps), invalidando as respostas de forma implacável sempre que as regras são quebradas.

- Manutenção do Desacoplamento: A tentação técnica de instanciar classes do Android (como Contextou SharedPreferences) diretamente na lógica do jogo foi um desafio constante. Exigiu grande disciplina arquitetural o uso contínuo de Injeção de Dependências e de Interfaces/Contratos (Repositories),garantindo que o Domínio se mantinha agnóstico e testável.

12.3. Considerações Finais

O desenvolvimento do DiceMania revelou-se uma experiência de grande maturidade técnica. O projeto transcendeu a simples escrita de código, exigindo a produção de artefactos de design profundos (Modelos UML, Diagramas de Sequência e de Componentes) que serviram como um verdadeiro guião de implementação.

O produto final não é apenas uma aplicação de entretenimento funcional, mas sim um sistema tolerante a falhas, escalável e arquiteturalmente limpo, atestando a aplicabilidade prática de todas as metodologias de Engenharia de Software lecionadas.

13. Glossário

- Acelerómetro (Accelerometer): Sensor eletromecânico presente no dispositivo móvel que mede as forças de aceleração estáticas (gravidade) e dinâmicas (vibração ou movimento), utilizado para detetar o agitar do telemóvel.
- API (Application Programming Interface): Conjunto de rotinas e padrões de programação que permite a comunicação entre a aplicação móvel e serviços externos (ex: OpenTDB e Supabase).
- Background Thread: Linha de execução secundária do processador, utilizada para realizar operações lentas (como chamadas de rede ou leituras de base de dados) sem bloquear a interface de utilizador.
- Callback: Função ou método passado como argumento para outro código, concebido para ser invocado de forma assíncrona quando uma determinada operação (ex: resposta HTTP ou leitura de sensor) é concluída.
- Caso de Uso (Use Case): Especificação de uma interação entre um ator (utilizador ou sistema) e a aplicação, representando um requisito funcional (ex: "Jogar Partida" ou "Registar Utilizador").
- Clean Architecture (Arquitetura Limpa): Padrão de desenho de software que promove a separação estrita do código em camadas (Apresentação, Domínio e Infraestrutura), garantindo que as regras de negócio não dependem de tecnologias externas.
- Cold Start (Arranque a Frio): O processo de arranque de uma aplicação móvel a partir do zero, quando o sistema operativo precisa de carregar todos os processos para a memória (RAM).
- CRUD (Create, Read, Update, Delete): Acrónimo que representa as quatro operações básicas de gestão de dados num sistema de informação persistente.
- Debounce / Histerese: Técnica de engenharia utilizada no processamento de sinais de hardware para ignorar leituras repetidas

num curto espaço de tempo (ex: a janela de 500ms entre abanões), evitando sobrecargas de eventos.

- Domínio (Domain): A camada mais interna e protegida da arquitetura, que contém os modelos e as lógicas de negócio puras (ex: cálculo de pontuação, verificação temporal) de forma totalmente agnóstica a frameworks.
- DTO (Data Transfer Object): Padrão de desenho estrutural (objeto simples) utilizado exclusivamente para transportar e serializar dados entre camadas ou redes (ex: converter o JSON sujo antes de criar uma Entidade limpa).
- Easter Egg: Funcionalidade oculta e não documentada publicamente na interface de utilizador (neste projeto, o padrão de 5 toques para aceder ao modo de depuração).
- G-Force (Força G): Medida de aceleração vetorial sentida pelo dispositivo móvel, calculada através dos três eixos (X, Y, Z) do acelerómetro.
- Injeção de Dependências (Dependency Injection): Padrão de desenho onde uma classe (ex: GameSession) recebe as instâncias de que necessita (ex: Repositórios ou Sensores) através do seu construtor, em vez de as instanciar internamente, facilitando testes e modularidade.
- JSON (JavaScript Object Notation): Formato universal, estruturado e leve de formatação e troca de dados, utilizado nas transações HTTP entre a aplicação Android e os servidores externos.
- Leaderboard: Tabela classificativa global, persistida remotamente, que agrega e ordena as pontuações máximas dos jogadores.
- Main Thread (UI Thread): A linha de execução principal do sistema Android, responsável exclusiva e unicamente pela atualização dos elementos gráficos no ecrã.
- Mock-up: Modelo ou protótipo visual estático da Interface de Utilizador, utilizado nas fases de desenho para validar a usabilidade (UI/UX) antes da programação.
- OpenTDB (Open Trivia Database): API pública e gratuita utilizada como fornecedora externa das perguntas categorizadas para as sessões de jogo.

- Repositório (Repository Pattern): Padrão de desenho que atua como uma interface de abstração entre a camada de Domínio e as fontes de dados (bases de dados, sensores ou rede), ocultando a complexidade da origem da informação.
- REST (Representational State Transfer): Padrão de arquitetura de rede para sistemas distribuídos que define um conjunto de restrições e propriedades para a criação de serviços web (Web Services).
- Retrofit: Framework (biblioteca) Open Source para Android e Java, baseada em anotações, que simplifica e agiliza a conversão e o consumo de serviços REST HTTP.
- RNF (Requisito Não Funcional): Critério que especifica as características e restrições de qualidade, segurança e arquitetura de um sistema (ex: performance), não definindo a funcionalidade em si.
- SensorManager: API nativa do framework Android dedicada à gestão, inicialização e escuta de eventos provenientes do hardware inercial do dispositivo.
- SharedPreferences: Interface nativa e assíncrona do Android, baseada no modelo chave-valor, utilizada para guardar dados primitivos locais e persistentes (como o Nickname).
- Streak: Métrica de progressão no jogo que contabiliza o número de vitórias consecutivas ininterruptas do utilizador, sendo reiniciada a zero perante um erro.
- Supabase: Plataforma remota de Backend-as-a-Service (BaaS) open-source, assente numa base de dados relacional (PostgreSQL), utilizada para persistir as métricas globais do projeto.
- UML (Unified Modeling Language): Linguagem visual padronizada utilizada na engenharia de software para especificar, construir e documentar artefactos de sistemas de informação (ex: Diagramas de Classes e Diagramas de Sequência).
- UI (User Interface): Camada interativa e visual da aplicação (ecrãs, botões, modais) com a qual o jogador comunica de forma direta.
- Activity: Componente central do sistema Android que representa um ecrã único com uma interface de utilizador (ex: GameActivity), gerindo o seu próprio ciclo de vida.

- FURPS+: Modelo de classificação de qualidade de software utilizado para categorizar os Requisitos Não Funcionais em Usabilidade, Fiabilidade, Desempenho e Suportabilidade.